Spring Boot 3

核心技术与最佳实践

周红亮 | 著

電子工業出版社
Publishing House of Electronics Industry
北京•BEIJING

内容简介

本书是一本针对Java开发人员的图书，旨在帮助Java开发人员掌握Spring Boot的基本使用，以及深入了解Spring Boot的应用及原理。

本书内容由浅入深、循序渐进，第1～5章介绍Spring Boot的基础知识（基础入门、配置管理、Starter、自动配置、启动过程与扩展应用、日志管理），第6～9章介绍Spring Boot的综合应用（Web、数据访问、计划任务、缓存、消息队列），第10～12章介绍Spring Boot应用的附加能力（调试、单元测试、打包、部署、监控、报警），全面覆盖了Spring Boot的核心知识要点。

本书涵盖了笔者多年的研究和实践经验，从中提炼出了核心知识要点，从Spring Boot的基本概念和基础实践入手，再通过大量的知识点分析及代码实践，详细介绍如何利用Spring Boot简化开发过程，提高开发效率。

未经许可，不得以任何方式复制或抄袭本书之部分或全部内容。
版权所有，侵权必究。

图书在版编目（CIP）数据

Spring Boot 3核心技术与最佳实践 / 周红亮著. —北京：电子工业出版社，2023.4
ISBN 978-7-121-45290-1

Ⅰ. ①S… Ⅱ. ①周… Ⅲ. ①JAVA语言—程序设计 Ⅳ. ①TP312.8

中国国家版本馆CIP数据核字（2023）第049830号

责任编辑：陈晓猛
印　　刷：三河市良远印务有限公司
装　　订：三河市良远印务有限公司
出版发行：电子工业出版社
　　北京市海淀区万寿路173信箱　　邮编：100036
开　　本：787×980　1/16　　印张：31.75　　字数：711.2千字
版　　次：2023年4月第1版
印　　次：2023年4月第1次印刷
定　　价：158.00元

凡所购买电子工业出版社图书有缺损问题，请向购买书店调换。若书店售缺，请与本社发行部联系，联系及邮购电话：（010）88254888，88258888。

质量投诉请发邮件至zlts@phei.com.cn，盗版侵权举报请发邮件至dbqq@phei.com.cn。

本书咨询联系方式：（010）51260888-819，faq@phei.com.cn。

前　言

写作背景

Spring 作为 Java 开发界的万能框架，曾经和 Struts2、Hibernate 组成 SSH 框架，成为 Java Web 开发领域的“三驾马车”。在 2013 年左右，Spring 又和 Spring MVC、MyBatis 组成 SSM，成为新一代 Web 开发框架全家桶，一直流行至今。

为了简化 Spring 的上手难度，Spring Boot 于 2014 年诞生，可以帮助开发者更加轻松、快捷地使用 Spring 的组件，它是 Spring、Spring MVC 等框架更上一层的框架，它需要依赖 Spring、Spring MVC 等原生框架，而不能独立存在。

笔者最初接触和学习 Spring Boot 是在 2016 年，工作之余经常和同事们聊聊最新的 Java 技术，Spring Boot 就是其中之一。那时候，Spring Boot 虽然应用还不是很广泛，但很火热，逐渐成为炙手可热的 Java 框架。

自 2016 年起，笔者有幸负责和参与公司的多个系统重构，其中就包括从 SSM 框架到以 Spring Boot 为基础的技术转型，也包括以 Spring Cloud 为基础的微服务系统的设计和研发，Spring Cloud 就是基于 Spring Boot 构建的。在多年的实际开发和架构设计工作中，笔者积累了大量 Spring Boot 的使用经验，也见证了 Spring Boot 1.x ~ 3.x 的发展历史。

虽然笔者熟练掌握了 Spring Boot，但在其不断发展的情况下，某些知识点更新了也可能浑然不知，直到在使用出错时才后知后觉。Spring Boot 至今已经发展了近十年，最新的版本已经发布到了 3.x 版本，其底层实现逻辑、使用方式发生了翻天覆地的变化，同时在 Spring Boot 各个版本接二连三地停止维护的情况下，大部分版本已经不能满足技术更新的需要了。

市面上的很多图书、博客都是基于 Spring Boot 低版本的，即使介绍了最新的 Spring Boot 技术，也不是很系统，并没有对最新的技术要点进行系统的梳理、解读及应用举例，如果还继续学习低版本的内容，就会给很多初学者带来不必要的知识更新成本，也会走很多弯路。鉴于市面上关于

Spring Boot 3.x的系统性学习资料比较匮乏，笔者撰写了本书，以帮助国内Java开发人员正确地学习、理解和使用最新的Spring Boot技术。

笔者从2017年开始，陆续写了一些关于Spring Boot的文章，本以为出版此书会相当顺利，结果远没有想象中的简单。写博客没有什么太重的思想负担，所以写得比较随意、零乱，也不成体系，与成体系的图书相差甚远，而且博客中的部分内容已经过时，所以仅能作为一个基石，笔者又花费了近一年的时间，在此基础上深度打磨、加强和完善，并新增了90%以上的内容，最后形成了本书的全部内容。

本书章节安排

本书共12章，这是一个由浅入深、循序渐进的学习过程。

第1章为Spring Boot基础入门，主要介绍Spring Boot的基础知识，包括背景介绍、核心特性、核心模块、核心思想、版本选择，以及Maven、Spring Boot CLI等相关工具的使用，还会分享简单的应用案例。

第2章为Spring Boot配置管理，主要介绍Spring Boot应用中的各种配置技巧，包括配置类、配置文件、外部化配置、配置绑定、导入配置、随机值配置、多文档配置、Profile、加载机制、配置加密、配置迁移，有助于后续章节的融会贯通。

第3章为Spring Boot Starter与自动配置，主要介绍Spring Boot Starter及其应用，包括命名规范、分类、自动配置原理及扩展机制，还介绍了邮件Starter及其应用，以及如何自定义一个Spring Boot Starter。

第4章为Spring Boot启动过程与扩展应用，主要介绍Spring Boot的启动过程，包括引导方式、启动入口方法、启动流程源码分析，以及启动过程中丰富的扩展应用，包括启动日志、启动图案、启动失败分析、启动事件和监听器、全局懒加载、启动运行器等。

第5章为Spring Boot日志管理，主要介绍Spring Boot日志的使用，包括日志格式、日志文件、日志级别、日志分组、日志归档、自定义日志配置文件、切换Log4j2日志框架、输出彩色日志等。

第6章为Spring Boot Web核心应用，这是Spring Boot项目最基本、最核心的部分，包括嵌入式容器、Web组件注册、静态资源处理、模板引擎、异常处理、参数校验、国际化、跨域、分布式会话、安全性、REST服务调用等常用的Web技术的原理介绍、集成与应用。

第7章为Spring Boot数据访问，主要介绍Spring Boot与关系型数据库及非关系型数据库的集成应用，包括数据源、连接池、事务管理、Spring Data JPA、MyBatis、MyBatis-Plus、Redis、MongoDB、Elasticsearch的原理、集成与应用。

第 8 章为 Spring Boot 计划任务，主要介绍计划任务在 Spring Boot 中的应用，包括 Spring 中的计划任务和 Quartz 计划任务的原理、集成与应用。

第 9 章为 Spring Boot 缓存与消息队列，主要介绍缓存和消息队列在 Spring Boot 中的应用，缓存机制的原理及 Redis 缓存的实现与集成，消息队列的原理及 ActiveMQ、RabbitMQ、Kafka 的原理、集成与应用。

第 10 章为 Spring Boot 调试与单元测试，主要介绍 Spring Boot 应用的调试方法、开发者工具的详细使用，以及如何在 Spring Boot 应用中做单元测试。

第 11 章为 Spring Boot 打包与部署，主要介绍 Spring Boot 应用的打包方式，以及如何将 Spring Boot 应用包以不同的方式运行、以不同的方式部署到 Linux 和 Docker 容器中，包括 Spring Boot 3.0 支持的构建 GraalVM 原生镜像的应用方式。

第 12 章为 Spring Boot 监控与报警，主要介绍 Spring Boot Actuator 监控模块及其应用，包括端点、指标，以及 Spring Boot 3.0+ 支持的可观测性技术的应用，还包括第三方监控平台 Spring Boot Admin、Prometheus+Grafana 的集成与应用。

本书特色

本书以最新的 Spring Boot 3.0 为基础，解读 Spring Boot 核心技术，包括最新的核心知识点、技术原理、应用方式与第三方主流技术集成的方法等，再到服务的测试、调试、部署和监控等，帮助读者一站式学习和掌握最新的 Spring Boot 核心技术。本书具有以下特色。

1. 全面

本书全面介绍了 Spring Boot，覆盖 Spring Boot 所有主流知识点，包括基础入门、配置管理、Starter 与自动配置、启动过程与扩展应用、日志管理、Web 核心应用、数据访问、计划任务、缓存与消息队列、调试与单元测试、打包与部署、监控与报警等。

2. 全新

本书的讲解和实战均基于Spring Boot最新主版本3.0.0，书中带有“Spring Boot 3.0新变化”“Spring Boot 3.0+”等标识，方便读者对 Spring Boot 3.0 的新特性和变更项有一个更清晰的认识，读者从 Spring Boot 2.x 迁移到 Spring Boot 3.x 也可以有一个对比和参考。

3. 实用

本书不仅介绍了 Spring Boot 的理论知识，还提供了大量的底层原理分析，并为核心知识点、

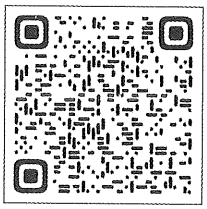
第三方主流技术的集成与应用提供了大量实战案例，理论和实践相结合，清晰易懂，使读者可以更好地吸收和理解。

4. 权威

本书以 Spring Boot 的官方文档和框架源码作为主要参考依据，包括里程碑版本和正式版本的文档、框架源码，再辅以知识点实践和验证，以确保知识点的权威性和正确性。

实战源代码

本书提供了所有实战源代码，虽然本书介绍的内容全部基于 Spring Boot 3.0.0，但是 Spring Boot 后续发布的新版本、新特性，也会进行更新和适配，读者可以“Star”、克隆、下载，学习最新的 Spring Boot 技术。



扫码关注微信公众号“Java 技术栈”，在后台回复关键字“boot”，可获取本书实战源代码。

勘误和交流

因能力、时间和精力有限，本书难免存在纰漏之处，恳请广大读者朋友批评指正，笔者会第一时间在源代码仓库 Issues 一栏中发布最新勘误。读者可以通过以下方式向笔者反馈勘误或进行技术交流。

- 博客：https://www.javastack.cn。
- 邮箱：admin@javastack.cn。
- 微信公众号：Java 技术栈（ID：javastack）。

致谢

本书能顺利出版，离不开电子工业出版社的编辑人员，特别是陈晓猛编辑，他在笔者写作过程中提供了很多建议和帮助，修正了不少错误。在此，特别感谢他们的默默付出。还要感谢笔者的家人，因为在这一年时间中，撰写此书占用了大量本该和他们共处的美好时光，导致没能好好陪伴他们，感谢家人对笔者写书的大力支持、包容和理解。

周红亮

目　录

第 1 章　Spring Boot 基础入门 1
1.1　Spring Boot 概述 1
1.1.1　诞生背景 1
1.1.2　基本介绍 2
1.1.3　核心思想 2
1.1.4　基本特性 2
1.1.5　核心模块 4
1.1.6　Spring Boot 与 Spring MVC、Spring 之间的关系 5
1.2　Spring Boot 安装集成 6
1.2.1　版本周期 6
1.2.2　支持版本 10
1.2.3　环境要求 11
1.2.4　集成方式 11
1.3　快速开发一个 Spring Boot 接口 15
1.3.1　生成项目 15
1.3.2　导入依赖 19
1.3.3　编写接口 19
1.3.4　启动应用 20
1.3.5　测试接口 21
1.4　快速使用 Maven 22
1.4.1　基本介绍 22

1.4.2 使用方式 23
1.4.3 Gradle Wrapper 24
1.5 Spring Boot CLI 24
1.5.1 概述 24
1.5.2 安装方式 24
1.5.3 命令行自动补全 27
1.5.4 快速开始 28
1.6 开启 Spring Boot 之旅 29
第 2 章 Spring Boot 配置管理 30
2.1 配置类 30
2.1.1 自定义配置类 30
2.1.2 导入配置 31
2.2 配置文件 34
2.2.1 application 34
2.2.2 bootstrap 35
2.2.3 配置文件类型 35
2.3 配置绑定 36
2.3.1 Spring 中的配置绑定 36
2.3.2 参数绑定 37
2.3.3 构造器绑定 40
2.3.4 Bean 配置绑定 43
2.3.5 参数类扫描 45
2.3.6 配置验证 45
2.4 外部化配置 46
2.4.1 配置源 46
2.4.2 配置优先级 47
2.4.3 命令行参数 48
2.5 导入配置 49
2.6 随机值配置 50
2.7 多文档配置 51
2.7.1 配置格式 51
2.7.2 激活多文档配置 52

2.8 Profile 54
2.8.1 默认的 Profile 54
2.8.2 激活 Profile 54
2.8.3 切换 Profile 56
2.8.4 Profile 分组 56
2.8.5 指定 Profile 配置文件 58
2.8.6 使用限制 59
2.9 加载机制 60
2.10 配置加密 65
2.10.1 概述 65
2.10.2 使用配置中心（支持自动解密） 65
2.10.3 使用数据库机制 66
2.10.4 使用自定义加 / 解密机制 66
2.10.5 Jasypt Spring Boot 67
2.11 配置迁移 74
2.11.1 迁移方案 74
2.11.2 实现原理 75

第 3 章 Spring Boot Starter 与自动配置 77
3.1 概述 77
3.2 Starter 的命名规范 78
3.3 Starter 的分类 79
3.3.1 application starter 79
3.3.2 production starter 81
3.3.3 technical starter 81
3.4 自动配置 82
3.4.1 概述 82
3.4.2 命名规范 83
3.4.3 自动配置文件的加载原理 85
3.4.4 自动配置原理 93
3.4.5 自动配置报告 98
3.4.6 排除自动配置 100
3.4.7 替换自动配置 103

3.5 邮件 Starter 103
3.5.1 概述 103
3.5.2 发邮件实践 105
3.6 自定义 Starter 108
3.6.1 创建 Starter 工程 108
3.6.2 创建自动配置类 108
3.6.3 注册自动配置类（spring.factories） 109
3.6.4 使用 Starter 110
3.6.5 注册自动配置类（新规范） 112

第 4 章 Spring Boot 启动过程与扩展应用 113
4.1 启动入口 113
4.1.1 应用启动类 113
4.1.2 应用启动方法 115
4.1.3 启动引导类 117
4.2 关闭启动日志 122
4.3 启动失败分析 123
4.3.1 失败分析器 123
4.3.2 自定义失败分析器 125
4.4 全局懒加载 128
4.5 启动图案 129
4.5.1 默认图案 129
4.5.2 输出模式 129
4.5.3 图案实现类 131
4.5.4 自定义图案 132
4.6 启动事件和监听器 133
4.6.1 启动事件的顺序 134
4.6.2 自定义事件监听器 135
4.7 启动运行器 136
4.7.1 概述 136
4.7.2 使用方式 138
4.8 应用启动流程 138
4.8.1 实例化流程 138
4.8.2 启动流程 148

第 5 章 Spring Boot 日志管理 166
5.1 概述 166
5.2 日志格式 167
5.3 控制台日志 168
5.4 日志文件 168
5.5 日志级别 170
5.6 日志分组 171
5.7 日志归档 172
5.8 日志配置文件 173
5.8.1 概述 173
5.8.2 日志配置模板 174
5.8.3 自定义日志配置文件 177
5.9 切换 Log4j2 日志框架 180
5.10 切换日志框架版本 180
5.11 输出彩色日志 181
5.11.1 开启彩色日志输出 181
5.11.2 日志上色原理 183
5.11.3 自定义日志颜色 185
5.12 日志关闭钩子 186

第 6 章 Spring Boot Web 核心应用 187
6.1 概述 187
6.2 嵌入式容器 189
6.2.1 概述 189
6.2.2 容器配置 189
6.2.3 切换容器 191
6.2.4 随机空闲端口 191
6.2.5 SSL 192
6.2.6 持久化 194
6.2.7 优雅关闭 194
6.3 自定义 Web 配置 196
6.4 注册拦截器 197
6.5 注册消息转换器 198

6.6 注册类型转换器 201
6.7 注册 Servlet、Filter、Listener 202
6.7.1 Spring Boot 的手动注册 202
6.7.2 组件扫描注册 203
6.7.3 动态注册 205
6.8 静态资源处理 207
6.9 模板引擎 208
6.10 异常处理 210
6.10.1 默认的异常处理 210
6.10.2 自定义全局异常 211
6.10.3 自定义异常状态码页面 212
6.11 参数校验 212
6.11.1 概述 212
6.11.2 约束注解 213
6.11.3 参数校验示例 214
6.12 国际化 216
6.12.1 概述 216
6.12.2 自动国际化 218
6.12.3 切换国际化 220
6.13 分布式会话 222
6.14 跨域 228
6.15 安全性 229
6.15.1 默认的安全机制 229
6.15.2 自定义安全机制 233
6.16 REST 服务调用 235
6.16.1 RestTemplate（Servlet） 235
6.16.2 WebClient（Reactive） 240
第 7 章 Spring Boot 数据访问 246
7.1 概述 246
7.2 嵌入式数据库 247
7.3 数据源 247
7.3.1 概述 247

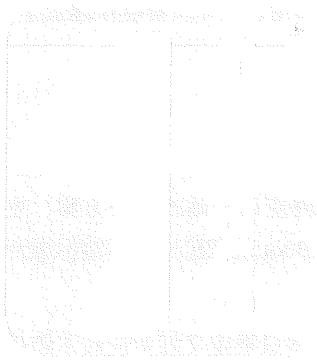
7.3.2 自定义数据源 250
7.4 连接池 251
7.4.1 概述 251
7.4.2 使用 Druid 连接池 253
7.5 数据库初始化 255
7.6 事务管理 257
7.6.1 概述 257
7.6.2 事务失效的场景 258
7.7 JdbcTemplate 262
7.7.1 数据库操作 262
7.7.2 自定义 JdbcTemplate 264
7.8 Spring Data JPA 265
7.8.1 概述 265
7.8.2 数据库操作 266
7.9 MyBatis 270
7.9.1 概述 270
7.9.2 数据库操作 271
7.10 MyBatis-Plus 274
7.10.1 概述 274
7.10.2 通用数据库操作 276
7.10.3 自定义数据库操作 279
7.11 Redis 280
7.11.1 概述 280
7.11.2 Redis 环境搭建 280
7.11.3 Spring Boot 集成 Redis 283
7.12 MongoDB 288
7.12.1 概述 288
7.12.2 MongoDB 环境搭建 289
7.12.3 Spring Boot 集成 MongoDB 291
7.13 Elasticsearch 297
7.13.1 概述 297
7.13.2 Elasticsearch 环境搭建 298
7.13.3 Spring Boot 集成 Elasticsearch 301

第 8 章 Spring Boot 计划任务 310
8.1 Spring 计划任务 310
8.1.1 概述 310
8.1.2 线程池工作流程 313
8.1.3 实现计划任务 315
8.1.4 Cron 表达式 318
8.1.5 自定义线程池 320
8.2 Quartz 计划任务 324
8.2.1 概述 324
8.2.2 实现计划任务 325
8.2.3 自定义配置 327
8.2.4 持久化任务数据 328
8.2.5 动态维护任务 331
第 9 章 Spring Boot 缓存与消息队列 333
9.1 缓存 333
9.1.1 概述 333
9.1.2 开启缓存 335
9.1.3 默认简单缓存 336
9.1.4 Redis 缓存 338
9.2 消息系统 340
9.2.1 概述 340
9.2.2 ActiveMQ 342
9.2.3 RabbitMQ 349
9.2.4 Kafka 358
第 10 章 Spring Boot 调试与单元测试 365
10.1 断点调试 365
10.1.1 使用 main 方法启动调试 365
10.1.2 使用 Maven 插件启动调试 366
10.2 开发者工具 369
10.2.1 概述 369
10.2.2 默认值 370
10.2.3 自动重启 371

10.2.4 实时重载 375
10.2.5 全局配置 378
10.3 单元测试 379
10.3.1 概述 379
10.3.2 真实环境测试 381
10.3.3 Mock 环境测试 382
10.3.4 Mock 组件测试 382
10.3.5 技术框架测试 384

第 11 章 Spring Boot 打包与部署 387
11.1 应用打包（jar） 387
11.1.1 概述 387
11.1.2 快速打包 388
11.1.3 自定义打包 391
11.2 应用打包（war） 393
11.2.1 概述 393
11.2.2 配置 war 包 393
11.2.3 开始打包 396
11.3 应用运行（嵌入式容器） 398
11.3.1 使用 java 命令运行 398
11.3.2 直接运行 398
11.3.3 系统服务运行 399
11.3.4 拆包运行 401
11.4 部署 Docker 容器 401
11.4.1 概述 401
11.4.2 Docker 环境搭建 402
11.4.3 基于 Dockerfile 构建镜像 404
11.4.4 基于 Cloud Native Buildpacks 构建镜像 412
11.5 GraalVM 原生镜像（Spring Boot 3.0+） 416
11.5.1 概述 416
11.5.2 GraalVM 应用与传统应用的区别 418
11.5.3 创建 GraalVM 原生镜像的应用 419
11.5.4 构建基于 GraalVM 的原生镜像应用 421

第 12 章 Spring Boot 监控与报警 426
12.1 Spring Boot Actuator 概述 426
12.2 Endpoints（端点） 427
12.2.1 概述 427
12.2.2 内置端点 427
12.2.3 启用端点 428
12.2.4 暴露端点 429
12.2.5 端点安全性 431
12.2.6 自定义端点映射 433
12.2.7 端点实现机制 435
12.2.8 自定义端点 437
12.3 loggers（日志端点） 441
12.4 Observability（可观测性，Spring Boot 3.0+） 444
12.5 Metrics（指标） 446
12.5.1 内置指标 446
12.5.2 自定义指标 448
12.6 Traces（链路跟踪，Spring Boot 3.0+） 450
12.6.1 概述 450
12.6.2 链路跟踪环境搭建 452
12.6.3 链路跟踪 / 展示 453
12.7 Spring Boot Admin 455
12.7.1 概述 455
12.7.2 环境搭建 456
12.7.3 监控页面 458
12.7.4 监控报警 465
12.8 Prometheus+Grafana 472
12.8.1 概述 472
12.8.2 Prometheus 指标暴露 473
12.8.3 Prometheus 环境搭建 475
12.8.4 Grafana 数据可视化 477
12.8.5 监控报警 485



第1章 Spring Boot基础入门

本章是Spring Boot基础入门章节，笔者会介绍Spring Boot的一些基础知识和应用，包括Spring Boot的诞生背景、基本介绍、安装和集成的方式，以及使用Maven的方式安装和集成Spring Boot，并快速开发一个Spring Boot接口。另外，本章也会介绍如何在Spring Boot中免安装、快速使用Maven和Gradle，以及Spring Boot CLI的使用，读者可以对Spring Boot框架有一个基本的认识，并且初步使用Spring Boot。

1.1 Spring Boot概述

1.1.1 诞生背景

在Java后端框架繁荣发展和遍地开花的今天，Spring框架无疑是最火热，也是必不可少的开源框架，更是稳坐Java后端框架的“龙头”位置，它几乎能与任何Java主流框架进行结合。

Spring能流行的原因是它的两把利器：IoC和AOP。IoC可以帮助我们管理对象的依赖关系，极大减少对象的耦合性，而AOP的切面编程功能可以使我们更方便地使用动态代理来实现各种动态方法功能，如数据库事务、缓存、日志等。

传统的Spring框架，不管是Spring旗下的组件，还是第三方框架，如果要基于Spring集成使用，就必须用到XML配置文件，或者注解式的Java代码配置。但无论是使用XML配置文件还是代码配置方式，都需要对相关组件的配置有足够的了解，再编写大量冗长的配置代码，这显然加大了开发者对Spring的使用难度。

所以，为了降低 Spring 框架的使用难度，让 Spring 组件能实现开箱即用，让开发者能快速上手，Spring Boot 框架便诞生了！

1.1.2 基本介绍

Spring Boot 是 Spring 全家桶项目中的一个子项目，也是 Spring 组件应用一站式解决方案，主要是为了降低 Spring 框架的使用难度，简省繁重的配置，所以，Spring Boot 现在也成了后端标准开发框架。

Spring Boot 提供了各种技术组件的一站式启动器（Starter），开发者只要定义好对应技术组件的配置参数，Spring Boot 就会自动配置，让开发者能快速搭建基于 Spring 生态的 Java 应用。

Spring Boot 不但能创建传统的 war 包应用，还能创建独立的、不依赖于任何外部 Servlet 容器的应用（内置 Servlet 容器），使用 java -jar 命令就能启动一个应用，Spring Boot 3.0 更是添加了对 GraalVM 原生镜像的支持，大大提升了 Spring 应用的使用体验。

1.1.3 核心思想

Spring Boot 框架的核心思想是：约定大（优）于配置（Convention over Configuration），即按约定进行编程，这是一种软件设计范式，旨在减少软件开发人员自主配置的数量，主要体现在以下两个方面：

- 约定并提供一些推荐的默认配置参数。
- 开发者只需要定义约定以外的配置参数。

这样做的好处是，如果约定的默认配置符合开发要求，则不用配置，反之，再进行额外的自定义参数配置，这样就能大大减少开发者的配置量。

关于约定大于配置，在后续的章节中都会有体现，比如 Spring Boot 中提供了默认的配置文件、配置格式，再到默认值自动配置，或者仅提供少量的个性化配置，这些都可以看出约定大于配置在 Spring Boot 框架中带来的便利性。

1.1.4 基本特性

1. 独立运行

Spring Boot 框架的诞生，改变了 Java 应用部署的格局。

Spring Boot 框架最大的亮点就是内嵌了各种 Servlet 容器，包括 Tomcat、Jetty、Undertow，应

用不再需要被打成 war 包再部署到 Tomcat 等 Servlet 容器中了，Spring Boot 应用可以被打包成一个可执行的 jar 包，所有的依赖都在一个 jar 包内，使用 java -jar 命令就能独立运行。

> 更多的 Spring Boot 部署和运行的方式可以参考第 11 章 Spring Boot 打包和部署。

2. 简化配置

传统的 Spring 应用中充斥着各种配置，比如各种技术组件的依赖配置、参数配置，以及 Spring 的各种配置文件，配置琳琅满目，不是专业的技术人员、架构师，去搭建一个 Spring 应用还是有一定难度的。

Spring Boot 提供了各种开箱即用的“Starter”一站式启动器，只要引入对应技术组件的 Starter 就能简化很多依赖配置，另外，Spring Boot 自动配置包中提供了各种技术的默认底层实现组件、默认的组件配置实例、默认的配置参数等，这些都有默认的实现。

以 Web 开发为例，只要添加一个 spring-boot-starter-web 启动器依赖，不需要其他任何配置，应用就能拥有 Spring Web 的能力，极大减少了开发人员对技术组件的配置管理。

> 关于 Spring Boot Starter 及自动配置原理可以参考第 3 章 Spring Boot Starter 与自动配置。

3. 自动配置

对于 Java 相关技术框架的集成，传统的 Spring 应用需要开发人员手动配置，包括应用所依赖的各种技术组件的核心配置类、Spring Bean 等组件，如果不是非常了解这些技术的配置组件，那么配置起来相当有难度。

Spring Boot 提供了自动配置能力，只要引入对应技术组件的 Starter，然后通过 Spring Boot 提供的默认配置参数，或者应用自定义的配置参数，就能自动配置，自动配置后这些技术就能快速集成并开箱使用。

自动配置功能可以推断应用可能需要加载哪些 Spring Bean，比如，如果类路径下有一个 HikariCP 连接池依赖包，并且此时应用没有提供任何有效的连接池配置类，那么 Spring Boot 就能推断应用可能需要一个连接池，并根据 HikariCP 依赖提供相应的连接池进行配置。如果应用已经配置了其他连接池，那么 Spring Boot 会放弃自动配置。

对于 Java 常用的主流技术，Spring Boot 官方都提供了自动配置能力及相关的 Starter，如果是官方不支持的第三方技术，它们一般也会提供带有自动配置的 Spring Boot Starter 启动器依赖，比如 MyBatis 提供的 mybatis-spring-boot-starter 启动器等。

> 如果没有特别复杂的定制需求，Spring Boot 默认自动配置后的组件就能轻松上手使用，如果不符合需求，Spring Boot 完全支持自定义并覆盖默认的自动配置。更多内容可以参考第 3 章 Spring Boot Starter 与自动配置。

4. 无代码生成和无须 XML 配置文件

Spring Boot 配置过程中无代码生成，意思就是 Spring Boot 不是通过生成代码的方式来完成自动配置的，也无须 XML 配置文件，而是通过 Spring & Spring Boot 中的各种条件注解完成的，Spring Boot 应用在运行时根据各种指定的条件、上下文环境参数等完成自动配置。

5. 各种生产级特性

Spring Boot 提供一系列生产级别的特性，比如端点、指标、健康检查、应用监控等，这对部署在生产环境中的 Spring 应用特别有帮助。

详细介绍可以参考第 12 章 Spring Boot 监控与报警。

1.1.5 核心模块

和 Spring 框架一样，Spring Boot 框架也是由许多核心模块组成的，每个模块负责不同的功能点，笔者整理了以下 10 大 Spring Boot 核心模块。

1. spring-boot

这是 Spring Boot 框架的主模块，也是支持其他模块的核心模块，主要功能如下：

- 提供了一个启动 Spring 应用的主类，并提供了一个相当方便的静态方法，它的主要作用是负责创建和刷新 Spring 容器的上下文。
- 提供了内嵌式的并可自由选择搭配的 Servlet 应用容器，如 Tomcat、Jetty、Undertow 等。
- 提供了一流的配置外部化支持。
- 提供了一个很方便的 Spring 容器上下文初始化器，包括对合理记录日志默认参数的支持。

2. spring-boot-autoconfigure

这个模块提供了常用的 Java 主流技术的自动配置组件，其提供的 @EnableAutoConfiguration 注解就能启用 Spring Boot 的自动配置功能，它能根据类路径下的内容决定是否自动配置。

3. spring-boot-starters

这个模块是所有 Starter 启动器的基础依赖，Starter 主要包括一系列技术组件依赖，它可以一站式开发 Spring 及相关技术应用，而不需要你到处找依赖和示例配置代码，它都帮你做好了。

例如 spring-boot-starter-web 这个 Starter 启动器，它引入了 spring-boot-starters 基础依赖及其相关的其他技术依赖。

4. spring-boot-cli

这是 Spring Boot 提供的命令行工具，也是另外一种创建 Spring 应用的方式，支持编译和运行 Groovy 应用，所以，它可以十分简单地编写并运行一个应用。

5. spring-boot-actuator

这是 Spring Boot 提供的监控模块，比如，它提供了健康端点、环境端点、Spring Bean 端点等端点，可以更好地帮助开发者监控应用并和应用进行交互。

6. spring-boot-actuator-autoconfigure

这个模块是为 spring-boot-actuator 监控模块提供自动配置的模块。

7. spring-boot-test

这是模块是 Spring Boot 的测试模块，为应用提供了许多非常有用的单元测试功能，包含了单元测试所需要的核心组件及注解等。

8. spring-boot-test-autoconfigure

这个模块是为 spring-boot-test 测试模块提供自动配置的模块。

9. spring-boot-loader

这个模块用于将 Spring Boot 应用构建为一个单独可执行的 jar 包，使用 java -jar 就能直接运行，一般不会直接使用这个来打包，而是使用 Spring Boot 提供的 Maven 或者 Gradle 插件。

10. spring-boot-devtools

这是 Spring Boot 开发者工具模块，主要用于 Spring Boot 应用的开发阶段，它提供了一些显著提升开发效率的特性，如修改了代码就自动重启应用等，以帮助开发者获得更流畅的应用开发体验。

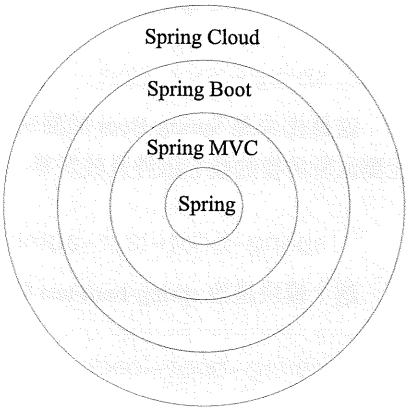
这个模块的功能是可选的，只限于本地开发环境，当打成整包运行时，这些功能会自动被禁用。

1.1.6 Spring Boot 与 Spring MVC、Spring 之间的关系

Spring Boot、Spring MVC、Spring 三者之间是互相依存的关系，如下图所示。

```
spring-boot-starter-web : 3.0.0 [compile]
  spring-boot-starter : 3.0.0 [compile]
  spring-boot-starter-json : 3.0.0 [compile]
  spring-boot-starter-tomcat : 3.0.0 [compile]
  spring-web : 6.0.2 [compile]
  spring-webmvc : 6.0.2 [compile]
    spring-aop : 6.0.2 [compile]
    spring-beans : 6.0.2 [compile]
    spring-context : 6.0.2 [compile]
    spring-core : 6.0.2 [compile]
    spring-expression : 6.0.2 [compile]
    spring-web : 6.0.2 [compile]
```

spring-boot-starter-web 依赖了 spring-webmvc，spring-webmvc 又依赖了 spring-beans、spring-core 等 Spring 底层组件，所以 Spring 还是底层的框架，Spring Boot、Spring MVC 只是其上层的封装。



Spring Boot 既不是完全摒弃，也不是用来代替 Spring MVC、Spring 框架，Spring Boot 只是简化了它们的使用而已。

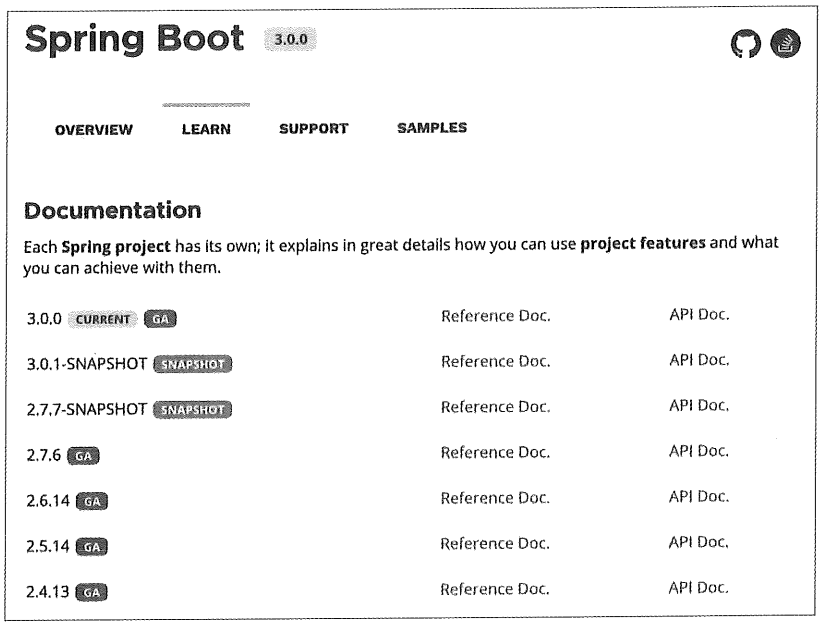
另外，再说说微服务框架 Spring Cloud，其本身也是基于 Spring Boot 框架进行构建的，Spring Cloud 也不能脱离以上任何一个 Spring 组件而独立存在。

这几个框架的依赖关系如右图所示。

1.2 Spring Boot 安装集成

1.2.1 版本周期

Spring Boot 项目同时维护了几条版本线，其中又包括不同意义的版本，比如 CURRENT、GA、PRE、SNAPSHOT 等代号，它们都代表着不同生命周期的版本，如下图所示。



1. GA（正式版本）

GA 全称为 General Availability，表示一般可用的版本，即已经向大众公开发行的正式版本，一旦发布，日后就不会修改该发行版本的任何内容，如果发现 Bug，则会在下一个版本中修复。GA 版本号一般不会带任何标识后缀，比如 3.0.0、2.7.2，但是 2.4.x 之前的版本是带 .RELEASE 标识的，比如 2.3.12.RELEASE。

在生产项目中必须要使用 GA 版本，我们从 Maven 中央仓库中获取的依赖也都是 GA 版本。

2. CURRENT（最新正式版本）

CURRENT 表示最新的 GA 版本，如上图所示，Spring Boot 3.0.0 正式发布的时候，其右侧显示了 CURRENT 标识，说明它就是目前最新的可用发行版本。

如果是学习研究使用，则可以使用 CURRENT 版本，但如果是生产项目，那么还是要谨慎使用，因为最新的发行版本没有多少用户量，可能存在潜在的问题及漏洞，所以生产项目最好和最新发行版本保持几个版本差。

3. SNAPSHOT（快照版本）

SNAPSHOT 即快照版本，一般每晚进行构建以包含最新的变更。一般每个版本只有一个 SNAPSHOT 快照版本，比如 3.0.0-SNAPSHOT、2.7.3-SNAPSHOT 等，如果发现 Bug，那么也会在

当前 SNAPSHOT 快照版本中修复。

SNAPSHOT 快照版本是最新测试版本，可能包含大量的 Bug，开发者可以拿来学习研究使用，强烈不建议用于生产项目。

4. PRE（预览版本）

PRE表示 Pre Release 版本，即预览版本，预览版本又有两个不同的阶段：

- **里程碑版本**：比如 3.0.0-M3，这里的 M 指的是 milestone，即里程碑版本，里程碑版本的版本号从 M1 开始递增。
- **候选发行版本**：比如 3.0.0-RC2，这里的 RC 指的是 Release Candidate，即候选发行版本，正式发行前的候选版本，和里程碑版本一样，从 1 开始递增，比如 Spring Boot 3.0.0 经历了 3.0.0-RC1 和 3.0.0-RC2。

预览版本发布后也是不会改变的，比如在 M1 中发现 Bug 时会在 SNAPSHOT 快照版本中修复并发布 M2，以此类推，当里程碑版本逐渐稳定之后就会发布候选发行版本，候选发行版本逐渐稳定之后就会发布正式版本。

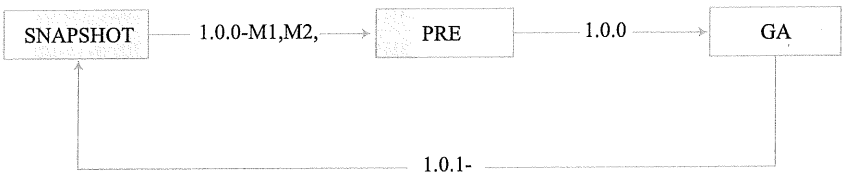
PRE预览版本也是测试版本，同样不建议在生产项目中使用，它可能会包含少量 Bug，开发者可以提前学习研究即将发布的 Spring Boot 最新正式版本的特性，比如笔者撰写本书的初衷是要基于最新的 Spring Boot 3.0，但不可能等它发布后再去写，那时就太迟了，所以笔者就基于 3.0.0-M* 里程碑系列版本开始撰写，然后又更新到了 3.0.0-RC2，最后等 3.0.0 正式版本发布后再进行复核。

SNAPSHOT 快照版本和 PRE 预览版本没有发布到 Maven 中央仓库，仅发布在 Spring 仓库中，如果要使用这两个版本，则需要在 Maven 配置文件中配置 Spring 仓库：

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/
2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/
maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    ...
    <repositories>
        <repository>
            <id>spring-snapshots</id>
            <url>https://repo.spring.io/snapshot</url>
            <snapshots><enabled>true</enabled></snapshots>
        </repository>
        <repository>
            <id>spring-milestones</id>
```

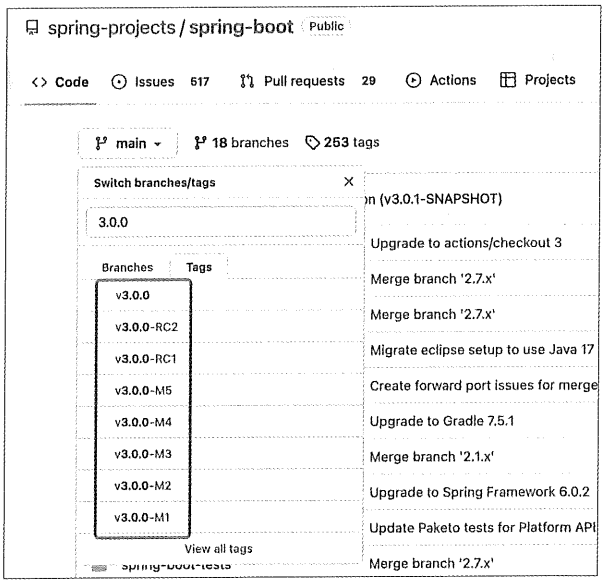
```
            <url>https://repo.spring.io/milestone</url>
        </repository>
    </repositories>

    <pluginRepositories>
        <pluginRepository>
            <id>spring-snapshots</id>
            <url>https://repo.spring.io/snapshot</url>
        </pluginRepository>
        <pluginRepository>
            <id>spring-milestones</id>
            <url>https://repo.spring.io/milestone</url>
        </pluginRepository>
    </pluginRepositories>

</project>
```

这些版本的发布流程如下图所示。



从 SNAPSHOT 快照版本开始，到多个 PRE 预览版本，再到正式版本。一旦正式版本发布，就不再维护相同版本的 SNAPSHOT 快照版本和 PRE 预览版本了，然后下一个版本就会继续从 SNAPSHOT 快照版本开始，如此往复，从 Spring Boot 版本图中可以看到，所有的 GA 版本都是没有 SNAPSHOT 快照版本和 PRE 预览版本的，只有还未正式发布的版本才会有这两个版本。

Spring Boot 官方源代码仓库的发布记录如右图所示。



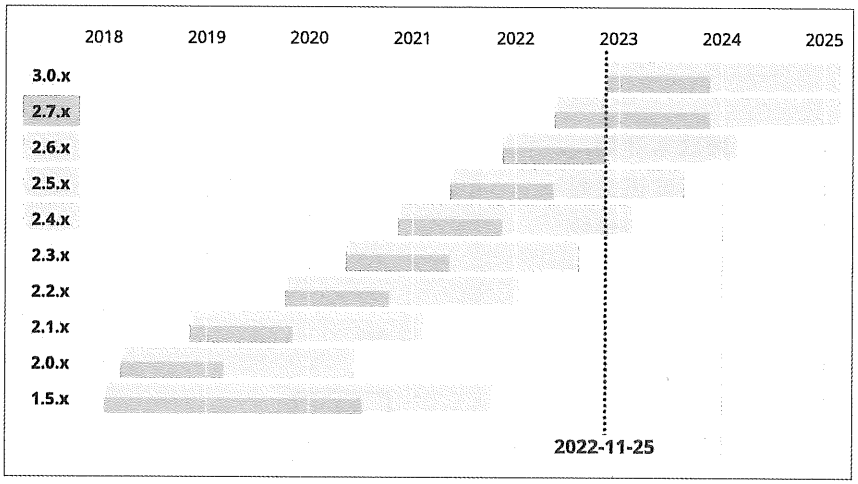
发布时间从近到远由上到下排列。

这也见证了 Spring Boot 从预览初始版本 M1 到 v3.0.0 正式发行版本的发布过程。

1.2.2 支持版本

Spring Boot 版本线如下表和下图所示。

版本线	发布日期	停止维护时间	商业支持停止时间
3.0.x	2022-11-24	2023-11-24	2025-02-24
2.7.x	2022-05-19	2023-11-18	2025-02-18
2.6.x	2021-11-17	2022-11-24	2024-02-24
2.5.x	2021-05-20	2022-05-19	2023-08-24
2.4.x	2020-11-12	2021-11-18	2023-02-23
2.3.x	2020-05-15	2021-05-20	2022-08-20
2.2.x	2019-10-16	2020-10-16	2022-01-16
2.1.x	2018-10-30	2019-10-30	2021-01-30
2.0.x	2018-03-01	2019-03-01	2020-06-01
1.5.x	2017-01-30	2019-08-06	2020-11-06



以上数据和图表来自 Spring Boot 官方文档（链接 1）。

随着 Spring Boot 3.0.0 的发布，免费受支持的版本就只有 Spring Boot 2.7+ 了，受商业支持的版本为 Spring Boot 2.4+，其他的低版本不建议使用了。

如果只是学习或者测试使用，使用哪个版本都没有问题，如果是公司实际生产环境项目，建议

不要使用最新的版本，避免“踩坑”，也不要使用已经停止维护的版本，避免带来潜在的漏洞和重大 Bug。如果只是公司内部系统，不会暴露到外网，在不存在重大 Bug 的情况下，选用哪个版本理论上都是没问题的。

1.2.3 环境要求

不同的 Spring Boot 版本对 Java 开发环境的要求是不一样的，以及对支持的 Servlet 版本和内置容器版本也是不一样的，以下以 Spring Boot 不同三条版本线的最新版本为例列出环境要求的对比。

> 笔者编写本书时，Spring Boot 2.5 已经不再继续维护，所以这里不再列出。另外，随着 Spring Boot 3.0.0 的正式发布，Spring Boot 2.6.x 也到了停止维护的时间。

对 Java 开发环境的要求对比如下表所示。

Spring Boot	JDK	Spring	Maven	Gradle
3.0.0	17 ～ 19	6.0.2+	3.5+	7.5+
2.7.6	8 ～ 19	5.3.24+	3.5+	6.8.x、6.9.x、7.x
2.6.14	8 ～ 19	5.3.24+	3.5+	6.8.x、6.9.x、7.x

Spring Boot 3.0 最低要求 JDK 17，并向上兼容支持 JDK 19，虽然 Oracle 宣布现在 Java 17+ 是可以免费使用及商用的，但不确定在免费时间截止后是否会继续收费，未来使用 Spring Boot 3.0+ 可以转战开源的 OpenJDK 或者其他开源的 JDK 版本。

Spring Boot 3.0 最低支持的 Spring 框架也变成了 Spring 6.0.2+，虽然是框架自动集成依赖的，但需要注意这一点，因为在 Spring Boot 3.0 发布之前的 Spring 6.0 也有不少的底层升级。

对 Servlet 容器的版本要求对比如下表所示。

Spring Boot	Servlet	Tomcat	Jetty	Undertow
3.0.0	5.0+	10+	11+	2.2+
2.7.6	3.1+	9.0	9.4/ 10.0	2.0
2.6.14	3.1+	9.0	9.4/ 10.0	2.0

Spring Boot 3.0 最低支持 Servlet 5.0，并已将所有底层依赖项从 Java EE 迁移到了 Jakarta EE API，基于 Jakarta EE 9 并尽可能地兼容 Jakarta EE 10。因为早在几年前，Java EE 已经正式更名为 Jakarta，所有相关的名称都变了，包括包名，所以升级 Spring Boot 时要考虑兼容性。

1.2.4 集成方式

Spring Boot 有两种集成方式：

- 使用 Maven/Gradle 插件，具体见 1.2.5 节。
- 使用 Spring Boot CLI 命令行工具，具体见 1.5 节。

大部分 Java 开发者日常的主要开发工作还是基于 Maven 进行的，Maven 也是 Java 开发最主流的项目构建工具，这从各种开源项目的演示教程和使用文档中也可以看出来，所以本书也是以 Maven 作为标准进行实战的。

> 笔者并不会介绍 Maven 的具体使用，读者可以关注微信公众号“Java 技术栈”，在后台回复关键字“mvn”获取最新完整版高清教程。

1.2.5 使用 Maven 集成 Spring Boot

使用 Maven 方式集成又有两种方式：

1. 继承 spring-boot-starter-parent 父项目

Spring Boot 应用继承 spring-boot-starter-parent 父项目依赖：

```
<!-- 从 Spring Boot 父项目中继承 -->
<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>${spring-boot.version}</version>
</parent>
```

2. 导入 spring-boot-dependencies 依赖

Spring Boot 应用导入 spring-boot-dependencies 依赖：

```
<dependencyManagement>
  <dependencies>
    <dependency>
      <!-- Import dependency management from Spring Boot -->
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-dependencies</artifactId>
      <version>${spring-boot.version}</version>
      <type>pom</type>
      <scope>import</scope>
    </dependency>
  </dependencies>
</dependencyManagement>
```

两种集成方式的区别：

前者需要继承 spring-boot-starter-parent 依赖，其实也继承了后者 spring-boot- dependencies，spring-boot-starter-parent 的 POM 配置文件如下：

```
<project xmlns="http://maven.apache.org/POM/4.0.0" xsi:schemaLocation="
http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
    <modelVersion>4.0.0</modelVersion>
    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-dependencies</artifactId>
        <version>3.0.0-M3</version>
    </parent>
    <artifactId>spring-boot-starter-parent</artifactId>
    <packaging>pom</packaging>
    <name>spring-boot-starter-parent</name>
    <description>Parent pom providing dependency and plugin management for applications
built with Maven</description>
    <properties>
        <java.version>17</java.version>
        <resource.delimiter>@</resource.delimiter>
        <maven.compiler.source>${java.version}</maven.compiler.source>
        <maven.compiler.target>${java.version}</maven.compiler.target>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
        <project.reporting.outputEncoding>UTF-8</project.reporting.outputEncoding>
    </properties>

  ...
```

前者 spring-boot-starter-parent 继承了 spring-boot-dependencies，定义了一些常用的项目相关的配置，比如 properties（参数）、plugins（插件）：

```
<properties>
    <java.version>17</java.version>
    <resource.delimiter>@</resource.delimiter>
    <maven.compiler.source>${java.version}</maven.compiler.source>
    <maven.compiler.target>${java.version}</maven.compiler.target>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    <project.reporting.outputEncoding>UTF-8</project.reporting.outputEncoding>
</properties>

<build>
    <pluginManagement>
        <plugins>
```

```
            ...
            <plugin>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-maven-plugin</artifactId>
                <executions>
                    <execution>
                        <id>repackage</id>
                        <goals>
                            <goal>repackage</goal>
                        </goals>
                    </execution>
                </executions>
                <configuration>
                    <mainClass>${start-class}</mainClass>
                </configuration>
            </plugin>
        </plugins>
    </pluginManagement>
</build>
```

使用继承集成的方式有以下几个优点。

继承方式的优点一：

插件只需要定义即可，不需要进行额外配置，如下所示。

```
<build>
  <plugins>
    <plugin>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-maven-plugin</artifactId>
    </plugin>
  </plugins>
</build>
```

因为其他插件配置已经在父项目中定义好了，子项目直接定义插件坐标即可。如果使用导入 spring-boot-dependencies 依赖的集成方式，则需要额外配置该插件。

继承方式的优点二：

可以直接定义 properties 参数覆盖依赖的版本，比如，不想使用 Spring Boot 继承的版本，而想使用自定义的版本，那么直接定义对应的 properties 参数覆盖版本号即可，如下所示。

```
<slf4j.version>1.7.36</slf4j.version>
```

如果使用导入 spring-boot-dependencies 依赖的集成方式，则需要显式导入依赖并指定版本号：

```
<dependencies>
    <dependency>
        <groupId>org.slf4j</groupId>
        <artifactId>slf4j-api</artifactId>
        <version>${slf4j.version}</version>
    </dependency>
 </dependencies>
```

继承集成的缺点就是不利于扩展，比如公司某个子项目需要继承某个父项目，因为只能单继承，所以一旦继承了 Spring Boot 的父依赖，就无法再继承公司的父项目了，扩展性不是很好。

两种集成方式各有优劣，笔者建议可以继承就继承，毕竟方便快捷。如果不能继承就使用导入依赖的方式。不过站在设计的原则上讲，一般推荐使用导入依赖的方式，避免后面出现多项目继承问题。笔者觉得，使用哪种方式还是要看具体的项目，集成方式本身并不会影响 Spring Boot 的正常使用。

1.3 快速开发一个 Spring Boot 接口

在软件编程领域，Hello World 经常用于某种编程语言、技术的快速入门体验，下面笔者就用 Spring Boot 框架开发一个最简洁的 Hello World 接口应用。

1.3.1 生成项目

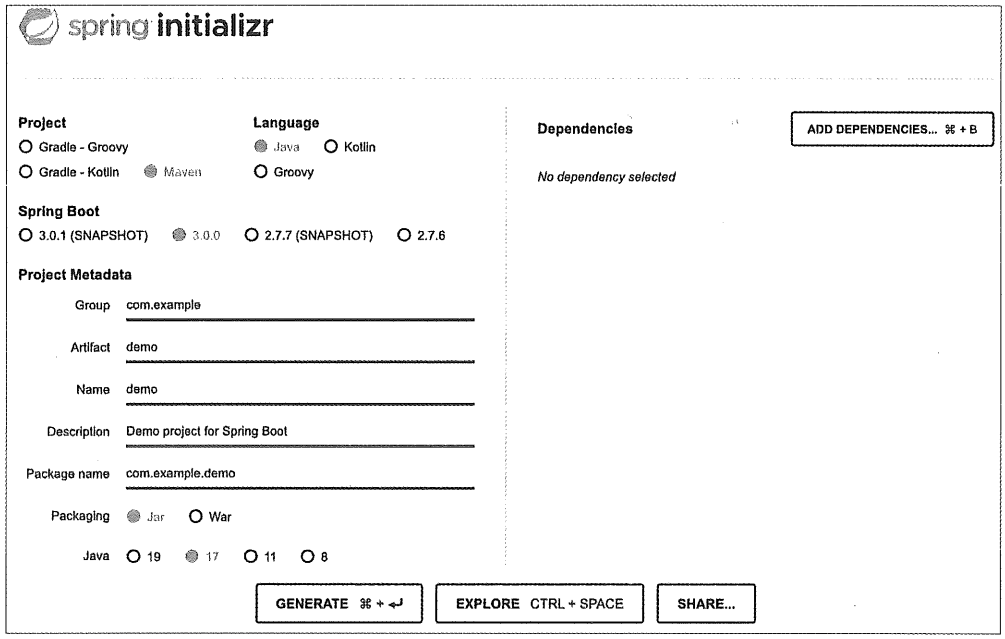
Spring 提供了一站式生成 Spring 应用的网站（链接 2）。

首先在该网站上指定要生成的项目环境信息，也可以添加相关的依赖或者后续再导入，笔者生成的应用环境如下：

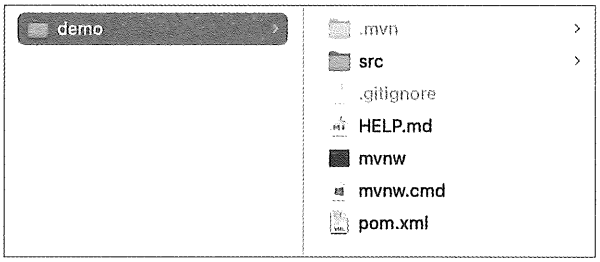
- Maven。
- Java。
- Spring Boot 3.0.0。
- JDK 17。

单击 GENERATE 按钮就能自动生成并下载该项目，如下图所示。然后打开 IDE 开发工具，通

过 Maven 的方式导入该项目即可。



生成的 demo 项目结构如下图所示。



这是一个典型的 Spring Boot 应用结构：

```
.
├── HELP.md
├── mvnw
├── mvnw.cmd
├── pom.xml
└── src
    ├── main
    │   ├── java
    │   │   └── com
```

```
    |   |       └── example
    |   |           └── demo
    |   |               └── DemoApplication.java
    |   └── resources
    |       └── application.properties
    └── test
        └── java
            └── com
                └── example
                    └── demo
                        └── DemoApplicationTests.java
```

打开 pom.xml 配置文件：

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/
XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.
org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>3.0.0-M3</version>
        <relativePath/> <!-- lookup parent from repository -->
    </parent>
    <groupId>com.example</groupId>
    <artifactId>demo</artifactId>
    <version>0.0.1-SNAPSHOT</version>
    <name>demo</name>
    <description>Demo project for Spring Boot</description>
    <properties>
        <java.version>17</java.version>
    </properties>
    <dependencies>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter</artifactId>
        </dependency>

        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-test</artifactId>
```

```
            <scope>test</scope>
        </dependency>
    </dependencies>

    <build>
        <plugins>
            <plugin>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-maven-plugin</artifactId>
            </plugin>
        </plugins>
    </build>
    <repositories>
        <repository>
            <id>spring-milestones</id>
            <name>Spring Milestones</name>
            <url>https://repo.spring.io/milestone</url>
            <snapshots>
                <enabled>false</enabled>
            </snapshots>
        </repository>
    </repositories>
    <pluginRepositories>
        <pluginRepository>
            <id>spring-milestones</id>
            <name>Spring Milestones</name>
            <url>https://repo.spring.io/milestone</url>
            <snapshots>
                <enabled>false</enabled>
            </snapshots>
        </pluginRepository>
    </pluginRepositories>

</project>
```

这种方式默认使用的是继承集成的方式，会添加默认的依赖：

- spring-boot-starter。
- spring-boot-starter-test。

还会添加一个插件：

- spring-boot-maven-plugin。

另外会生成一个启动类：

```
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class DemoApplication {

    public static void main(String[] args) {
        SpringApplication.run(DemoApplication.class, args);
    }

}
```

这是 Spring Boot 应用的入口类，而 @SpringBootApplication 注解则用来标识 Spring Boot 应用的入口类，更多深入分析请参考第 4 章 Spring Boot 启动过程与扩展应用。

1.3.2 导入依赖

编写 Spring Boot 接口，如果在生成项目时没有选择 spring-boot-starter-web 依赖，则需要手动导入：

```
<dependencies>

    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>

</dependencies>
```

前面的章节也介绍了，导入一个这样的启动器依赖就能拥有 Spring Web 开发的能力。

1.3.3 编写接口

编写接口就和 Spring Boot 框架本身没有关系了，使用 Spring MVC 框架的相关注解即可，笔者这里编写了一个 /hello 的测试接口：

```
@SpringBootApplication
@RestController
public class Application {
```

```
    public static void main(String[] args) {
        SpringApplication.run(Application.class);
    }

    @RequestMapping("/hello")
    public String helloWorld() {
        return "hello world.";
    }

}
```

1.3.4 启动应用

在 IDE 中启动 Spring Boot 应用有两种方法：

1. 使用 Java 中的 main 方法启动

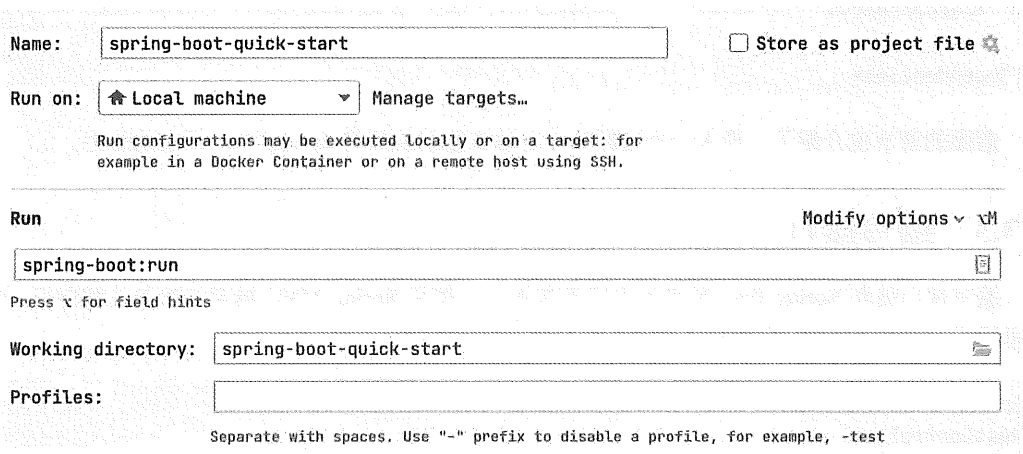
Spring Boot 应用可以直接运行应用启动类的 main 方法，Spring Boot 启动类一般位于根包下面，比如 demo 项目中的 DemoApplication 类。

2. 使用应用构建插件运行

如果应用集成了 Maven 插件 spring-boot-maven-plugin，就可以使用 Maven 命令启动：

```
mvn spring-boot:run
```

一般在 IDE 中运行 Maven 命令，可以省略 mvn 命令前缀，如下图所示。



Maven 应用建议使用这种启动方式，因为这样可以使用 Maven 的各种插件，如在使用资源插

件时的配置文件中的占位符替换处理，如果使用第一种方法直接运行 main 方法，就会因为无法替换占位符而启动报错。

Gradle 也有类似的插件，这里不再赘述。

应用启动日志，如下所示。

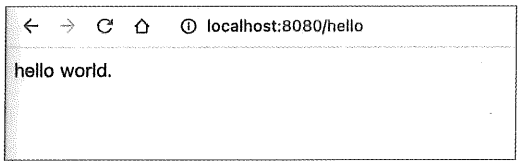
```
  .   ____          _            __ _ _
 /\\ / ___'_ __ _ _(_)_ __  __ _ \ \ \ \
( ( )\___ | '_ | '_| | '_ \/ _` | \ \ \ \
 \\/  ___)| |_)| | | | | || (_| |  ) ) ) )
  '  |____| .__|_| |_|_| |_\__, | / / / /
 =========|_|==============|___/=/_/_/_/
 :: Spring Boot ::                (v3.0.0)

。... : Starting Application using Java 17.0.4 with PID 85087 ...
。... : No active profile set, falling back to 1 default profile: "default"
。... : Devtools property defaults active! Set 'spring.devtools.add-properties' to
'false' to disable
。... : For additional web related logging consider setting the 'logging.level.web'
property to 'DEBUG'
。... : Tomcat initialized with port(s): 8080 (http)
。... : Starting service [Tomcat]
。... : Starting Servlet engine: [Apache Tomcat/10.1.1]
。... : Initializing Spring embedded WebApplicationContext
。... : Root WebApplicationContext: initialization completed in 1070 ms
。... : LiveReload server is running on port 35729
。... : Tomcat started on port(s): 8080 (http) with context path '/javastack'
。... : Started Application in 2.336 seconds (process running for 2.963)
```

这样，一个 Spring Boot 应用就运行起来了，默认的绑定端口是 8080。

1.3.5 测试接口

在浏览器中直接访问刚才编写的 /hello 接口，效果如下图所示。

```
http://localhost:8080/hello
```



接口正常响应了，网页正常显示了 hello world.。

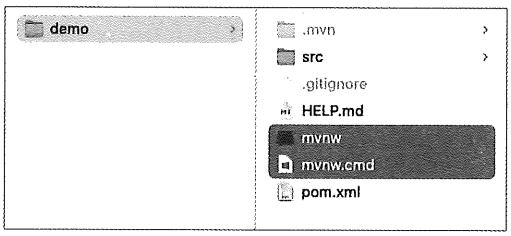
1.4 快速使用 Maven

Spring Boot 还提供了相关快捷脚本，可以免安装和快速使用 Maven、Gradle。

1.4.1 基本介绍

如下图所示，在生成的 Demo 项目中，除了常规的项目文件，还有两个特殊的脚本文件：

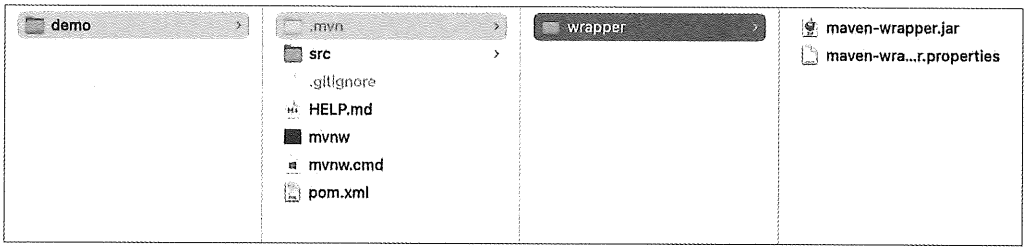
- mvnw（Linux 版本）。
- mvnw.cmd（Windows 版本）。



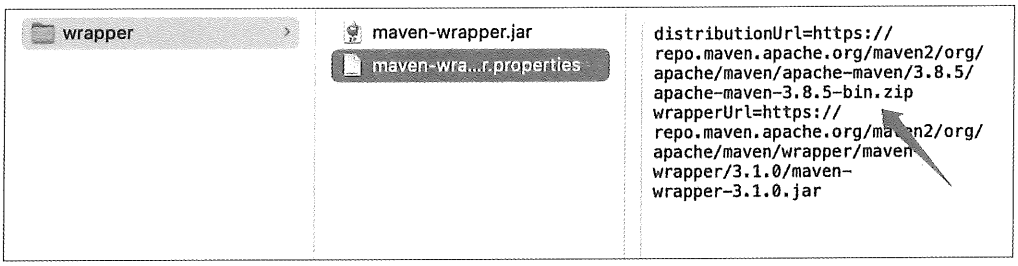
mvnw 的全称为 Maven Wrapper，使用 mvnw 可以快速将 Maven 集成到项目中，它的适用场景有以下两个：

- 想省心，不想自己安装配置 Maven 环境。
- 已有的 Maven 版本不合适，需要使用特定的 Maven 版本。

除了上面所说的 mvnw 脚本，在当前目录下会初始化一个 .mvn/wrapper 目录，如下图所示。

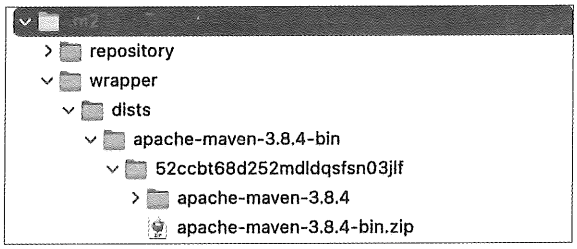


如果 Maven Wrapper 使用的并不是最新版本，但想使用最新的 Maven 3.8.5，则可以在 maven-wrapper.properties 配置文件中指定版本，如下图所示。



这意味着如果想使用 mvnw，只要复制这些生成的文件到对应的项目目录中，想使用哪个版本，改一改参数即可。

然后在 $USER_HOME/.m2/wrapper 目录中可以看到安装信息，如下图所示。



虽然不用自己另行安装，但本地仓库还是会自动下载对应版本的 Maven，只是省去了自己动手安装的流程。

1.4.2 使用方式

使用 Spring Initializr 网站一键生成的 Spring Boot 项目不需要单独安装 mvnw，生成后默认就带有 mvnw 系列文件，直接使用即可。

比如，我们切换到 demo 项目：

```
cd demo
```

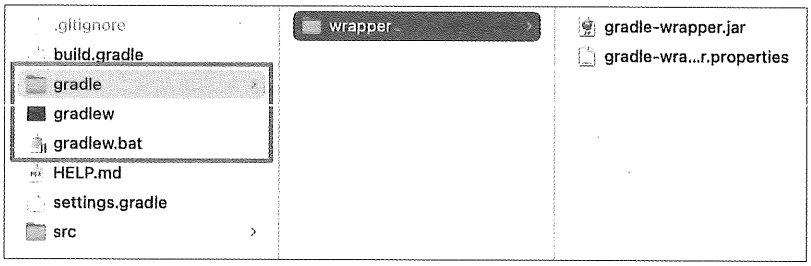
运行项目清理安装命令：

```
./mvnw clean install
```

就像直接使用 mvn 命令一样，mvnw 只是在其基础上封装了一层而已，底层还是 mvn。所以，如果有多个 Maven 版本管理的需求，或者是不想自己动手安装 Maven 的场景，那么这个 mvnw 脚本还是可以用来试试的，但实际工作中很少会使用。

1.4.3 Gradle Wrapper

Gradle 也有类似的包装，如下图所示。



和 Maven 类似，这里就不再赘述了。

1.5 Spring Boot CLI

1.5.1 概述

Spring Boot CLI 的全称为 Spring Boot Command-Line Interface，是一个构造 Spring Boot 项目的命令行工具，可用于从 start.spring.io 网站构建新项目或用来加密。它可以运行 Groovy 脚本，它也是 JVM 系语言，拥有和 Java 类似的语法，但比 Java 要简洁得多，无须太多冗余的项目结构配置代码，从而可以快速构建项目。

Spring Boot 并不一定需要 Spring Boot CLI，但有了 Spring Boot CLI，可以在不需要 IDE 的情况下快速启动基于 Spring 的应用。

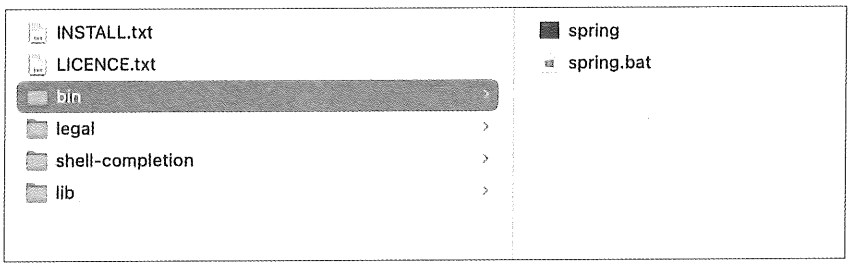
1.5.2 安装方式

Spring Boot CLI 要求 JDK 17 环境，Groovy 不需要安装，已经被打包在 Spring Boot CLI 包中了，已经安装的也会被忽略。

1. 手动安装

在 Spring 官网下载相应的版本 ZIP 包（链接 3）。

下载后，解压并复制到某个目录中，如下图所示。



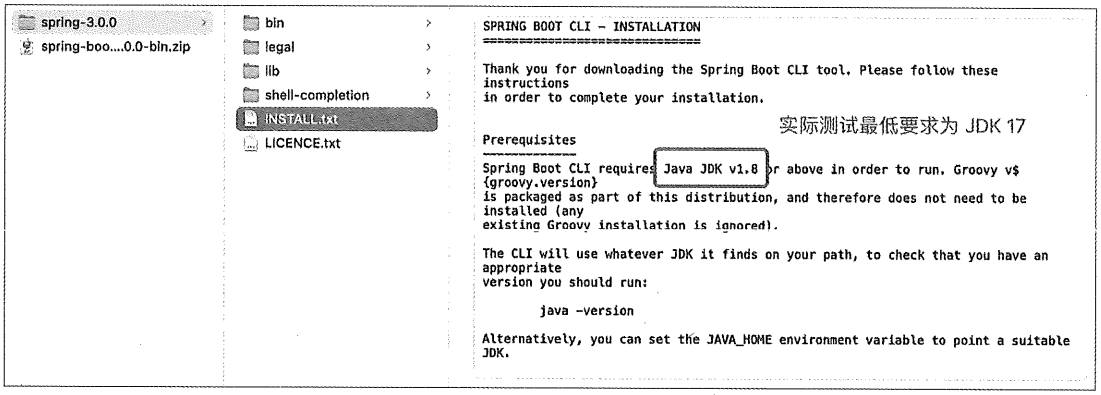
如上图所示，目录中有区别 Linux/ Windows 的运行程序，根目录中还有一个 INSTALL.txt 安装说明文件，主要介绍环境要求及如何设置 SPRING_HOME/PATH 环境变量：

- SPRING_HOME：设置环境变量指向 Spring Boot CLI 安装包解压后的根目录。
- PATH：设置环境变量指向 Spring Boot CLI 安装包解压后的 /bin 目录。

配置好之后，通过以下命令验证是否安装成功：

```
$ spring --version
Error: A JNI error has occurred, please check your installation and try again
Exception in thread "main" java.lang.UnsupportedClassVersionError: org/springframework/boot/loader/
JarLauncher has been compiled by a more recent version of the Java Runtime (class file version 61.0),
this version of the Java Runtime only recognizes class file versions up to 52.0
 at java.lang.ClassLoader.defineClass1(Native Method)
 at java.lang.ClassLoader.defineClass(ClassLoader.java:756)
 at java.security.SecureClassLoader.defineClass(SecureClassLoader.java:142)
 at java.net.URLClassLoader.defineClass(URLClassLoader.java:468)
 at java.net.URLClassLoader.access$100(URLClassLoader.java:74)
 at java.net.URLClassLoader$1.run(URLClassLoader.java:369)
 at java.net.URLClassLoader$1.run(URLClassLoader.java:363)
 at java.security.AccessController.doPrivileged(Native Method)
 at java.net.URLClassLoader.findClass(URLClassLoader.java:362)
 at java.lang.ClassLoader.loadClass(ClassLoader.java:418)
 at sun.misc.Launcher$AppClassLoader.loadClass(Launcher.java:355)
 at java.lang.ClassLoader.loadClass(ClassLoader.java:351)
 at sun.launcher.LauncherHelper.checkAndLoadMain(LauncherHelper.java:601)
```

上面显示抛出了异常，说明本机的 JDK 版本和 Spring Boot CLI 中运行的 JDK 版本不兼容，查看 /lib/spring-boot-cli-3.0.0-*.jar 包中的配置文件，实际用的是 JDK 17，和安装说明文件描述的 JDK 1.8 有出入，所以需要调整环境变量到 JDK 17，如下图所示。



调整 JDK 环境变量后再来验证一下：

```
$ spring --version
Spring CLI v3.0.0.RELEASE
```

显示版本号就说明安装成功了，Spring Boot 3.0.0 最低要求 JDK 17，这一点也体现在 Spring Boot CLI 工具上了。

2. 通过 SDKMAN安装

SDKMAN 的全称为 Software Development Kit Manager，是一个用于在大多数基于 UNIX 的系统上管理多个软件开发工具包的并行版本的工具，它提供了一个方便的命令行界面（CLI）和 API，可用于应用安装、切换、删除、列出候选版本等。SDKMAN 就支持包括 Groovy 和 Spring Boot CLI 的安装，本节就通过 SDKMAN 的方式来安装。

首先需要前往 SDKMAN 网站下载 SDKMAN（链接 4）。

SDKMAN 官网提供了一键安装脚本：

```
curl -s "https://get.sdkman.io" | bash
```

SDKMAN 安装完成之后，使用以下命令安装 Spring Boot CLI：

```
$ sdk install springboot
$ spring --version
Spring CLI v3.0.0.RELEASE
```

3. OSX Homebrew/ MacPorts

Spring Boot CLI 还支持 macOS 操作系统下 OSX Homebrew、MacPorts 等安装方式。

OSX Homebrew：

```
$ brew tap spring-io/tap
$ brew install spring-boot
```

MacPorts：

```
$ sudo port install spring-boot-cli
```

1.5.3 命令行自动补全

Spring Boot CLI 提供了命令行自动补全 Shell 脚本，支持 BASH 和 ZSH 集成，Spring Boot CLI 是通过手动下载压缩包安装的，需要自行建立符号链接，参考命令如下：

```
$ ln -s ./shell-completion/bash/spring /etc/bash_completion.d/spring
$ ln -s ./shell-completion/zsh/_spring /usr/local/share/zsh/site-functions/_spring
```

如果是通过 SDKMAN安装的，则使用以下参考命令：

```
$ . ~/.sdkman/candidates/springboot/current/shell-completion/bash/spring
```

如果是通过 Homebrew 或者 MacPorts 方式安装的，则会注册自动补全 Shell 脚本。

设置好之后，输入 spring 命令、空格，按 tab 键，会自动出现命令补全提示：

```
$ spring 按tab键
encodepassword  -- Encode a password for use with Spring Security
grab            -- Download a spring groovy script's dependencies to ./repository
help            -- Get help on commands
init            -- Initialize a new project using Spring Initializr (start.spring.io)
install         -- Install dependencies to the lib/ext directory
jar             -- Create a self-contained executable jar file from a Spring Groovy script
run             -- Run a spring groovy script
shell           -- Start a nested shell
uninstall       -- Uninstall dependencies from the lib/ext directory
version         -- Show the version
war             -- Create a self-contained executable war file from a Spring Groovy script
```

比如这里的 encodepassword 指令是使用 Spring Security 框架对密码进行加密的，这样在某些使用 Spring Security 框架需要手动生成密码的场合，就不需要在代码中写 main 方法生成密码了。

1.5.4 快速开始

前面介绍了Spring Boot CLI可以运行Groovy脚本以快速构建Spring项目，本节来快速实战一下，以下是笔者在桌面创建的一个简单的app.groovy脚本文件：

```
@RestController
class ThisWillActuallyRun {

    @RequestMapping("/hi")
    String home() {
       "Hello Spring Boot!"
    }

}
```

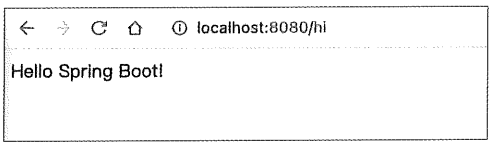
然后使用Spring Boot CLI运行命令就能直接运行该脚本文件了，第一次运行会比较慢一点，因为要下载必要的依赖项，后续运行会很快。

运行app.groovy脚本参考命令：

```
$ spring run app.groovy
Resolving dependencies.....................

.   ____          _            __ _ _
/\\ / ___'_ __ _ _(_)_ __  __ _ \ \ \ \
( ( )\___ | '_ | '_| | '_ \/ _` | \ \ \ \
\\/  ___)| |_)| | | | | || (_| |  ) ) ) )
'  |____| .__|_| |_|_| |_\__, | / / / /
=========|_|==============|___/=/_/_/_/
:: Spring Boot ::                (v3.0.0)

...

...Started application in 2.544 seconds (JVM running for 4.453)
...Tomcat started on port(s): 8080 (http) with context path ''
...Started application in 2.544 seconds (JVM running for 4.453)
```

如上启动日志，启动端口为8080，就像启动正常的Java应用一样，因为Groovy本身就是JVM系编程语言，在Java中创建一个接口可能需要许多项目配置，而Groovy只需要一个脚本就够了。

访问Groovy脚本中的/hi接口，能正常输出结果，如下图所示。



只需一个 Groovy 脚本、一条 Spring Boot CLI 命令就能运行一个 Spring 应用，这太轻松了，省去了 Java 项目工程配置、依赖配置等大量烦琐工作。所以，如果你了解 Groovy 语言，使用 Spring Boot CLI 命令行工具可以比使用传统 Java & Maven 方式更快速地构建 Spring 应用，极大提升生产力。

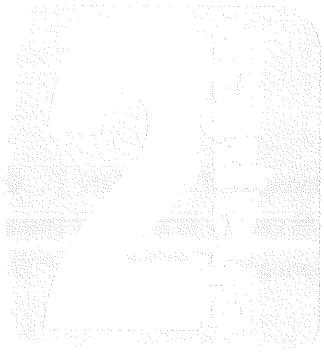
1.6 开启 Spring Boot 之旅

在前面的章节中，我们了解了 Spring Boot 的背景、基本介绍、集成方式、启动类及其注解的使用，也用了五个小步骤演示了如何从零开始快速开发一个 Spring Boot 应用接口，除了导入指定的依赖，几乎没有任何的配置，甚至 Maven 和 Gradle 都不需要开发人员额外安装。

如果是传统的 Spring应用呢?

我们需要花费更多的时间在工程和 Spring 框架配置上面，还有 Maven、Gradle、Tomcat 容器的安装、配置、部署都需要花费不少时间，而 Spring Boot 从开始到接口运行只要几分钟，这正是 Spring Boot 框架所能解决的问题—极大简化了 Spring 的使用难度，让 Java 开发者能快速上手开发一个 Spring 应用。

另外，使用 Spring Boot CLI & Groovy 集成方式，一个脚本就能运行一个应用，所以，笔者建议开发者尽快上手使用 Spring Boot 框架。



第 2 章
Spring Boot 配置管理

本章将介绍 Spring Boot 配置相关的一些内容，比如如何自定义配置类、导入配置，以及应用的核心配置文件、配置绑定、外部化配置、多文档配置、基于 Profiles 的配置、配置加密、配置迁移等，只有先理解这些基础配置的用法和原理，打好基础，才能更好地理解 Spring Boot 后续的内容。

2.1 配置类

2.1.1 自定义配置类

第 1 章介绍了 Spring Boot 的特性，其中有一条就是无代码生成、无 XML 配置，虽然 Spring Boot 能做到大部分代码自动配置，但应用还有自定义的组件配置，以及应用自身的配置等，这些都是需要通过 Java 自定义配置类提供的。

Spring 3.0 之前，要使用 Spring 组件就必须要有一个 XML 配置文件，这也是 Spring 的核心配置文件，而 Spring 3.0 之后就不需要 XML 配置文件了，可以通过新的 @Configuration 注解代替 XML 配置文件。

第 1 章的 Spring Boot 集成实战中用到了 Spring Boot 的核心注解 @SpringBootApplication，它其实就包含了 @SpringBootConfiguration 配置类注解，也就是 Spring Boot 应用的专用配置类注解。

@SpringBootApplication 注解的详细介绍见第 4 章 Spring Boot 启动过程与扩展应用。

@SpringBootConfiguration 注解的源码如下：

```
@Target({ElementType.TYPE})
@Retention(RetentionPolicy.RUNTIME)
@Documented
@Configuration
@Indexed
public @interface SpringBootConfiguration {
    @AliasFor(annotation = Configuration.class)
    boolean proxyBeanMethods() default true;
}
```

如源码所示，它组合了 Spring 框架的原生 @Configuration 注解，用来代替 @Configuration 注解，示例代码如下：

```
@SpringBootConfiguration
public class MainConfig {

    @Bean
    public RestTemplate restTemplate() {
        return new RestTemplate();
    }

}
```

如上，笔者使用 @SpringBootConfiguration 注解替换了 @Configuration 注解，其实使用哪个都一样，一个是 Spring Boot 换了个名字的专用注解，另一个是 Spring 原生注解，它们两个是等效的，所以，在 Spring Boot 中不需要 XML 配置，只需要自定义若干用 @Configuration 或者 @SpringBootConfiguration 注解修饰的 Java 配置类即可，Spring Boot 应用推荐使用 @SpringBootConfiguration 注解。

以上的 @Bean 注解就等于 XML 配置文件中的 <bean ...> 标签，Spring 3.0+ 中有一整套的注解来代替 XML 配置文件中的定义，比如 @Lazy、@Primary、@Scope 等，完全不需要再用 XML 进行配置，并且在 Spring Boot 框架中都可以正常使用，Spring Boot 也会有自己扩展的相关注解，比如上面提到的 @SpringBootConfiguration 注解。

2.1.2 导入配置

1. 导入配置类

我们知道在 Spring Boot 中可以使用一个 @SpringBootConfiguration 或者 @Configuration 注解的配置文件来配置所有 Bean 及其他组件，但其实没必要这么做，我们可以把相关的配置独立出来，

放到多个配置类中，这样更好管理。

比如可以有以下参考配置类：

- MainConfiguration：项目主要配置。
- DataSoureceConfiguration：数据源配置。
- RedisConfiguration：Redis 配置。
- MongoDBConfiguration：MongoDB 配置。

……

但实际开发过程中有另外一种情况，可能某些配置类在其他子包中，或者在依赖的 jar 包中，这些配置无法被 Spring Boot 默认扫描注册到，需要额外导入，基于这种情况，Spring 框架提供的 @Import 导入注解就能派上用场了，它的源码如下：

```
@Target(ElementType.TYPE)
@Retention(RetentionPolicy.RUNTIME)
@Documented
public @interface Import {

        /**
         * {@link Configuration @Configuration}, {@link ImportSelector},
         * {@link ImportBeanDefinitionRegistrar}, or regular component classes to import.
         */
        Class<?>[] value();

}
```

从 value 注释可以看出，value 是一个 Class 数组，用于导入多个配置，具体类型可以是以下几种：

- @Configuration。
- ImportSelector。
- ImportBeanDefinitionRegistrar。
- 任何一个 component 组件。

来看下面的示例，笔者创建了两个配置类，并通过 @Import 注解导入：

```
@SpringBootConfiguration
@Import({Configuration1.class, Configuration2.class})
public class MainConfig {
```

```
    @Bean
    public RestTemplate restTemplate() {
        return new RestTemplate();
    }

}
```

其他 @Import 注解用法在后续的章节中会有更多的应用。

当然，如果这些配置类都在类扫描路径下，就不用通过 @Import 导入了，直接用 @ComponentScan 注解添加要扫描的路径即可，因为 @SpringBootApplication 注解包含了 @ComponentScan 注解并扫描所有子包目录，所以不必重复添加。但是，像上面那种情况，即配置类不在默认的类扫描路径下，此时可以用 @ComponentScan 注解指定要扫描的包目录。

2. 导入 XML 配置

Spring 框架还提供了一个 @ImportResource 注解，用于导入额外的 XML 配置文件，它的源码如下：

```
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.TYPE)
@Documented
public @interface ImportResource {

    @AliasFor("locations")
    String[] value() default {};

    @AliasFor("value")
    String[] locations() default {};

    Class<? extends BeanDefinitionReader> reader() default BeanDefinitionReader.class;

}
```

使用方式很简单，指定要导入的 XML 配置文件路径即可，比如有些老的技术组件是 XML 配置形式的，如果某些 XML 标签无法找到对应的 Spring 替代注解，或者无法通过 @Configuration 方式配置，那么此时使用 @ImportResource 注解就非常管用。

其实，这两个导入注解也都是 Spring 框架的原生注解，和 Spring Boot 框架本身也并没有什么关系，但在平时开发时要知道如何在 Spring Boot 框架中灵活运用。

2.2 配置文件

2.2.1 application

Spring Boot 应用的配置参数主要在 application 配置文件中，见 StandardConfigDataLocationResolver 类源码中的定义，它可以有不同形式的配置方式，该配置文件也不是必需的，可以有多个，也可以一个都没有。

StandardConfigDataLocationResolver 源码中的定义如下：

```
static final String CONFIG_NAME_PROPERTY = "spring.config.name";

static final String[] DEFAULT_CONFIG_NAMES = { "application" };
```

如果不想使用默认的 application 配置文件，则可以在命令行使用 spring.config.name 参数名指定其他的配置文件名称：

```
$ java -jar demo.jar --spring.config.name=app
```

默认的配置文件搜索路径为：

- optional:classpath:/;optional:classpath:/config/。
- optional:file:./;optional:file:./config/;optional:file:./config/*/。

这个在 ConfigDataEnvironment 类静态初始块中可以找到，在后面介绍配置加载机制源码的章节中也会介绍。

默认的配置文件搜索路径可以通过 spring.config.location 命令行参数指定，既可以是目录（需要以“/”结束），也可以是具体文件，比如下面指定了多个配置文件命名：

```
$ java -jar myproject.jar --spring.config.location=\
 optional:classpath:/default.properties,\
 optional:classpath:/override.properties
```

其中的 optional 前缀表示配置是可选的，如果指定的配置文件不存在，默认会报告异常，所以指定 optional 前缀就可以不用管配置文件是否存在。

如果应用想忽略所有的类似异常而正常启动，则可以通过以下参数来设置：

```
spring:
  config:
    on-not-found: ignore
```

另外，这个 spring.config.location 命令行参数会覆盖应用默认的配置文件，如果不想覆盖，则可以使用 spring.config.additional-location 参数追加其他的配置文件。

2.2.2 bootstrap

除了 application 配置文件，还有另外一种 bootstrap 配置文件，但它不能独立于 Spring Boot 应用生效，它属于 Spring Cloud 环境，需要引入 Spring Cloud 依赖。bootstrap 配置文件会优先于 application 配置文件被加载，它主要用于从额外的资源中加载配置参数，通常用来加载外部配置，如 Spring Cloud 用于加载配置中心的配置参数，也可以用来定义系统不会被改变的参数，它们默认不能被本地相同的配置所覆盖。

因此，对比 application 配置文件，bootstrap 配置文件具有以下几个特性：

- bootstrap 由父 ApplicationContext 加载，比 application 优先被加载。
- bootstrap 中的参数不能被覆盖。

一般的 Spring Boot 应用使用 application 配置文件即可，除非用到了 Spring Cloud，并遇到了以下几个应用场景：

- 使用了外部化配置中心。
- 有一些固定的不能被覆盖的参数。

2.2.3 配置文件类型

配置文件类型可以是以下两种：

- .properties：key = value 格式。
- .yml：key: value 树状格式（全称：yaml）。

比如下面的示例，分别用 .properties 和 .yml 两种格式指定了应用的访问端口和访问路径，如果不指定，则默认分别为 8080 和 /。

.properties 格式：

```
server.port = 8080
server.servlet.contextPath= /javastack
```

.yml 格式：

```
server:
  port: 8081
  servlet:
    contextPath: /javastack
```

使用 .yml 配置文件的格式说明如下：

- 键冒号后面需要带一个空格。
- 每个层级的缩进用两个空格。

一般推荐使用 .yml 格式文件，树状的层次结构比较清晰，且节省冗余的参数前缀配置量，一个 .yml 配置文件就能适配多个特定环境，而使用 .properties 格式则需要多个配置文件，这个会在后面的 profiles 章节介绍。

需要注意的是，.yml 配置文件不支持通过 @PropertySource 注解来导入配置，因为这个注解本来就是只针对 .properties 文件的，有用到这个注解的地方需要注意，但也可以有其他解决方案，比如使用 @ConfigurationProperties 注解，它同时支持两种文件格式，这个在接下来的配置绑定一节中会讲到。

虽然 .yml 配置文件不支持 @PropertySource 注解，但它可以通过 YamlPropertiesFactoryBean 类被加载为一个 Properties 参数类，或者通过 YamlMapFactoryBean 类被加载为一个 Map，还可以通过 YamlPropertySourceLoader 被加载为 Spring 环境中的 PropertySource。

2.3 配置绑定

2.3.1 Spring 中的配置绑定

在 Spring 框架中，所有已加载到 Spring 环境中的配置都可以通过注入 Environment 环境 Bean 来获取：

```
@Autowired
private Environment env;

// 获取参数
String getProperty(String key);
```

使用 @Value(“${property}”) 注解在类成员变量上绑定配置参数，这是最原始的做法，一般不推荐这样做，一方面是太不优雅，另一方面是，如果注入多个参数就会很麻烦，比如以下同时注入

多个前缀一样的参数：

```
@Value("${user.username}")
private String username;

@Value("${user.age}")
private int age;

@Value("${user.sex}")
private int sex;

...
```

这样注入还会显得代码太冗余了，也非常不优雅。

另外，Spring 框架也有 @PropertySource 注解绑定配置，但也仅限于 .properties 配置文件，比如绑定资源目录下的 config/db-config.properties 配置参数：

```
@Data
@Component
@PropertySource(value = {"config/db-config.properties"})
public class DbProperties {

    @Value("${db.username}")
    private String username;

    @Value("${db.password}")
    private String password;

}
```

由于 @PropertySource 注解并不支持主流的 .yml 配置文件绑定，其本身也需要结合 @Value 注解使用，所以这也不是推荐的用法。

Spring Boot 提供了多种配置参数注入方法，比如参数绑定、构造器绑定等，支持多种配置文件格式，并且还提供了参数验证，让配置参数注入更加方便、灵活、安全。

2.3.2 参数绑定

本节通过 Java Bean 提供的 setter 方法进行配置参数与 Java Bean 字段的绑定，首先在 application.yml 配置文件中添加以下 javastack.* 配置参数：

```yaml
javastack:
  name: Java 技术栈
  site: www.javastack.cn
  author: 栈长
  users:
    - Jom
    - Lucy
    - Jack
  params:
    tel: 18800008888
    address: China
  security:
    security-key: 123321
    security-code: 666666
```

新建 JavastackProperties 参数类进行参数绑定：

```
@Data
@ConfigurationProperties(prefix = "javastack")
public class JavastackProperties {

    private boolean enabled;

    private String name;

    private String site;

    private String author;

    private List<String> users;

    private Map<String, String> params;

    private Security security;

}

@Data
class Security {

    private String securityKey;

    private String securityCode;

}
```

一般参数类要以 XxxProperties 命名，比如 Spring Boot 中的各种参数类：

- ServerProperties。
- DataSourceProperties。
- JdbcProperties。

……

这样当我们配置任意其他框架时，都可以直接找到其配置参数类，如要配置 Redis，则可以直接打开 RedisProperties 参数类，看看都有什么配置。

通过 @ConfigurationProperties 注解可以将配置参数映射到一个 Java Bean 上，该注解中的 prefix 或者 value 参数用于指定要映射的参数前缀，前缀格式为英文小写，多个前缀以“-”分隔，例如：

```
javastack.user-agent
```

因为 @ConfigurationProperties 注解绑定配置参数需要用到类的 setter 方法，所以这里用到了 Lombok 的 @Data 注解，用于在类编译时生成类的 getter、setter、构造器、toString 等方法，这里不再赘述，具体可以参考笔者的博客文章（链接 5）。

然后在启动类上添加 @EnableConfigurationProperties 注解，用于指定要启用的 @ConfigurationProperties 参数类 JavastackProperties，使其生效：

```
@SpringBootApplication
@RequiredArgsConstructor
@EnableConfigurationProperties(value = {JavastackProperties.class})
@Slf4j
public class Application {

    private final JavastackProperties javastackProperties;

    @Bean
    public CommandLineRunner commandLineRunner() {
        return (args) -> {
            log.info("javastack properties: {}", javastackProperties);
        };
    }

    ....

}
```

然后在要使用参数类的地方直接将其注入，以上 JavastackProperties 参数类使用的是构造器注入方式。

最后在启动类中添加一个 CommandLineRunner 测试一下，这个 CommandLineRunner 是 Spring Boot 内置的命令行运行器，用于在应用启动之后执行一些特定的操作，比如上述程序用于在应用启动时输出一些配置参数。

更多关于 CommandLineRunner 的详细介绍请参考第 4 章 Spring Boot 启动过程与扩展应用。

输出结果如下：

```
javastack properties: JavastackProperties(enabled=false, name=Java 技术栈,
site=www.javastack.cn, author=栈长, users=[Jom, Lucy, Jack], params={tel=18800008888,
address=China}, security=Security(securityKey=123321, securityCode=666666))
```

配置参数正常输出，下面总结一下 @ConfigurationProperties 注解配置绑定：

- 支持按配置参数的前缀进行绑定，前缀一样的配置参数将被绑定到同一个类上。
- 支持配置参数使用默认值，比如示例中的 enabled 参数并没有配置，但输出了默认值 false。
- 支持松绑定（xx1-xx2-xx3 -> xx1Xx2Xx3），这里的连接符 - 也可以是下画线 _，比如示例中的 Security 嵌套类的 securityKey 字段，对应配置文件中的 security-key 参数，还可以是 XX_XX_XX 的大写格式，这个在绑定环境变量参数时非常有用。
- 支持 Java 集合绑定，比如示例中的 Map、List 字段值正常绑定了。
- 支持嵌套类，比如示例中特别使用了一个 Security 嵌套类，其所有字段值都能正常绑定。
- 支持主要的配置途径，比如 .yml 文件、.properties 文件、环境变量参数等。
- 需要搭配 @EnableConfigurationProperties 注解共同使用。

2.3.3 构造器绑定

本节通过 Java Bean 提供的构造器进行配置参数与 Java Bean 字段的绑定，首先在 application.yml 配置文件中添加以下 member.* 配置参数：

```
member:
  name: Tom
  sex: 1
  age: 18
  birthday: 2000-12-12 12:00:00
```

然后新建一个 MemberProperties 参数类进行参数绑定：

```
@Data
@NoArgsConstructor
@ConfigurationProperties(prefix = "member")
public class MemberProperties {

    private String name;
    private int sex;
    private int age;
    private String country;
    private Date birthday;

    public MemberProperties(String name, int sex, int age) {
        this.name = name;
        this.sex = sex;
        this.age = age;
    }

    @ConstructorBinding
    public MemberProperties(String name,
                            int sex,
                            int age,
                            @DefaultValue("China") String country,
                            @DateTimeFormat(pattern = "yyyy-MM-dd HH:mm:ss") Date birthday) {
        this.name = name;
        this.sex = sex;
        this.age = age;
        this.country = country;
        this.birthday = birthday;
    }

}
```

通过 @ConfigurationProperties 注解指定要绑定的配置参数的前缀，再使用 @ConstructorBinding 注解指定要绑定的构造器方法。

注解说明如下：

- @DefaultValue：提供默认值。
- @DateTimeFormat：指定日期格式。

然后在启动类 @EnableConfigurationProperties 注解中指定要启用的 @ConfigurationProperties 配

置参数类 MemberProperties 并使其生效：

```
@SpringBootApplication
@RequiredArgsConstructor
@EnableConfigurationProperties(value = {JavastackProperties.class, MemberProperties.class})
@Slf4j
public class Application {

    private final JavastackProperties javastackProperties;

    private final MemberProperties memberProperties;

    @Bean
    public CommandLineRunner commandLineRunner() {
        return (args) -> {
            log.info("javastack properties: {}", javastackProperties);
            log.info("member properties: {}", memberProperties);
        };
    }

    ...

}
```

最后注入其参数类，并在 commandLineRunner 方法中添加测试代码。

输出结果如下：

```
member properties: MemberProperties(name=Tom, sex=1, age=18, country=China,
birthday=Tue Dec 12 12:00:00 CST 2000)
```

可以看出，构造器的绑定也很简单，可以共同使用 @ConfigurationProperties 注解和 @EnableConfigurationProperties 注解，如果有多个构造器，则需要通过一个 @ConstructorBinding 注解指定要使用的构造器。

@ConstructorBinding 注解的源码如下：

```
@Target(ElementType.CONSTRUCTOR)
@Retention(RetentionPolicy.RUNTIME)
@Documented
public @interface ConstructorBinding {

}
```

Spring Boot 3.0 新变化：

（1）@ConstructorBinding 注解由 org.springframework.boot.context.properties 包移到了 org.springframework.boot.context.properties.bind 包，但目前还是 Deprecated 状态，在后续版本中会被彻底删除。

（2）@ConstructorBinding 注解在 Spring Boot 2.x 中是可以同时用于类和构造器方法上的，而在 3.0 版本中只能用在具体某个构造器方法上，如上面的 @ConstructorBinding 注解的源码所示。

（3）如果配置类只有一个参数化的构造器，则无须使用 @ConstructorBinding 注解指定。如果参数类有多个参数化的构造器，那么还是需要使用 @ConstructorBinding 注解指定要绑定的构造器，否则会调用默认的无参构造器并通过 setters 方法注入绑定。

这几点变化能大大简化 @ConfigurationProperties 注解的使用难度，如果通过构造器注入 Bean 时不想使用 @ConstructorBinding 构造器绑定的方式，则可以使用 @Autowired 注解替代。

2.3.4 Bean 配置绑定

除了使用 @ConfigurationProperties 注解在一个类上，还可以在 @Bean 方法上使用它，比如将参数绑定到一个第三方类上时，因为它们可能在 jar 包中，所以无法在其类、字段上添加注解等设置。

比如新建一个 OtherMember 其他成员类：

```
@Data
@NoArgsConstructor
public class OtherMember {

    private String name;
    private int sex;
    private int age;

}
```

再新建一个参数类，并添加一个 OtherMember 类的 Bean：

```
@SpringBootConfiguration
public class MainConfig {

    ...

    @Bean
```

```
    @ConfigurationProperties(prefix = "member")
    public OtherMember otherMember() {
        return new OtherMember();
    }

}
```

可以看到，还是用 @ConfigurationProperties 注解绑定的配置参数，只是把该注解从参数类中移到了 @Bean 方法之上。因为使用 @Bean 方式，所以不需要使用 @EnableConfigurationProperties 注解来使 @ConfigurationProperties 配置类生效，直接注入就能使用，示例如下：

```
@SpringBootApplication
@RequiredArgsConstructor
@EnableConfigurationProperties(value = {JavastackProperties.class, MemberProperties.class})
@Slf4j
public class Application {

    private final JavastackProperties javastackProperties;

    private final MemberProperties memberProperties;

    private final OtherMember otherMember;

    @Bean
    public CommandLineRunner commandLineRunner() {
        return (args) -> {
            log.info("javastack properties: {}", javastackProperties);
            log.info("member properties: {}", memberProperties);
            log.info("other member: {}", otherMember);
        };
    }

    ...

}
```

最后注入其参数类，并在 commandLineRunner 方法中添加测试代码。

输出结果：

```
other member: OtherMember(name=Tom, sex=1, age=18)
```

配置参数成功绑定，在不能修改配置参数类的情况下确实有用，但这种方式只能绑定简单的数

据类型，像 Date 这种数据类型就没有对应的解决方案。

2.3.5 参数类扫描

前面章节中的参数绑定和构造器绑定需要将 @ConfigurationProperties 注解搭配 @EnableConfigurationProperties 注解使用，不能自行使用 @Component 等注解进行注册，如果后面新增了别的参数类，而忘了在 @EnableConfigurationProperties 注解中指定，则参数类不会生效，比较麻烦。

为此，Spring Boot 提供了一种参数类扫描方式：

```
@SpringBootApplication
@RequiredArgsConstructor
//@EnableConfigurationProperties(value = {JavastackProperties.class, MemberProperties.class})
@ConfigurationPropertiesScan
@Slf4j
public class Application {
```

在启动类上使用 @ConfigurationPropertiesScan 注解，就可以扫描所有包目录下的参数类，如果要扫描指定包目录，则在注解的 basePackages 参数中指定具体的包即可。

2.3.6 配置验证

使用 @ConfigurationProperties 注解还能有效地进行参数验证，Spring Boot 支持 JSR-303 javax.validation 规范注解，首先添加一个遵循 JSR-303 标准实现的依赖：

```
<dependency>
  <groupId>org.hibernate.validator</groupId>
  <artifactId>hibernate-validator</artifactId>
</dependency>
```

然后在字段 name 上面使用一个 @NotNull 不为空注解：

```
@Data
@Validated
@ConfigurationProperties(prefix = "javastack")
public class JavastackProperties {

    private boolean enabled;

    @NotNull
    private String name;
```

```
    ...
}
```

在配置文件中把 name 改为 names，重启应用进行测试。结果应用启动失败，并输出了以下错误信息：

```
***************************
APPLICATION FAILED TO START
***************************
Description:
Binding to target org.springframework.boot.context.properties.bind.BindException: Failed
to bind properties under 'javastack' to cn.javastack.springboot.properties.props.
JavastackProperties failed:
    Property: javastack.name
    Value: null
    Reason: 不能为 null
Action:
Update your application's configuration
```

提示 javastack.name 参数不能为 null，并提示更新应用的配置，所以这个配置验证也很有用，可避免配置参数被误删除等操作导致的配置缺失。

2.4 外部化配置

2.4.1 配置源

Spring Boot 还可以将配置外部化，即除了把配置放在应用配置文件中，还可以使用各种外部配置源，主要包含以下几种：

- properties 文件。
- YAML 文件。
- 环境变量。
- 命令行参数。

2.4.2 配置优先级

Spring Boot 使用了一个非常特殊的 PropertySource 顺序，该顺序可以按优先级自动覆盖参数值，如果多个配置源的参数值相同，则优先级高的参数值最后会覆盖优先级低的参数值。

配置源优先级从低到高如下：

（1）默认参数（SpringApplication.setDefaultProperties）。

（2）使用 @PropertySource 注解绑定的配置。

（3）应用配置文件中的参数（application）。

（4）配置了 random.* 随机数的参数。

（5）系统环境变量。

（6）Java System properties。

（7）java:comp/env 的 JNDI 参数。

（8）ServletContext 初始化参数。

（9）ServletConfig 初始化参数。

（10）来自 SPRING_APPLICATION_JSON 的参数。

（11）命令行参数。

（12）单元测试上的参数。

（13）使用 @TestPropertySource 注解绑定的配置。

（14）Devtools 全局设置参数（来自 $HOME/.config/spring-boot）。

多个应用配置文件（application）的优先级从低到高如下：

（1）应用配置文件（jar 包内）。

（2）指定了 profile 的配置文件，如 application-dev.properties（jar 包内）。

（3）应用配置文件（jar 包外）。

（4）指定了 profile 的配置文件，如 application-dev.properties（jar 包外）。

举个例子，比如注入一个 user.name 参数：

```
@Value("${user.name}")
private String username;
```

如果 application.yml 配置文件配置了该参数值，但系统环境变量中也有此参数值，因为系统环境变量的优先级高于应用配置文件，所以最后会使用系统环境变量中的参数值。

建议在整个应用中统一使用一种格式，但如果同一位置同时有 .properties 和 .yml 两种格式的配置文件，则 Spring Boot 会优先使用 .properties 配置文件。

2.4.3 命令行参数

命令行参数指的是使用 java -jar --key=value 命令启动应用时指定的参数，即以 "--" 开头的参数，比如：

```
java -jar demo.jar --server.port=9090
```

从配置优先级可以看到，命令行参数的优先级始终是高于配置文件的，所以以上命令会使用 9090 端口代替配置文件中的端口，或者默认的 8080 端口。

默认情况下，Spring Boot 会将所有命令行参数转换并添加到 Spring 环境中，如果不想添加到 Spring 环境中，则可以将其禁用：

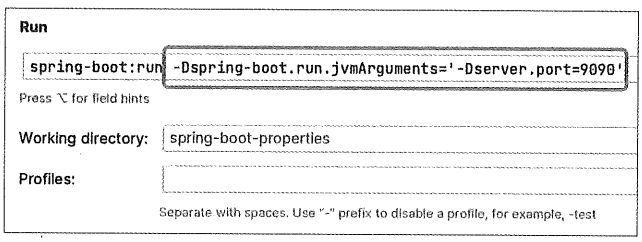
```
public static void main(String[] args) {
    SpringApplication.setAddCommandLineProperties(false);
    SpringApplication.run(Application.class, args);
}
```

测试以上设置，先注入 server.port 参数并把它打印出来：

```
@Value("${server.port}")
private int serverPort;

@Bean
public CommandLineRunner commandLineRunner() {
    return (args) -> {
        log.info("server.port: {}", serverPort);
    };
}
```

如果是在 IDE 中使用 Maven 启动应用，则可以参考以下命令使用方法，具体如下图所示。

```
spring-boot:run -Dspring-boot.run.jvmArguments='-Dserver.port=9090'
```



应用重启后输出结果如下：

```
...

Tomcat initialized with port(s): 9090 (http)

...

server.port: 9090
```

端口绑定正确，Spring 环境中的参数值取的也是命令行参数的值。所以，有时候想临时改变应用的配置参数，可以使用命令行参数，因为它的优先级够高，而且使用也很方便。

2.5 导入配置

可以使用 spring.config.import 参数来指定要导入的配置文件的路径，比如下面指定了要导入一个 app.yml 配置文件：

```
spring:
  config:
    import:
      - optional:classpath:/config/app.yml
```

然后新建该配置文件：

```
javastack:
  enabled: true
  site: app.javastack.cn
```

使用之前的示例重启测试，输出结果：

```
javastack properties: JavastackProperties(enabled=true, name=Java 技术栈 ,
site=app.javastack.cn, author= 栈长 , users=[Jom, Lucy, Jack], params={tel=18800008888,
address=China}, security=Security(securityKey=123321, securityCode=666666))
```

字段 enabled 有了指定的 true 值，site 值也被覆盖了，这是因为导入的其他位置的配置文件的优先级要高于其触发导入配置文件的优先级。

这里的 optional 前缀及配置文件不存在时抛出异常的处理和 spring.config.location 参数的处理一致，具体可参考前面的配置文件章节。

2.6 随机值配置

Spring Boot 提供的 RandomValuePropertySource 类可用于注入随机值，它可以生成整数（int）、长整数（long）、UUID 及字符串。

比如下面的示例，修改了前面章节的 .yml 配置文件：

```
javastack:
  name: Java 技术栈
  site: www.javastack.cn
  author: 栈长
  users:
    - Jom
    - Lucy
    - Jack
  params:
    tel: 18800008888
    address: China
  security:
    # 生成随机 32 位 MD5 字符串
    security-key: ${random.value}
    security-code: ${random.uuid}

member:
  name: Tom
  sex: 1
  age: ${random.int[18,100]}
  birthday: 2000-12-12 12:00:00
```

上面以 ${random.*} 标识的就是随机数，配置了随机字符串（value）、UUID，以及带范围的随机整数。

重启应用来看一下效果：

```
javastack properties: JavastackProperties(enabled=true, name=Java 技术栈 ,
site=app.javastack.cn, author= 栈长 , users=[Jom, Lucy, Jack], params={tel=18800008888,
address=China}, security=Security(securityKey=d2b8c180f07844c733a1fcbc8561164d,
securityCode=58ec8e15-4604-4708-8662-0cb1a3ae7d4b))

member properties: MemberProperties(name=Tom, sex=0, age=61, country=China,
birthday=Tue Dec 12 12:00:00 CST 2000)
```

输出结果达到了预期，下面看一下获取指定随机值的方法的源码：

```
private Object getRandomValue(String type) {
    if (type.equals("int")) {
        return getSource().nextInt();
    }
    if (type.equals("long")) {
        return getSource().nextLong();
    }
    String range = getRange(type, "int");
    if (range != null) {
        return getNextIntInRange(Range.of(range, Integer::parseInt));
    }
    range = getRange(type, "long");
    if (range != null) {
        return getNextLongInRange(Range.of(range, Long::parseLong));
    }
    if (type.equals("uuid")) {
        return UUID.randomUUID().toString();
    }
    return getRandomBytes();
}
```

实现很简单，其实就是使用 Java 自带的 java.util.Random 和 java.util.UUID 等工具类，这里就不再详细解析了，感兴趣的读者可以自行查看 RandomValuePropertySource 类的实现源码。

实际工作中很少使用随时数配置，但不能不知道它的存在，笔者知道的一个典型应用场景就是微服务随机端口，这样就能快速地进行扩容，当然也需要处理端口重复占用的情况。

2.7 多文档配置

Spring Boot 可以将单个物理的配置文件拆分为多个独立的逻辑配置文档，配置参数从上到下按顺序处理，后面的配置参数值可以覆盖前面的配置参数值。

多文档配置一般用于指定多套不同的配置，比如开发、测试、生产环境，它们的环境配置可能是不同的，比如端口，所以多文档配置中配置只在指定激活的环境中才会生效。

2.7.1 配置格式

.yaml 配置文件多文档之间用三个连接符(---)进行分隔，.properties 配置文件多个文档之间用 #---

进行分隔，如下面示例所示。

.yaml 配置文件：

```
spring:
  application:
    name: "javastack"
---
spring:
  application:
    name: "javastack-cloud"
  config:
    activate:
      on-cloud-platform: "kubernetes"
```

.properties 配置文件：

```
spring.application.name=javastack
#---
spring.application.name=javastack-cloud
spring.config.activate.on-cloud-platform=kubernetes
```

以上代码在一个物理配置文件中定义了 2 个逻辑文档。

使用多文档配置需要注意以下几点：

- 多文档配置的分隔符前面不能有空格，并且分隔符要连续。
- 多文档配置的分隔符前、后一行不能是注释。
- 多文档配置不能被 @PropertySource 和 TestPropertySource 注解加载。

2.7.2 激活多文档配置

多文档配置一般要配合指定的条件激活，比如在指定的环境中才激活生效，Spring Boot 支持根据指定的 Profile 和云平台进行激活，激活参数及说明如下表所示。

激活参数	说明
spring.config.activate.on-profile	根据指定的 Profile 激活
spring.config.activate.on-cloud-platform	根据指定的云平台激活

下面新建了两个配置文档，而第二个配置文档只在 dev 和 test 环境中生效：

```
spring:
```

```
  profiles:
    active: dev
  config:
    import:
      - optional:classpath:/config/app.yml

javastack:
  name: Java 技术栈
  site: www.javastack.cn
  author: 栈长
  users:
    - Jom
    - Lucy
    - Jack
  params:
    tel: 18800008888
    address: China
  security:
    # 生成随机 32 位 MD5 字符串
    security-key: ${random.value}
    security-code: ${random.uuid}

member:
  name: Tom
  sex: 1
  age: ${random.int[18,100]}
  birthday: 2000-12-12 12:00:00

---
member:
  name: Jack
  sex: 1
  age: 20
  birthday: 2000-01-01 12:00:00

spring:
  config:
    activate:
      on-profile: "dev | test"
```

然后通过 spring.profiles.active 激活了 dev 环境。

重启应用再测试一下：

```
...

The following 1 profile is active: "dev"

...

member properties: MemberProperties(name=Jack, sex=1, age=20, country=China,
birthday=Sat Jan 01 12:00:00 CST 2000)
```

从启动日志可以看到，dev 环境被激活了，member 参数的系列值被覆盖为 Jack 的了，即后面激活的文档配置覆盖了前面的文档配置。

2.8 Profile

基于 Profile 的配置也是 Spring Boot 的重要特性，在前面的章节中也基本介绍了 Profile 的功能，Profile 提供了一种分离应用配置的方法，让配置仅在某些具体的环境中才被激活生效，但在 Spring Boot 系列版本中也发生了不少变更，使用时特别需要注意。

2.8.1 默认的 Profile

如果不指定要激活的 Profile，Spring Boot 就会使用默认的 Profiledefault，这个可以从一开始的启动日志中看到：

```
No active profile set, falling back to 1 default profile: "default"
```

也可以调整默认的 Profile，如下所示。

```
spring:
  profiles:
    default: dev
```

以上表示如果没有指定 Profile，则默认使用 dev 环境。

2.8.2 激活 Profile

可以使用 spring.profiles.active 参数指定要激活的 Profile，比如在 application.yml 配置文件中指定激活 dev 和 test 环境：

```
spring:
  profiles:
    active: dev, test
```

还可以在应用启动方法上进行激活：

```
public static void main(String[] args) {
    SpringApplication springApplication = new SpringApplication(Application.class);
    springApplication.setAdditionalProfiles("dev", "main");
    springApplication.run(args);
}
```

Profile 不仅可激活配置文件，还可以激活 Java 配置类，比如可以用在 @Component、@Configuration 或 @ConfigurationProperties 注解的类上。

下面是一个 @Configuration 使用示例：

```
@Profile("main")
@SpringBootConfiguration
@Import({Configuration1.class, Configuration2.class})
public class MainConfig {

    ...

}
```

这样只有在激活 main 环境时，这个配置类中的配置才会被加载，然后通过配置文件被激活：

```
spring:
  profiles:
    default: dev
    active: dev, main
```

在使用 @Component、@Configuration 注解时，可以直接把 @Profile 注解用在该注解所在的类上，但 @ConfigurationProperties 的使用需要说明一下：

- 如果使用的是直接扫描的注册方式，则 @Profile 注解可以直接用在 @ConfigurationProperties 类上。
- 如果使用的是 @EnableConfigurationProperties 注册方式，则需要把 @Profile 注解用在 @EnableConfigurationProperties 配置类上。

2.8.3 切换 Profile

这个 spring.profiles.active 参数和其他参数遵循一样的顺序加载规则，比如即使在配置文件这里激活了，还是可以通过命令行参数临时替换 Profile：

```
$ java -jar demo.jar --spring.profiles.active=test
```

如果使用 IDE 启动：

```
$ mvn spring-boot:run -Dapp.profiles=test
```

因为命令行参数的优先级比配置文件的优先级高，所以这样就使用了 test 环境，配置文件中激活的 dev 在 main 环境中不会被用到。

所以，如果不想直接被优先级更高的配置源替换，也可以使用 spring.profiles.include 附加模式：

```
spring:
  profiles:
    default: dev
    # 可以被命令行替换
    active: dev, main
    # 不会被替换
    include:
      - dev
      - main
```

这样即使在命令行用 --spring.profiles.active 参数切换了 Profile，也不会影响 dev 和 main 环境的激活。

2.8.4 Profile 分组

有时候，除了 dev、test、main 这些 Profile，可能还会有一些独立存在的 Profile，比如以下示例：

```
...

---
member:
  name: Jack1

spring:
  config:
    activate:
```

```
    on-profile: main1

---
member:
  name: Jack2

spring:
  config:
    activate:
      on-profile: main2
```

这些 main1、main2 等 Profile 只有在激活 main 环境时才被激活，因为使用之前的参数配置就会比较麻烦，所以 Spring Boot 提供了 Profile 分组功能。

以下代码定义了一个 main 组，其中包含上面所说的两个 Profile：

```
spring:
  profiles:
    default: dev
    # 可以被命令行切换
    active: dev, main
    # 不会被切换
    include:
      - dev
      - main
    group:
      main:
        - main1
        - main2
```

只要 main 环境激活，main1 和 main2 也会一起被激活。

重启应用查看输出日志：

```
...

The following 4 profiles are active: "dev", "main", "main1", "main2"

...

member properties: MemberProperties(name=Jack2, sex=1, age=20, country=China,
birthday=Sat Jan 01 12:00:00 CST 2000)
```

日志显示输出了被激活的几个 Profile，Member 参数的 name 值也变成了 Jack2，这是因为最后

的 main2 覆盖了所有之前的 Profile 的参数值。

2.8.5 指定 Profile 配置文件

除了之前章节介绍的多文档配置，现在有了 Profile 功能，应用就可以有多个基于 Profile 的配置文件，格式为：

```
application-${profile}
```

比如可以有以下配置文件：

- application.yml。
- application-dev.yml。
- application-test.yml。
- application-prod.yml。
- application-main.yml。

……

这样只有激活指定的 Profile，对应 Profile 的配置文件才会被激活。

从之前的配置优先级章节中可以知道，基于 Profile 的配置文件的优先级要高于默认的配置文件的优先级，所以如果按顺序同时激活了 dev 和 main 环境，则配置文件的优先级为：

```
默认> dev > main
```

优先级高的配置参数会覆盖优先级低的配置参数，所以如果以上三个配置文件都有相同的值，那么最后会取 main 中的配置参数。

下面新建两个配置文件：

- application-dev.yml。
- application-main.yml。

分别在这两个配置文件中把 member.name 参数的值设置为 Lucy1 和 Lucy2，再重启应用查看日志：

```
...

The following 4 profiles are active: "dev", "main", "main1", "main2"
```

```
...

member properties: MemberProperties(name=Lucy2, sex=1, age=20, country=China,
birthday=Sat Jan 01 12:00:00 CST 2000)
```

日志显示输出了被激活的几个 Profile，Member 参数的 name 值也变成了 Luck2，这样就看出基于 Profile 配置文件的优先级了。

2.8.6 使用限制

Profile 中常用的几个参数如下表所示。

参数	功能	说明
spring.profiles.default	指定默认的 Profile	当不指定 Profile 时，默认生效的 Profile
spring.profiles.active	激活指定的 Profile	可以被优先级高的配置源替换
spring.profiles.include	激活指定要包含的 Profile	不会被其他配置源替换
spring.profiles.group	指定 Profile 分组	激活一个分组，就会激活组下所有 Profile

注意，Profile 有一些使用上的限制，不能随意搭配使用，以上几个参数不能用于多文档配置中和指定 Profile 配置文件。

比如在 application.yml 多文档配置中使用 spring.profiles.active 参数激活：

```
...

---
member:
  name: Jack2

spring:
  config:
    activate:
      on-profile: main2
  profiles:
    active: main
```

重启应用就会抛出异常：

```
ERROR org.springframework.boot.SpringApplication - Application run failed
org.springframework.boot.context.config.InactiveConfigDataAccessException: Inactive
property source 'Config resource 'class path resource [application.yml]' via location
'optional:classpath:/' (document #3)' imported from location 'class path resource
```

```
[application.yml]' cannot contain property 'spring.profiles.active' [origin: class
path resource [application.yml] - 70:13]
        at org.springframework.boot.context.config.InactiveConfigDataAccessException.
throwIfPropertyFound(InactiveConfigDataAccessException.java:126)
        at org.springframework.boot.context.config.ConfigDataEnvironmentContributors
$InactiveSourceChecker.onSuccess(ConfigDataEnvironmentContributors.java:310)
```

比如在 application-dev.yml 配置文件中使用 spring.profiles.active 参数激活：

```
spring:
  profiles:
    active: main
```

重启应用后也会抛出异常：

```
ERROR org.springframework.boot.SpringApplication - Application run failed
org.springframework.boot.context.config.InvalidConfigDataPropertyException: Property
'spring.profiles.active' imported from location 'class path resource [application-dev.
yml]' is invalid in a profile specific resource [origin: class path resource [application-dev.
yml] - 3:13]
    at org.springframework.boot.context.config.InvalidConfigDataPropertyException.
lambda$throwIfPropertyFound$1(InvalidConfigDataPropertyException.java:121)
    at java.base/java.lang.Iterable.forEach(Iterable.java:75)
    at java.base/java.util.Collections$UnmodifiableCollection.forEach (Collections.
java:1092)
    at org.springframework.boot.context.config.InvalidConfigDataPropertyException.
throwIfPropertyFound
```

所以，如果要将这几个参数放在配置文件中，则只能放在默认配置文件且没有多文档标识的部分。

2.9 加载机制

Spring Boot 中的配置文件是通过 PropertySourceLoader 接口实现的：

```
public interface PropertySourceLoader {

        /**
         * Returns the file extensions that the loader supports (excluding the '.').
         * @return the file extensions
         */
```

```
    String[] getFileExtensions();

    /**
     * Load the resource into one or more property sources. Implementations may either
     * return a list containing a single source, or in the case of a multi-document format
     * such as yaml a source for each document in the resource.
     * @param name the root name of the property source. If multiple documents are loaded
     * an additional suffix should be added to the name for each source loaded.
     * @param resource the resource to load
     * @return a list property sources
     * @throws IOException if the source cannot be loaded
     */
    List<PropertySource<?>> load(String name, Resource resource) throws IOException;

}
```

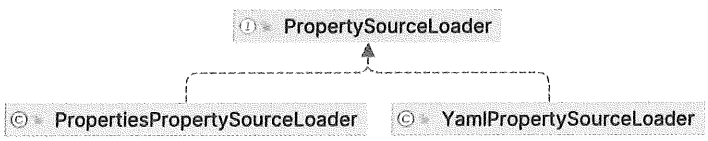
该接口提供了两个抽象方法：

- getFileExtensions：获取文件后缀。
- load：加载配置。

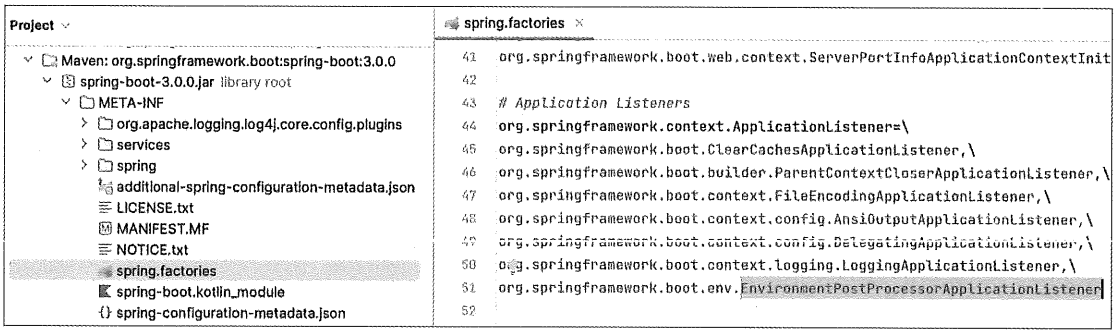
前面的章节中提到的 YamlPropertySourceLoader 就实现了该 PropertySourceLoader 接口，包括它在内，该接口一共有以下两个实现类：

- YamlPropertySourceLoader。
- PropertiesPropertySourceLoader。

类结构如下图所示。



它们分别用来加载 .yaml 和 .properties 配置文件，然后在 Spring Boot 主包 spring-boot-3.0.0.jar 的 /META-INF/spring.factories 自动配置文件中定义了一系列的应用监听器，如下图所示。



其中就有一个 EnvironmentPostProcessorApplicationListener 监听器，它监听了以下三个事件：

- ApplicationEnvironmentPreparedEvent。
- ApplicationPreparedEvent。
- ApplicationFailedEvent。

源码如下所示。

```
@Override
public void onApplicationEvent(ApplicationEvent event) {
    if (event instanceof ApplicationEnvironmentPreparedEvent) {
        onApplicationEnvironmentPreparedEvent((ApplicationEnvironmentPreparedEvent) event);
    }
    if (event instanceof ApplicationPreparedEvent) {
        onApplicationPreparedEvent();
    }
    if (event instanceof ApplicationFailedEvent) {
        onApplicationFailedEvent();
    }
}
```

其中监听的 ApplicationEnvironmentPreparedEvent 事件用于加载配置，具体可以跟踪到 ConfigDataEnvironment#processAndApply 类的方法：

```
static {
    List<ConfigDataLocation> locations = new ArrayList<>();
    locations.add(ConfigDataLocation.of("optional:classpath:/;optional:classpath:
/config/"));
    locations.add(ConfigDataLocation.of("optional:file:./;optional:file:./config/;
optional:file:./config/*/"));
    DEFAULT_SEARCH_LOCATIONS = locations.toArray(new ConfigDataLocation[0]);
```

```
}

void processAndApply() {
    ConfigDataImporter importer = new ConfigDataImporter(this.logFactory,
this.notFoundAction, this.resolvers,
            this.loaders);
    registerBootstrapBinder(this.contributors, null, DENY_INACTIVE_BINDING);

    // 初始化配置处理
    ConfigDataEnvironmentContributors contributors = processInitial(this.contributors, importer);
    ConfigDataActivationContext activationContext = createActivationContext(
            contributors.getBinder(null, BinderOption.FAIL_ON_BIND_TO_INACTIVE_SOURCE));

    // 处理没有 Profile 的配置
    contributors = processWithoutProfiles(contributors, importer, activationContext);
    activationContext = withProfiles(contributors, activationContext);

    // 最后处理带有 Profile 的配置
    contributors = processWithProfiles(contributors, importer, activationContext);
    applyToEnvironment(contributors, activationContext, importer.getLoadedLocations(),
            importer.getOptionalLocations());
}
```

在类源码上方看到了静态初始块中的配置文件的默认搜索路径，然后在 processAndApply 方法中看到这个 contributors 变量被先后定义了三次：

- 初始化配置处理。
- 处理没有 Profile 的配置。
- 最后处理带有 Profile 的配置。

从这个加载顺序就可以看到基于 Profile 配置的优先级了，然后在获取 contributors 的方法中可以一路追踪到调用 PropertySourceLoader 加载配置文件的地方。

既然有监听事件，就有发布事件的地方，可以在 Spring Boot 启动的 run 方法中找到：

```
public ConfigurableApplicationContext run(String... args) {
    long startTime = System.nanoTime();
    DefaultBootstrapContext bootstrapContext = createBootstrapContext();
    ConfigurableApplicationContext context = null;
    configureHeadlessProperty();
    SpringApplicationRunListeners listeners = getRunListeners(args);
    listeners.starting(bootstrapContext, this.mainApplicationClass);
```

```
    try {
        ApplicationArguments applicationArguments = new DefaultApplicationArguments(args);

        // 准备环境
        ConfigurableEnvironment environment = prepareEnvironment(listeners,
bootstrapContext, applicationArguments);
```

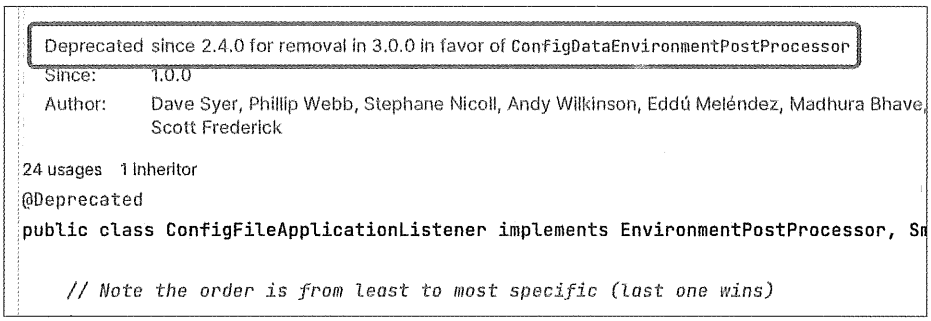
根据准备环境 prepareEnvironment 方法可以一路找到 EventPublishingRunListener# environmentPrepared 方法：

```
public void environmentPrepared(ConfigurableBootstrapContext bootstrapContext,
        ConfigurableEnvironment environment) {
    this.initialMulticaster.multicastEvent(
            new ApplicationEnvironmentPreparedEvent(bootstrapContext, this.application,
this.args, environment));
}
```

事件就是 Spring Boot 启动过程中发出的，所以整个流程就是 Spring Boot 启动时发布事件，然后通过注册的监听器进行监听，监听器监听到事件后就马上加载指定搜索路径的配置文件。

Spring Boot 3.0 新变化：

在 Spring Boot 2.4 之前使用的是 ConfigFileApplicationListener 监听器，而在 Spring Boot 2.4 中将它标识为 @Deprecated 状态了，并且在 Spring Boot 3.0 中正式删除了，如下图所示。



关于自动配置文件的变更：

自动配置文件 /META-INF/spring.factories 在 Spring Boot 2.7 中有了重大变更，这个会在第 3 章 Spring Boot Starter 与自动配置章节中详细介绍。

2.10 配置加密

2.10.1 概述

项目中会涉及各种敏感配置信息，比如各种中间件的连接用户名、密码信息，以及各种第三方的 Key、密钥等，它们有些存储在项目中，有些存储在项目之外，比如配置中心，一般主流的配置中心都支持自动加 / 解密。

如果这些信息存储在项目中，比如基于 Spring Boot 的传统 MVC 项目，所有的代码、配置都几乎在同一个项目中，那么 Spring Boot 项目的配置自然就是 application 配置文件了，如果这些敏感信息直接放在配置文件中，那么肯定是不安全的，甚至在很多行业及领域（比如支付领域）都是不合规的，所以需要保护 Spring Boot 中的敏感配置信息，这也是对一个项目最基本的安全要求。

Spring Boot 并没有对配置的加 / 解密这一块提供标准的解决方案，但是提供了一个 EnvironmentPostProcessor 接口，可用于在应用启动前控制 Spring 环境，所以通过它可以实现对配置在应用启动前进行解密，但是一般很少使用。对于 Spring Boot 应用的配置的加 / 解密可以有以下几种解决方案：

- 使用配置中心（支持自动加 / 解密）。
- 使用数据库机制。
- 使用自定义加 / 解密机制。
- 使用 Jasypt Spring Boot（第三方加 / 解密方案）。

下面的章节会分别介绍这几种方案的使用场景及处理方式，读者可以根据项目不同的需要进行选择。

2.10.2 使用配置中心（支持自动解密）

如果使用了外置的第三方配置中心，那种支持自动解密的中间件，就可以把所有的动态配置和敏感配置信息都存储在配置中心，比如 Spring Cloud 微服务生态中的 Spring Cloud Config 配置中心，使用其自带的加 / 解密机制来保护敏感信息：

```
spring:
  datasource:
  username: '{cipher}t1s293294187a31f35dea15e8bafaf7774532xxcc20d6d6dd0dfa5ae753d6836'
```

这里的密文由 Spring Cloud Config Server 生成，需要实现自动加 / 解密的内容以 {cipher} 开头标识，注意要使用单引号包起来，Spring Boot 配置文件中可以只存储一些无关紧要的应用配置。

2.10.3 使用数据库机制

一种传统的做法是把所有配置信息存储到数据库中，敏感信息以对称加密算法进行加密存储，当 Spring Boot 应用启动的时候，查询数据库配置并加载到内存中，如果是密文则解密后再加载到内存中。

这是最传统的配置管理方法，其实也可以理解为一个原始的、简易的配置中心，只是功能不那么强大而已，因为现在很多配置中心也都是把配置持久化到数据库中进行存储，然后提供一系列的配置管理功能而已。这里的数据库可以是关系型数据库（MySQL、Oracle）、内存数据库（Redis、ZooKeeper）等，这是用得比较多的中间件技术。

2.10.4 使用自定义加 / 解密机制

如果应用只有极少的敏感配置，比如只有数据库连接信息，那么可以考虑使用现有系统中的对称加密算法，再自定义一个数据源类实现自定义的加 / 解密机制，比如我们可以模仿 Spring Cloud 加密机制，先使用系统已有的对称加密算法对数据库连接信息加密：

```
spring:
  datasource:
  username: '{cipher}t1s293294187a31f35dea15e8bafaf7774532xxcc20d6d6dd0dfa5ae753d6836'
```

先排除 Spring Boot 系统自带的数据源自动配置，然后自行组装数据源 Spring Bean，判断获取的配置值是否以 {cipher} 这个标识开头，如果是，则使用系统约定的对称加密算法进行解密，然后设置数据源，比如以下示例代码：

```
@Bean
public DataSource dataSource(){
       DataSource dataSource = new DruidDataSource();

       // 解密
       String username = this.getUsername();
       if (username.startWith('{cipher}')){
              username = Encrypt.decrypt(username, this.getKey()))
       }
       dataSource.setUsername(username);
```

```
        ...

        return dataSource;
}
```

这种手动加 / 解密机制使用简单，不用额外引入任何第三方包，可以满足只有少量敏感配置的应用场景。但如果是 Spring Boot 自动配置的场景，比如数据源自动配置、Redis 自动配置等，在系统启动的时候就会默认加载配置并自动配置，所以需要在 Spring Boot 配置装载之后自动配置之前对配置进行后置的解密处理。

2.10.5 Jasypt Spring Boot

1. 加 / 解密实战

Jasypt Spring Boot 是一个专门为 Spring Boot 项目中的属性提供加密支持的框架，笔者在撰写本章时已经有 2.3K+ 的 Star 数了，还是挺受欢迎的，是现在市面上使用最广泛的 Spring Boot 第三方加 / 解密框架，也是事实上的 Spring Boot 应用加 / 解密标准。

Jasypt Spring Boot 的开源地址为链接 6。

在笔者撰写本章期间，Jasypt Spring Boot 最新只支持 Spring boot 2.x，最新的 Spring Boot 3.0 可能需要等一段时间才能适配，本章也是基于 Spring Boot 2.7.0 进行实战演示的。

Jasypt Spring Boot 有 3 种集成方法：

- 如果开启了 Spring Boot 的自动配置（使用了 @SpringBootApplication 或者 @EnableAutoConfiguration 注解），则只需要添加 jasypt-spring-boot-starter 依赖，这种会在整个 Spring Environment 中启用可加密属性。
- 添加 jasypt-spring-boot 依赖，同时在 Spring 主要配置类上添加 @EnableEncryptableProperties 注解，这种会在整个 Spring Environment 中启用可加密属性。
- 添加 jasypt-spring-boot 依赖，使用 @EncrytablePropertySource 注解声明各个可加密的参数上，这种只适用于独立配置参数的加 / 解密。

一般正常的 Spring Boot 应用都会开启自动配置，再排除个别的自动配置，所以很少会有全部禁用自动配置的情况，不然使用 Spring Boot 的意义不大，这里我们使用第 1 种集成方式进行演示。

首先引入 Jasypt Spring Boot 的依赖和插件：

```
<dependency>
```

```
    <groupId>com.github.ulisesbocchio</groupId>
    <artifactId>jasypt-spring-boot-starter</artifactId>
    <version>3.0.4</version>
</dependency>

<build>
   <plugins>
       <plugin>
           <groupId>com.github.ulisesbocchio</groupId>
           <artifactId>jasypt-maven-plugin</artifactId>
           <version>${jasypt-spring-boot.version}</version>
       </plugin>
   </plugins>
</build>
```

然后添加 jasypt 加密使用的密钥：

```
jasypt:
  encryptor:
  password: G9w0BAQEFAASCBKYwggSiAgEAAoIBAQC
  property:
  prefix: "ENC@["
  suffix: "]"
```

这个 jasypt.encryptor.password 参数是必需的，相当于 Salt（盐），以保证密码的安全性，prefix 和 prefix 是自定义的密码串标识，不配置则默认为 ENC(...)。实现原理可以参考 Jasypt Spring Boot 的自动配置类：

com.ulisesbocchio.jasyptspringbootstarter.JasyptSpringBootAutoConfiguration

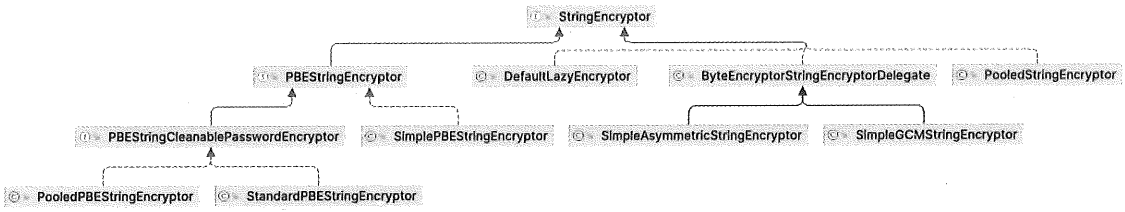
比如在自动配置类中注册了一个 StringEncryptor 字符串加密器，然后笔者基于此提供了一个敏感信息加密测试类：

```
@Slf4j
@SpringBootTest
public class JasyptTest {

  @Autowired
  private StringEncryptor stringEncryptor;

  @Test
  public void encrypt() {
  String usernameEnc = stringEncryptor.encrypt("javastack");
```

```
    String passwordEnc = stringEncryptor.encrypt("javastack.cn");

    log.info("test username encrypt is {}", usernameEnc);
    log.info("test password encrypt is {}", passwordEnc);

    log.info("test username is {}", stringEncryptor.decrypt(usernameEnc));
    log.info("test password is {}", stringEncryptor.decrypt(passwordEnc));
  }
}
```

Spring Boot 单元测试详解请参考第 10 章 Spring Boot 调试与单元测试。

笔者在以上单元测试类中注入了一个 StringEncryptor，其类结构如下图所示。



如果当前没有自定义 StringEncryptor，那么 Jasypt Spring Boot 的自动配置会默认创建一个 StringEncryptor 实例，直接使用即可，其构造器的默认值如下表所示。

Key	Required	Default Value
jasypt.encryptor.password	True	-
jasypt.encryptor.algorithm	False	PBEWITHHMACSHA512ANDAES_256
jasypt.encryptor.key-obtention-iterations	False	1000
jasypt.encryptor.pool-size	False	1
jasypt.encryptor.provider-name	False	SunJCE
jasypt.encryptor.provider-class-name	False	null
jasypt.encryptor.salt-generator-classname	False	org.jasypt.salt.RandomSaltGenerator
jasypt.encryptor.iv-generator-classname	False	org.jasypt.iv.RandomIvGenerator
jasypt.encryptor.string-output-type	False	base64
jasypt.encryptor.proxy-property-sources	False	false
jasypt.encryptor.skip-property-sources	False	empty list

然后运行测试用例，测试结果如下图所示。

```
springboot.jasypt.JasyptTest          : Started JasyptTest in 1.004 seconds (JVM running for 2.257)
springboot.jasypt.Application         : javastack.username = javastack
springboot.jasypt.Application         : javastack.password = javastack.cn
springboot.jasypt.JasyptTest          : test username encrypt is VA058GV86AaUx4wSoCKekKrDJv6tP4eTwWkoCG
springboot.jasypt.JasyptTest          : test password encrypt is 7Y7FVXO5qS4lAx3o5B4atyHW8kGUv0o6903xL2
springboot.jasypt.JasyptTest          : test username is javastack
springboot.jasypt.JasyptTest          : test password is javastack.cn
```

加 / 解密单元测试成功了，如果不想使用测试这种方法生成密文，则可以使用 Maven 插件，这就是前面为什么要加 Maven 插件的原因，使用方式如下：

```
mvn jasypt:encrypt-value -Djasypt.encryptor.password="G9w0BAQEFAASCBKYwggSiAgEAAoIBAQC" -Djasypt.plugin.value="javastack"
```

然后将生成的密文填充到 application 配置文件中：

```
javastack:
  username: ENC@[VA058GV86AaUx4wSoCKekKrDJv6tP4eTwWkoCGHnt6Tb3F7ETJnKAs3QiRkZFD5k]
  password: ENC@[7Y7FVXO5qS4lAx3o5B4atyHW8kGUv0o6903xL2ofTOicdcJw71/m+UeFUVfX0VPM]
```

注意 ENC@[] 这个占位符是上面自定义配置的，写一个程序尝试打印出来：

```
@Slf4j
@SpringBootApplication
public class Application {

    @Value("${javastack.username}")
    private String username;

    @Value("${javastack.password}")
    private String password;

    public static void main(String[] args) {
        SpringApplication.run(Application.class);
    }

    @Bean
    public CommandLineRunner commandLineRunner() {
        return (args) -> {
            log.info("javastack.username = {}", username);
            log.info("javastack.password = {}", password);
        };
    }

}
```

笔者这里写了一个 CommandLineRunner，在系统启动之后将密文的原文打印出来，不需要做任何处理，直接注入、打印就行。应用启动之后查看启动日志，看一下配置是否为明文，如下图所示。

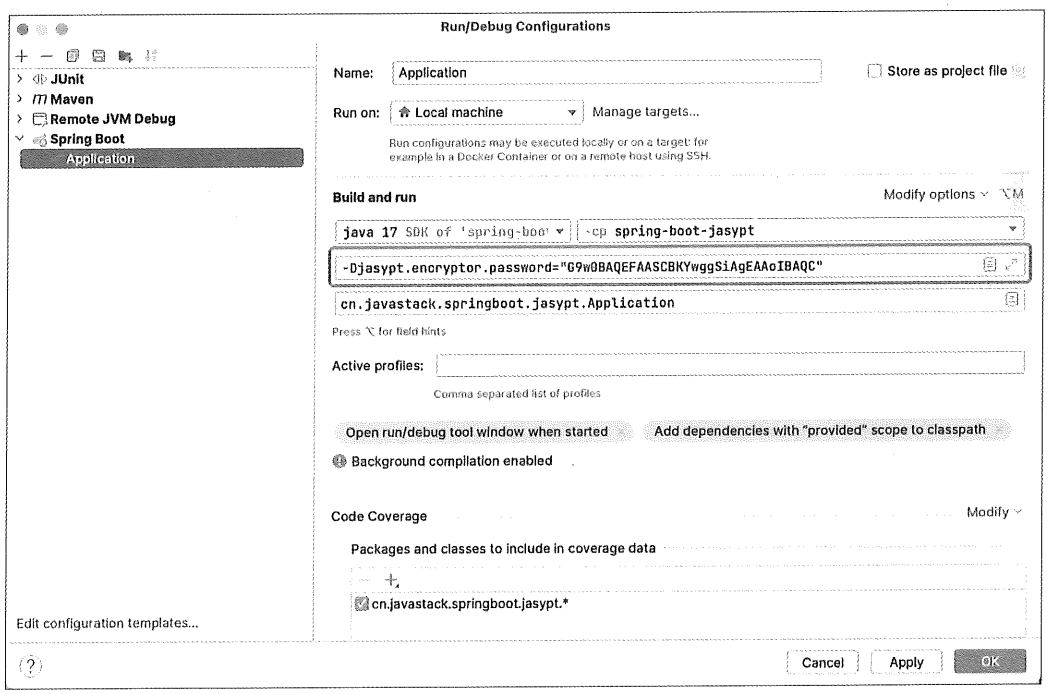
```
c.u.j.c.StringEncryptorBuilder          : Encryptor config not found for property jasypt.
c.u.j.c.StringEncryptorBuilder          : Encryptor config not found for property jasypt.
c.u.j.c.StringEncryptorBuilder          : Encryptor config not found for property jasypt.
c.u.j.c.StringEncryptorBuilder          : Encryptor config not found for property jasypt.
c.j.springboot.jasypt.Application       : Started Application in 1.308 seconds (JVM runni
c.j.springboot.jasypt.Application       : javastack.username = javastack
c.j.springboot.jasypt.Application       : javastack.password = javastack.cn
```

结果正常，配置显示明文了，配置自动解密成功。

CommandLineRunner 的详解介绍请参考第 3 章 Spring Boot Starter 与自动配置。

2. 密钥安全性

前面的章节中笔者把 Jasypt 密钥（password）存放在了 application 配置文件中，这样敏感信息还是在项目代码中，理论上也不是很安全，建议通过命令行参数的方式传入，比如在 IDEA 中可以这样设置 JVM 启动参数，如下图所示。



如果是生产环境，则可以通过命令的方式传入：

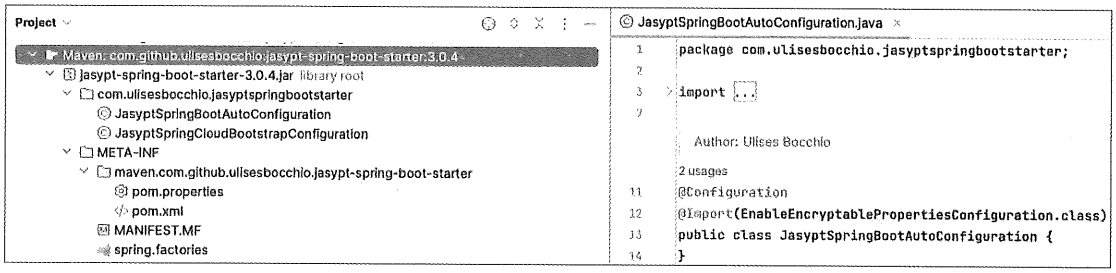
```
java -Djasypt.encryptor.password=password -jar xx.jar
```

甚至可以配置在服务器环境变量中，StringEncryptor可以通过系统参数、配置文件、命令行参数、环境变量等方式进行构造，这样Spring Boot中的配置信息就彻底安全了！

当然，Jasypt Spring Boot的功能远不止如此，实际功能要更强大和复杂，笔者这里只是进行基本介绍及简单的加/解密应用，更多的自定义加/解密需求可以参考官方文档。

3. 实现原理

Jasypt Spring Boot自动配置类的源码如下图所示。



从自动配置类引入的EnableEncryptablePropertiesConfiguration配置类中可以找到Jasypt Spring Boot，它注册了一个Spring后置处理器类EnableEncryptablePropertiesBeanFactoryPost- Processor，它修饰包含在Spring Environment中的所有PropertySource对象，并按照Jasypt的配置约定对属性进行加/解密，如下图所示。

```
@Slf4j
public class EnableEncryptablePropertiesBeanFactoryPostProcessor implements BeanFactoryPostProcessor, Ordered {

    2 usages
    private final ConfigurableEnvironment environment;
    2 usages
    private final EncryptablePropertySourceConverter converter;

    1 usage
    public EnableEncryptablePropertiesBeanFactoryPostProcessor(ConfigurableEnvironment environment, EncryptableP
        this.environment = environment;
        this.converter = converter;
    }

    @Override
    public void postProcessBeanFactory(ConfigurableListableBeanFactory beanFactory) throws BeansException {
        log.info("Post-processing PropertySource instances");
        MutablePropertySources propSources = environment.getPropertySources();
        converter.convertPropertySources(propSources);
    }

    @Override
    public int getOrder() { return Ordered.LOWEST_PRECEDENCE - 100; }
}
```

在 EncryptablePropertySourceConverter 转换类的初始化参数方法中，根据不同的参数类型定义了不同类型的加密参数包装类，如下图所示。

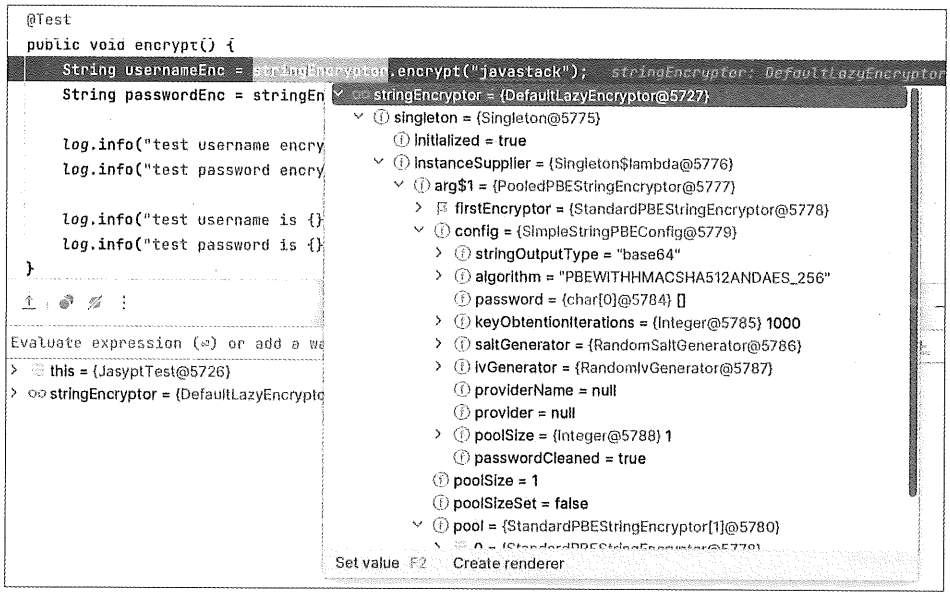
```
/unchecked, rawtypes/
private <T> PropertySource<T> instantiatePropertySource(PropertySource<T> propertySource) {
    PropertySource<T> encryptablePropertySource;
    if (needsProxyAnyway(propertySource)) {
        encryptablePropertySource = proxyPropertySource(propertySource);
    } else if (propertySource instanceof SystemEnvironmentPropertySource) {
        encryptablePropertySource = (PropertySource<T>) new EncryptableSystemEnvironmentPropertySourceW
    } else if (propertySource instanceof MapPropertySource) {
        encryptablePropertySource = (PropertySource<T>) new EncryptableMapPropertySourceWrapper((MapPro
    } else if (propertySource instanceof EnumerablePropertySource) {
        encryptablePropertySource = new EncryptableEnumerablePropertySourceWrapper<>((EnumerableProper
    } else {
        encryptablePropertySource = new EncryptablePropertySourceWrapper<>(propertySource, propertyReso
    }
    return encryptablePropertySource;
}
```

源码有点复杂，根据 *Wrapper 包装类中的获取参数 getProperty 方法，再一路找到了 DefaultPropertyResolver 这个参数解析处理器，如下图所示。

```
public class DefaultPropertyResolver implements EncryptablePropertyResolver {

    3 usages
    private final Environment environment;
    2 usages
    private StringEncryptor encryptor;
    3 usages
    private EncryptablePropertyDetector detector;

    public DefaultPropertyResolver(StringEncryptor encryptor, Environment environment) {
        this(encryptor, new DefaultPropertyDetector(), environment);
    }

    2 usages
    public DefaultPropertyResolver(StringEncryptor encryptor, EncryptablePropertyDetector detector, Envi
        this.environment = environment;
        Assert.notNull(encryptor,  message: "String encryptor can't be null");
        Assert.notNull(detector,  message: "Encryptable Property detector can't be null");
        this.encryptor = encryptor;
        this.detector = detector;
    }

    2 usages
    @Override
    public String resolvePropertyValue(String value) {
        return Optional.ofNullable(value)
                .map(environment::resolvePlaceholders)
                .filter(detector::isEncrypted)
                .map(resolvedValue -> {
                    try {
                        String unwrappedProperty = detector.unwrapEncryptedValue(resolvedValue.trim());
                        String resolvedProperty = environment.resolvePlaceholders(unwrappedProperty);
                        return encryptor.decrypt(resolvedProperty);
                    } catch (EncryptionOperationNotPossibleException e) {
```

它也是注入了 StringEncryptor 这个实例，获取配置时，会在解密后再返回。另外，这个 Resolver 解析处理器也是支持自定义的，感兴趣的读者可以再深入研究。

另外，笔者通过在单元测试类中进行 Debug 调试，可以看到 StringEncryptor 其实是一个 DefaultLazyEncryptor 实例，如下图所示。



这一点在自动配置类中也可以找到，当然也支持自定义的 Encryptor，有需要的读者可以自行定制。

2.11 配置迁移

2.11.1 迁移方案

在升级 Spring Boot 新版本时，某些配置参数可能已经被改名或者删除了，Spring Boot 提供了一个配置迁移依赖，方便开发者快速进行配置迁移，如下所示。

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-properties-migrator</artifactId>
    <scope>runtime</scope>
</dependency>
```

在应用中加入此依赖，就可以分析应用的环境并在启动时打印诊断信息，还可以在运行时进行临时的配置迁移。

笔者使用 Spring Boot 3.0 中的一个变更参数测试了一下，效果如下图所示。

```
The use of configuration keys that have been renamed was found in the environment:

Property source 'Config resource 'class path resource [application.yml]' via location 'optional:classpath:/'':
    Key: management.metrics.export.prometheus.pushgateway.enabled
        Line: 34
        Replacement: management.prometheus.metrics.export.pushgateway.enabled


Each configuration key has been temporarily mapped to its replacement for your convenience. To silence this warning,
```

Spring Boot 会在启动日志中打印需要替换的旧的配置参数和新的配置参数，替换后再重新启动测试即可。

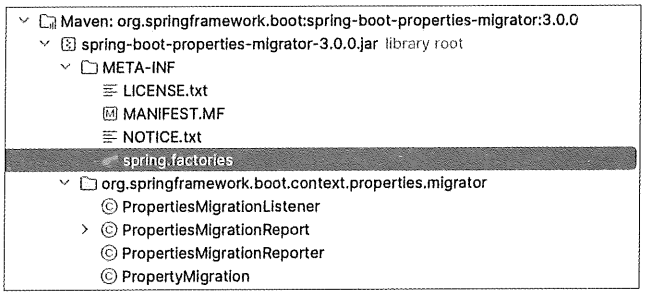
需要注意的是：

（1）在应用启动后再加入 Spring 环境中的配置参数不在配置迁移支持范围之内，比如通过 @PropertySource 注解加载的配置。

（2）应用完成迁移后，建议删除该配置迁移依赖，以免部署到生产环境后影响性能或者出现潜在的问题。

2.11.2 实现原理

spring-boot-properties-migrator 配置迁移依赖包的所有内容如下图所示。



在 spring.factories 自动配置文件中注册了一个 PropertiesMigrationListener 监听器：

```
org.springframework.context.ApplicationListener=\
org.springframework.boot.context.properties.migrator.PropertiesMigrationListener
```

监听器的部分源码如下所示。

```
class PropertiesMigrationListener implements ApplicationListener<SpringApplicationEvent> {

    private static final Log logger = LogFactory.getLog(PropertiesMigrationListener.class);

    private PropertiesMigrationReport report;

    private boolean reported;

    @Override
    public void onApplicationEvent(SpringApplicationEvent event) {
        if (event instanceof ApplicationPreparedEvent preparedEvent) {
            onApplicationPreparedEvent(preparedEvent);
        }
        if (event instanceof ApplicationReadyEvent || event instanceof
ApplicationFailedEvent) {
            logLegacyPropertiesReport();
        }
    }

    ...

}
```

它实现了 ApplicationListener 接口，并监听 SpringApplicationEvent 事件，事件是 Spring Boot 启动过程中发出的。

> Spring Boot 启动事件和监听器机制可以参考第 4 章 Spring Boot 启动过程与扩展应用中的启动事件和监听器一节。

在第一个应用准备事件发出后，监听器先生成配置参数迁移报告 PropertiesMigrationReport，然后在第二个应用就绪 / 失败事件发出后，再通过 PropertiesMigrationReport#getWarningReport 和 PropertiesMigrationReport#getErrorReport 方法获取相对应的报告并输出日志。

第 3 章 Spring Boot Starter 与自动配置

Spring Boot Starter 是 Spring Boot 框架能快速集成并使用其他技术框架的首要条件，它涉及大量的核心技术，本章会介绍 Spring Boot Starter 的方方面面，包括 Starter 的基本介绍、命名规范、分类、自动配置、自定义 Starter 等，这些都是掌握 Spring Boot 框架必不可少的技术要点。

3.1 概述

Starter 可以理解为 Spring Boot 中的一站式集成启动器，它包含了一系列可以集成到应用中的依赖项（dependencies），可以快速一站式集成 Spring 组件及其他框架，而不需要到处找示例代码、依赖包及复杂的组件配置，我们只需要通过简单的参数配置就可以实现“开箱即用”。

一个完整的 Spring Boot Starter 一般需要包含以下组件：

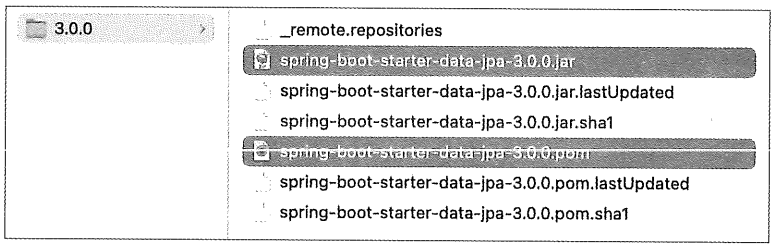
- 完成自动配置的自动配置模块。
- 为自动配置模块提供的所有依赖项。

简而言之，就是添加一个 Starter 应该提供使用该 Starter 所需的一切。比如，我们想使用 Spring JPA 访问数据库，只要加入官方提供的 spring-boot-starter-data-jpa 启动器依赖即可：

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
```

加入依赖后就能完成默认的自动配置并能直接使用，如果有扩展的需要，也可以通过一系列

spring.jpa.* 参数进行额外的自定义配置，下载下来的 spring-boot-starter-data-jpa 依赖坐标内容如下图所示。

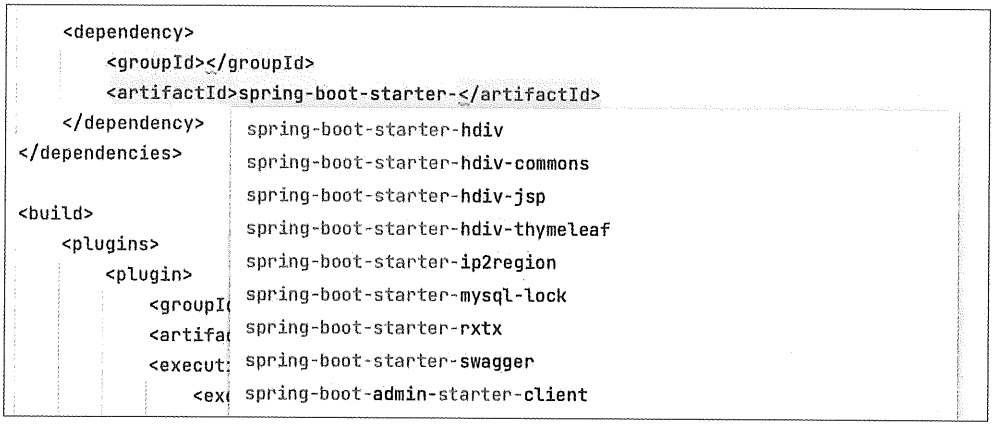


一个 Spring Boot Starter 包含一个 jar 包和一个依赖配置文件，官方 jar 包中没有任何实质内容，官方提供的自动配置类和自动配置逻辑全部存储在另外一个自动配置模块中（spring-boot-autoconfigure），具体内容可见 3.4 节，Starter 存在的意思是通过依赖配置文件导入所有需要的依赖。

3.2 Starter 的命名规范

Spring Boot 官方的 Starter 启动器都是以 spring-boot-starter-* 命名的，* 代表任意一个特定的应用类型，比如 Redis 的 spring-boot-starter-data-redis。而第三方的启动器不能以 spring-boot 开头命名，它们都被 Spring Boot 官方保留，一般一个第三方的 Starter 应该以 *-spring-boot-starter 这样的形式命名，即和官方的命名反过来，比如 druid 连接池：druid-spring-boot-starter。

有了这样的命名规范，就能帮助我们快速找到想要的 Starter 启动器，比如在开发工具编辑器中配置 Maven 依赖时，输入前缀就会弹出各种已有的依赖提示，如下图所示。



这样也能规范第三方的 Starter 启动器的开发和使用。

3.3 Starter 的分类

Spring Boot 官方提供了各种各样的 Starter，几乎涉及了市面上所有主流的技术，不需要我们额外开发，也不需要第三方技术厂商去定制，官方把它们分成了以下三个类别。

3.3.1 application starter

Spring Boot 所有应用级的 Starter 如下表所示。

编号	Starter 名称	Starter 描述
1	spring-boot-starter	核心 Starter，包括自动配置、日志及 YAML 支持等
2	spring-boot-starter-amqp	集成 Spring AMQP 和 Rabbit MQ 的消息队列
3	spring-boot-starter-aop	集成 Spring AOP 和 AspectJ 面向切面编程
4	spring-boot-starter-artemis	集成 Apache Artemis，基于 JMS 的消息队列
5	spring-boot-starter-batch	集成 Spring Batch（批处理）
6	spring-boot-starter-cache	集成 Spring Cache（缓存）
7	spring-boot-starter-data-cassandra	集成 Cassandra（分布式数据库）和 Spring Data Cassandra
8	spring-boot-starter-data-cassandra-reactive	集成 Cassandra（分布式数据库）和 Spring Data Cassandra Reactive
9	spring-boot-starter-data-couchbase	集成 Couchbase（文档型数据库）和 Spring Data Couchbase
10	spring-boot-starter-data-couchbase-reactive	集成 Couchbase（文档型数据库）和 Spring Data Couchbase Reactive
11	spring-boot-starter-data-elasticsearch	集成 Elasticsearch（搜索引擎）和 Spring Data Elasticsearch
12	spring-boot-starter-data-jdbc	集成 Spring Data JDBC
13	spring-boot-starter-data-jpa	集成 Spring Data JPA 结合 Hibernate
14	spring-boot-starter-data-ldap	集成 Spring Data LDAP
15	spring-boot-starter-data-mongodb	集成 MongoDB（文档型数据库）和 Spring Data MongoDB
16	spring-boot-starter-data-mongodb-reactive	集成 MongoDB（文档型数据库）和 Spring Data MongoDB Reactive
17	spring-boot-starter-data-neo4j	集成 Neo4j（图形数据库）和 Spring Data Neo4j
18	spring-boot-starter-data-r2dbc	集成 Spring Data R2DBC

续表

编号	Starter 名称	Starter 描述
19	spring-boot-starter-data-redis	集成 Redis（内存数据库）结合 Spring Data Redis 和 Lettuce 客户端
20	spring-boot-starter-data-redis-reactive	集成 Redis（内存数据库）结合 Spring Data Redis reactive 和 Lettuce 客户端
21	spring-boot-starter-data-rest	集成 Spring Data REST 暴露 Spring Data repositories 并输出 REST 资源
22	spring-boot-starter-freemarker	集成 FreeMarker 视图构建 MVC Web 应用
23	spring-boot-starter-graphql	集成 Spring GraphQL 构建 GraphQL 应用
24	spring-boot-starter-groovy-templates	集成 Groovy 模板视图构建 MVC Web 应用
25	spring-boot-starter-hateoas	集成 Spring MVC 和 Spring HATEOAS 构建超媒体 RESTful Web 应用
26	spring-boot-starter-integration	集成 Spring Integration
27	spring-boot-starter-jdbc	集成 JDBC 结合 HikariCP 连接池
28	spring-boot-starter-jersey	集成 JAX-RS 和 Jersey 构建 RESTful Web 应用，是 spring-boot-starter-web 的一种替代方案
29	spring-boot-starter-jooq	集成 jOOQ 访问 SQL 数据库，是 spring-boot-starter-data-jpa 或者 spring-boot-starter-jdbc 的替代 Starter
30	spring-boot-starter-json	用于读写 JSON
31	spring-boot-starter-mail	集成 Java Mail 和 Spring 框架的邮件发送功能
32	spring-boot-starter-mustache	集成 Mustache 视图构建 Web 应用
33	spring-boot-starter-oauth2-client	集成 Spring Security’s OAuth2/OpenID 连接客户端功能
34	spring-boot-starter-oauth2-resource-server	集成 Spring Security’s OAuth2 资源服务器功能
35	spring-boot-starter-quartz	集成 Quartz 任务调度
36	spring-boot-starter-rsocket	构建 RSocket 客户端和服务端
37	spring-boot-starter-security	集成 Spring Security
38	spring-boot-starter-test	集成 JUnit Jupiter、Hamcrest 和 Mockito 测试 Spring Boot 应用和类库
39	spring-boot-starter-thymeleaf	集成 Thymeleaf 视图构建 MVC Web 应用
40	spring-boot-starter-validation	集成 Java Bean Validation 结合 Hibernate Validator
41	spring-boot-starter-web	集成 Spring MVC 构建 RESTful Web 应用，使用 xa0;Tomcat 作为默认内嵌容器
42	spring-boot-starter-web-services	集成 Spring Web Services

续表

编号	Starter 名称	Starter 描述
43	spring-boot-starter-webflux	集成 Spring Reactive Web 构建 WebFlux 应用
44	spring-boot-starter-websocket	集成 Spring WebSocket 构建 WebSocket 应用

Spring Boot 3.0.0 共有 44 个应用级 Starter，较之前的版本略有变动。Spring Boot 官方自带的 Starter 可以直接拿来使用，用到哪个技术就引用哪个技术的 Spring Boot Starter，没有必要重复造轮子，也不用再手动配置复杂的技术组件了。

如果 Spring Boot 官方没有自带的应用级 Starter，一般第三方的应用框架也会提供自制的 Spring Boot Starter，如 Dubbo、ZooKeeper、MyBatis 等，这样只需几个 Starter 依赖、几行配置参数就能轻松集成并使用各种技术。

当然，除了 Spring Boot 官方和第三方技术的 Starter，使用 Spring Boot 框架的公司一般也会有内部私有定制的 Starter，用于在内部各业务部门快速集成和使用，避免各团队重复造轮子。

3.3.2 production starter

除了上面的应用级 Starter，还有生产级 Starter，如下表所示。

编号	Starter 名称	Starter 描述
1	spring-boot-starter-actuator	集成 Spring Boot Actuator，提供生产功能以帮助开发者监控和管理应用

截至 Spring Boot 3.0.0，Spring Boot 只有这一个生产级 Starter。生产级 Starter 意味着和任何技术、业务没有关系，只要用了 Spring Boot 框架，在生产环境中就能使用，也不是只有在生产环境中才能使用，只是在生产环境中使用才体现了它的意义，毕竟本地和测试环境没有增加监控功能的必要。

3.3.3 technical starter

除了应用级别和生产级别的 Starter，Spring Boot 还包括技术级别的 Starter，用于帮助开发者排除或者替换 Spring Boot 框架内部默认的技术组件，如下表所示。

编号	Starter 名称	Starter 描述
1	spring-boot-starter-jetty	集成 Jetty 作为内嵌的 Servlet 容器，可用于替代 spring-boot- starter-tomcat
2	spring-boot-starter-log4j2	集成 Log4j2 日志框架，可用于替代 spring-boot-starter-logging

续表

编号	Starter 名称	Starter 描述
3	spring-boot-starter-logging	集成 Logback 日志框架，这个也是默认的日志 Starter
4	spring-boot-starter-reactor-netty	集成 Netty 作为内嵌的响应式 HTTP 服务器
5	spring-boot-starter-tomcat	集成 Tomcat 作为内嵌的 Servlet 容器，这个也是默认内嵌的 Servlet 容器，它也被集成 spring-boot-starter-web 启动器中
6	spring-boot-starter-undertow	集成 Undertow 作为内嵌的 Servlet 容器，可用于替代 spring-boot-starter-tomcat

Spring Boot 默认内嵌的 Servlet 容器为 Tomcat，如果想换成 Jetty、Undertow 或者其他容器，又或者想换成其他的日志框架，都可以在上面的这个表格中找到，怎么换可以见后续的章节。

Spring Boot 3.0 共收集了 51 个官方 Spring Boot Starter，当然也不限于这 51 个，随着 Spring Boot 版本的不断升级，后续可能会增加更多的 Starter，当然也有少数 Starter 可能会被删除，相比之前的 2.x 版本，3.0.0 版本中的 Starter 是有减少和调整的，不同版本的 Starter 略有不同。

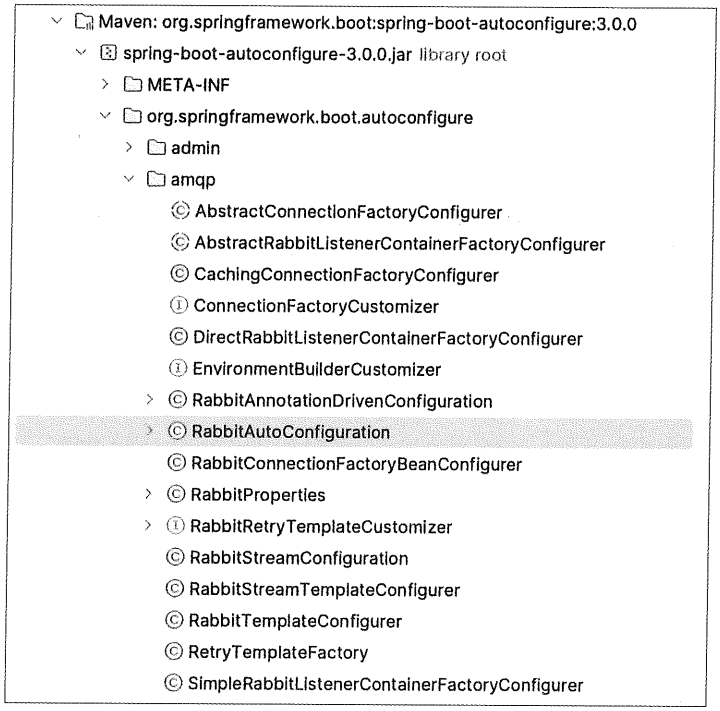
3.4 自动配置

3.4.1 概述

Spring Boot Starter 的核心原理就是自动配置，这也是整个 Spring Boot 框架的核心，开发者只需要按约定提供些许配置参数就能完成各种技术复杂组件的自动组装配置，这正是 Spring Boot 框架能迅速上手使用的原因。

开发者必须搞懂自动配置的原理，以便出现问题时知道怎么去迅速解决，也方便自己在 Spring Boot 提供的默认自动配置不满足要求的情况下定制自己的需求。

Spring Boot 官方所有的自动配置类都是由 spring-boot-autoconfigure 这个模块提供的，如下图所示。



比如在 amqp 子包中定义了 RabbitMQ 自动配置相关的类，其他自带技术的自动配置也都分布在该模块不同的包中，而底层依赖的具体的技术依赖项则需要再通过 Starter 引入，比如想使用 RabbitMQ，则需要引入 RabbitMQ 的启动器：

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-amqp</artifactId>
</dependency>
```

只有引入了对应组件的依赖，Spring Boot 扫描到相关类时才会自动配置，没有引入是不会启用自动配置的，这也是自动配置的魅力，具体内容可见自动配置原理一节。

3.4.2 命名规范

Spring Boot 自动配置类一般以 XxxAutoConfiguration 命名，比如以下几个组件：

- RabbitAutoConfiguration。
- FlywayAutoConfiguration。

- RedisAutoConfiguration。

……

有了这样的命名规范，任何一个技术的自动配置类都可以轻易地被找到，这也是约定大于配置思想的体现。

自动配置类需要注册到 Spring Boot 指定的自动配置文件中，低版本为类路径下的 META-INF/spring.factories 自动配置文件，Spring Boot 2.7 对自动配置类的注册文件路径进行了变更，如下图中第一条所示。

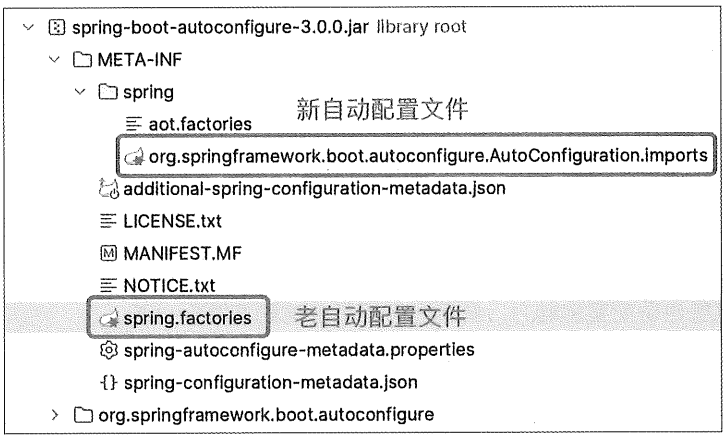
Deprecations in Spring Boot 2.7

- Loading auto-configurations from `spring.factories` is deprecated. See above for more details.
- `DatabaseDriver.GAE`
- Properties under `spring.security.saml2.relyingparty.registration.{id}.identityprovider` have been moved to `spring.security.saml2.relyingparty.registration.{id}.assertingparty`. Using the old property names result in log messages on WARN level on startup.

新的自动配置类注册文件的路径如下：

META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports

以 Spring Boot 3.0.0 为例，如下图所示。



老的自动配置文件 META-INF/spring.factories 虽然在 Spring Boot 2.7 中被宣布废弃了，但是为了向后兼容，Spring Boot 2.7.x 系列版本的自动配置文件仍然可以使用。

另外，自动配置项的编写格式也变了，Spring Boot 2.7+ 中的每一行是一个自动配置类，如下图所示。

org.springframework.boot.autoconfigure.AutoConfiguration.imports

Spring Boot 2.7+

```
org.springframework.boot.autoconfigure.admin.SpringApplicationAdminJmxAutoConfiguration
org.springframework.boot.autoconfigure.aop.AopAutoConfiguration
org.springframework.boot.autoconfigure.amqp.RabbitAutoConfiguration
org.springframework.boot.autoconfigure.batch.BatchAutoConfiguration
org.springframework.boot.autoconfigure.cache.CacheAutoConfiguration
org.springframework.boot.autoconfigure.cassandra.CassandraAutoConfiguration
org.springframework.boot.autoconfigure.context.ConfigurationPropertiesAutoConfiguration
org.springframework.boot.autoconfigure.context.LifecycleAutoConfiguration
org.springframework.boot.autoconfigure.context.MessageSourceAutoConfiguration
org.springframework.boot.autoconfigure.context.PropertyPlaceholderAutoConfiguration
org.springframework.boot.autoconfigure.couchbase.CouchbaseAutoConfiguration
org.springframework.boot.autoconfigure.dao.PersistenceExceptionTranslationAutoConfiguration
org.springframework.boot.autoconfigure.data.cassandra.CassandraDataAutoConfiguration
org.springframework.boot.autoconfigure.data.cassandra.CassandraReactiveDataAutoConfiguration
org.springframework.boot.autoconfigure.data.cassandra.CassandraReactiveRepositoriesAutoConfiguration
org.springframework.boot.autoconfigure.data.cassandra.CassandraRepositoriesAutoConfiguration
org.springframework.boot.autoconfigure.data.couchbase.CouchbaseDataAutoConfiguration
```

spring.factories

Spring Boot 2.6-

```
# Initializers
org.springframework.context.ApplicationContextInitializer=\
org.springframework.boot.autoconfigure.SharedMetadataReaderFactoryContextInitializer,\
org.springframework.boot.autoconfigure.logging.ConditionEvaluationReportLoggingListener

# Application Listeners
org.springframework.context.ApplicationListener=\
org.springframework.boot.autoconfigure.BackgroundPreinitializer

# Environment Post Processors
org.springframework.boot.env.EnvironmentPostProcessor=\
org.springframework.boot.autoconfigure.integration.IntegrationPropertiesEnvironmentPostProcessor

# Auto Configuration Import Listeners
org.springframework.boot.autoconfigure.AutoConfigurationImportListener=\
org.springframework.boot.autoconfigure.condition.ConditionEvaluationReportAutoConfigurationImportListener
```

新的自动配置文件的编写格式确实比之前方便多了，很大程度上能防止配置出错，但同时也有缺点，新的自动配置的文件名也太长了，比较难记，也难以手动编写。

Spring Boot 3.0.0 的新变化：

老的自动配置文件 META-INF/spring.factories 已经正式被废除了，虽然配置文件还在，但仅保留系统级别的组件注册，不再作为应用级别的自动配置类的注册文件了。也就是说，它已经不能再注册自定义的自动配置类了，必须使用新的自动配置文件规范进行注册，具体见下面的源码分析。

3.4.3 自动配置文件的加载原理

在之前的章节中介绍了 Spring Boot 开启自动配置使用的是 @EnableAutoConfiguration 注解，一

般使用组合了它的@SpringBootApplication主注解即可，@EnableAutoConfiguration注解的源码如下：

```
@Target(ElementType.TYPE)
@Retention(RetentionPolicy.RUNTIME)
@Documented
@Inherited
@AutoConfigurationPackage
@Import(AutoConfigurationImportSelector.class)
public @interface EnableAutoConfiguration {

        /**
         * Environment property that can be used to override when auto-configuration is
         * enabled.
         */
        String ENABLED_OVERRIDE_PROPERTY = "spring.boot.enableautoconfiguration";

        /**
         * Exclude specific auto-configuration classes such that they will never be applied.
         * @return the classes to exclude
         */
        Class<?>[] exclude() default {};

        /**
         * Exclude specific auto-configuration class names such that they will never be
         * applied.
         * @return the class names to exclude
         * @since 1.3.0
         */
        String[] excludeName() default {};

}
```

上面最关键的两个注解是：

- @AutoConfigurationPackage：注册需要自动配置的包，如果不指定就是当前注解所在类的包。
- @Import：导入配置类，这个注解在之前的章节介绍过，这里导入的就是其中的ImportSelector接口类。

自动配置@Import注解导入的是AutoConfigurationImportSelector.class类，这也是这个注解的关键所在，它实现了ImportSelector接口：

```
public interface ImportSelector {

        /**
         * Select and return the names of which class(es) should be imported based on
         * the {@link AnnotationMetadata} of the importing @{@link Configuration} class.
         * @return the class names, or an empty array if none
         */
        String[] selectImports(AnnotationMetadata importingClassMetadata);

        /**
         * Return a predicate for excluding classes from the import candidates, to be
         * transitively applied to all classes found through this selector's imports.
         * <p>If this predicate returns {@code true} for a given fully-qualified
         * class name, said class will not be considered as an imported configuration
         * class, bypassing class file loading as well as metadata introspection.
         * @return the filter predicate for fully-qualified candidate class names
         * of transitively imported configuration classes, or {@code null} if none
         * @since 5.2.4
         */
        @Nullable
        default Predicate<String> getExclusionFilter() {
                return null;
        }

}
```

这个注解只有两个方法：

- selectImports（**抽象方法**）：选择要导入配置类。
- Predicate（**接口默认方法**）：用于返回要排除的类。

然后找到 AutoConfigurationImportSelector 类的 selectImports 方法的源码：

```
@Override
public String[] selectImports(AnnotationMetadata annotationMetadata) {
    if (!isEnabled(annotationMetadata)) {
        return NO_IMPORTS;
    }
    AutoConfigurationEntry autoConfigurationEntry=getAutoConfigurationEntry(annotationMetadata);
    return StringUtils.toStringArray(autoConfigurationEntry.getConfigurations());
}
```

再根据 getAutoConfigurationEntry 方法进入 getCandidateConfigurations 方法。

以下是 Spring Boot 2.7 的获取自动配置的源码：

```
protected List<String> getCandidateConfigurations(AnnotationMetadata metadata,
AnnotationAttributes attributes) {
    // 1. 获取自动配置列表（Spring Boot 2.7）
    List<String> configurations = new ArrayList<>(
            SpringFactoriesLoader.loadFactoryNames(getSpringFactoriesLoaderFactoryClass(),
getBeanClassLoader()));

    // 2. 导入候选自动配置列表
    ImportCandidates.load(AutoConfiguration.class, getBeanClassLoader()).forEach
(configurations::add);
    Assert.notEmpty(configurations,
            "No auto configuration classes found in META-INF/spring.factories nor in
META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports. If you"
                    + "are using a custom packaging, make sure that file is correct.");
    return configurations;
}
```

代码注释的1、2处就是获取自动配置列表的关键位置，第1处是Spring Boot 2.7之前的保留逻辑，第2处是导入候选自动配置列表，这是Spring Boot 2.7变更的地方。现在第1处已经在Spring Boot 3.0.0中被彻底移除，第2处正式“转正”替换第1处。以下是 Spring Boot 3.0.0 的获取自动配置的源码：

```
protected List<String> getCandidateConfigurations(AnnotationMetadata metadata,
AnnotationAttributes attributes) {
    // 导入候选自动配置列表
    List<String> configurations = ImportCandidates.load(AutoConfiguration.class,
getBeanClassLoader())
        .getCandidates();

    Assert.notEmpty(configurations,
                    "No auto configuration classes found in "
                    + "META-INF/spring/org.springframework.boot.autoconfigure.
AutoConfiguration.imports. If you "
                    + "are using a custom packaging, make sure that file is correct.");
    return configurations;
}
```

下面分析获取老的自动配置列表（spring.factories）的逻辑，先看第1处获取自动配置列表的逻辑，进入关键的 loadFactoryNames 方法：

```
public static List<String> loadFactoryNames(Class<?> factoryType, @Nullable ClassLoader
classLoader) {
```

```
    return forDefaultResourceLocation(classLoader).loadFactoryNames(factoryType);
}

public static SpringFactoriesLoader forDefaultResourceLocation(@Nullable ClassLoader
classLoader) {
    return forResourceLocation(classLoader, FACTORIES_RESOURCE_LOCATION);
}
```

这里可以看到加载自动配置文件的地方，常量 FACTORIES_RESOURCE_LOCATION 的定义如下：

```
/**
* The location to look for factories.
* <p>Can be present in multiple JAR files.
*/
public static final String FACTORIES_RESOURCE_LOCATION = "META-INF/spring.factories";
```

再进入最后的 forResourceLocation 方法：

```
public static SpringFactoriesLoader forResourceLocation(@Nullable ClassLoader classLoader,
String resourceLocation) {
    Assert.hasText(resourceLocation, "'resourceLocation' must not be empty");
    ClassLoader resourceClassLoader = (classLoader != null) ? classLoader
            : SpringFactoriesLoader.class.getClassLoader();
    Map<String, SpringFactoriesLoader> loaders = SpringFactoriesLoader.cache.get
(resourceClassLoader);
    if (loaders == null) {
        loaders = new ConcurrentReferenceHashMap<>();
        SpringFactoriesLoader.cache.put(resourceClassLoader, loaders);
    }
    SpringFactoriesLoader loader = loaders.get(resourceLocation);
    if (loader == null) {
        Map<String, List<String>> factories = loadFactoriesResource(resourceClassLoader,
resourceLocation);
        loader = new SpringFactoriesLoader(classLoader, factories);
        loaders.put(resourceLocation, loader);
    }
    return loader;
}

static Map<String, List<String>> loadFactoriesResource(ClassLoader classLoader, String
resourceLocation) {
    Map<String, List<String>> result = new LinkedHashMap<>();
```

```
    try {
        Enumeration<URL> urls = classLoader.getResources(resourceLocation);
        while (urls.hasMoreElements()) {
            UrlResource resource = new UrlResource(urls.nextElement());
            Properties properties = PropertiesLoaderUtils.loadProperties(resource);
            properties.forEach((name, value) -> {
                List<String> implementations = result.computeIfAbsent(((String) name).
trim(), key -> new ArrayList<>());
                Arrays.stream(StringUtils.commaDelimitedListToStringArray((String) value))
                    .map(String::trim).forEach(implementations::add);
            });
        }
        result.replaceAll(SpringFactoriesLoader::toDistinctUnmodifiableList);
    }
    catch (IOException ex) {
        throw new IllegalArgumentException("Unable to load factories from location ["
+ resourceLocation + "]", ex);
    }
    return Collections.unmodifiableMap(result);
}
```

即加载 META-INF/spring.factories 自动配置文件中的资源，然后遍历自动配置文件中的所有行并放到一个 Map<String, List<String>> 中返回。

Spring Boot 自动配置包 spring-boot-autoconfigure-3.0.0.jar 中 META-INF/spring.factories 自动配置文件的所有内容如下：

```
# Initializers
org.springframework.context.ApplicationContextInitializer=\
org.springframework.boot.autoconfigure.SharedMetadataReaderFactoryContextInitializer,\
org.springframework.boot.autoconfigure.logging.ConditionEvaluationReportLoggingListener

# Application Listeners
org.springframework.context.ApplicationListener=\
org.springframework.boot.autoconfigure.BackgroundPreinitializer

# Environment Post Processors
org.springframework.boot.env.EnvironmentPostProcessor=\
org.springframework.boot.autoconfigure.integration.IntegrationPropertiesEnvironmentPostProcessor

# Auto Configuration Import Listeners
org.springframework.boot.autoconfigure.AutoConfigurationImportListener=\
org.springframework.boot.autoconfigure.condition.ConditionEvaluationReportAutoConfigura-
tionImportListener
```

```
# Auto Configuration Import Filters
org.springframework.boot.autoconfigure.AutoConfigurationImportFilter=\
org.springframework.boot.autoconfigure.condition.OnBeanCondition,\
org.springframework.boot.autoconfigure.condition.OnClassCondition,\
org.springframework.boot.autoconfigure.condition.OnWebApplicationCondition

# Failure analyzers
org.springframework.boot.diagnostics.FailureAnalyzer=\
org.springframework.boot.autoconfigure.data.redis.RedisUrlSyntaxFailureAnalyzer,\
org.springframework.boot.autoconfigure.diagnostics.analyzer.NoSuchBeanDefinitionFailureAnalyzer,\
org.springframework.boot.autoconfigure.flyway.FlywayMigrationScriptMissingFailureAnalyzer,\
org.springframework.boot.autoconfigure.jdbc.DataSourceBeanCreationFailureAnalyzer,\
org.springframework.boot.autoconfigure.jdbc.HikariDriverConfigurationFailureAnalyzer,\
org.springframework.boot.autoconfigure.jooq.NoDslContextBeanFailureAnalyzer,\
org.springframework.boot.autoconfigure.r2dbc.ConnectionFactoryBeanCreationFailureAnalyzer,\
org.springframework.boot.autoconfigure.r2dbc.MissingR2dbcPoolDependencyFailureAnalyzer,\
org.springframework.boot.autoconfigure.r2dbc.MultipleConnectionPoolConfigurationsFailureAnalzyer,\
org.springframework.boot.autoconfigure.r2dbc.NoConnectionFactoryBeanFailureAnalyzer

# Template availability providers
org.springframework.boot.autoconfigure.template.TemplateAvailabilityProvider=\
org.springframework.boot.autoconfigure.freemarker.FreeMarkerTemplateAvailabilityProvider,\
org.springframework.boot.autoconfigure.mustache.MustacheTemplateAvailabilityProvider,\
org.springframework.boot.autoconfigure.groovy.template.GroovyTemplateAvailabilityProvider,\
org.springframework.boot.autoconfigure.thymeleaf.ThymeleafTemplateAvailabilityProvider,\
org.springframework.boot.autoconfigure.web.servlet.JspTemplateAvailabilityProvider

# DataSource initializer detectors
org.springframework.boot.sql.init.dependency.DatabaseInitializerDetector=\
org.springframework.boot.autoconfigure.flyway.FlywayMigrationInitializerDatabaseInitializerDetector

# Depends on database initialization detectors
org.springframework.boot.sql.init.dependency.DependsOnDatabaseInitializationDetector=\
org.springframework.boot.autoconfigure.batch.JobRepositoryDependsOnDatabaseInitializationDetector,\
org.springframework.boot.autoconfigure.quartz.SchedulerDependsOnDatabaseInitializationDetector,\
org.springframework.boot.autoconfigure.session.JdbcIndexedSessionRepositoryDependsOnDa-
tabaseInitializationDetector
```

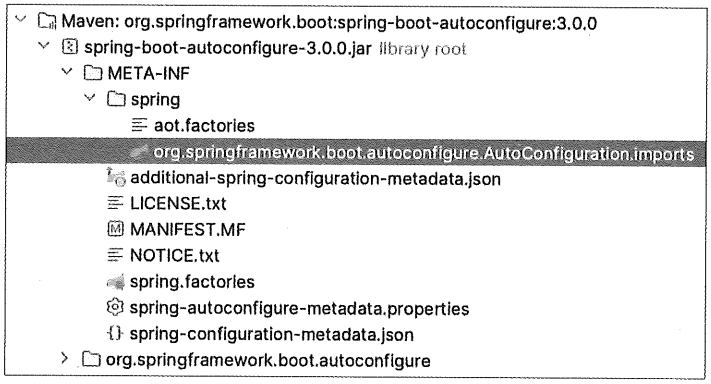
其中配置了各种初始化器、监听器等，这些初始化器和监听器都会在 Spring Boot 启动时完成自动配置。

再看一下获取新的自动配置列表的逻辑，进入 getCandidateConfigurations 方法，再进入导入候

选自动配置列表的 ImportCandidates#load 方法：

```
private static final String LOCATION = "META-INF/spring/%s.imports";

public static ImportCandidates load(Class<?> annotation, ClassLoader classLoader) {
    Assert.notNull(annotation, "'annotation' must not be null");
    ClassLoader classLoaderToUse = decideClassloader(classLoader);
    String location = String.format(LOCATION, annotation.getName());
    Enumeration<URL> urls = findUrlsInClasspath(classLoaderToUse, location);
    List<String> importCandidates = new ArrayList<>();
    while (urls.hasMoreElements()) {
        URL url = urls.nextElement();
        importCandidates.addAll(readCandidateConfigurations(url));
    }
    return new ImportCandidates(importCandidates);
}
```

如以上源代码所示，最新的自动配置逻辑是加载 META-INF/spring/%s.imports 配置文件中的自动配置，这个 %s 是占位符，指的是 @AutoConfiguration 注解的类的全路径名称，如下图所示。



该自动配置文件的内容如下：

```
org.springframework.boot.autoconfigure.admin.SpringApplicationAdminJmxAutoConfiguration
org.springframework.boot.autoconfigure.aop.AopAutoConfiguration
org.springframework.boot.autoconfigure.amqp.RabbitAutoConfiguration
org.springframework.boot.autoconfigure.batch.BatchAutoConfiguration
org.springframework.boot.autoconfigure.cache.CacheAutoConfiguration
org.springframework.boot.autoconfigure.cassandra.CassandraAutoConfiguration
org.springframework.boot.autoconfigure.context.ConfigurationPropertiesAutoConfiguration
org.springframework.boot.autoconfigure.context.LifecycleAutoConfiguration
org.springframework.boot.autoconfigure.context.MessageSourceAutoConfiguration
```

```
org.springframework.boot.autoconfigure.context.PropertyPlaceholderAutoConfiguration
... Spring Boot 3.0.0 共注册了 142 个自动配置类
```

可以看到，编写格式确实简单了，一些应用级别的自动配置类已经全移到这个新配置文件中了，而老的自动配置文件（spring.factories）还保留了一些系统级别的组件。

3.4.4 自动配置原理

自动配置文件被加载后，就会注册里面提供的自动配置类了，比如 3.4.3 节中的下面这条：

org.springframework.boot.autoconfigure.jdbc.DataSourceAutoConfiguration

这是一个数据源的自动配置类，其核心源码如下：

```
@AutoConfiguration(before = SqlInitializationAutoConfiguration.class)
@ConditionalOnClass({ DataSource.class, EmbeddedDatabaseType.class })
@ConditionalOnMissingBean(type = "io.r2dbc.spi.ConnectionFactory")
@EnableConfigurationProperties(DataSourceProperties.class)
@Import(DataSourcePoolMetadataProvidersConfiguration.class)
public class DataSourceAutoConfiguration {

    @Configuration(proxyBeanMethods = false)
    @Conditional(EmbeddedDatabaseCondition.class)
    @ConditionalOnMissingBean({ DataSource.class, XADataSource.class })
    @Import(EmbeddedDataSourceConfiguration.class)
    protected static class EmbeddedDatabaseConfiguration {

    }

    @Configuration(proxyBeanMethods = false)
    @Conditional(PooledDataSourceCondition.class)
    @ConditionalOnMissingBean({ DataSource.class, XADataSource.class })
    @Import({ DataSourceConfiguration.Hikari.class, DataSourceConfiguration.Tomcat.class,
        DataSourceConfiguration.Dbcp2.class, DataSourceConfiguration.OracleUcp.class,
        DataSourceConfiguration.Generic.class, DataSourceJmxConfiguration.class })
    protected static class PooledDataSourceConfiguration {

    }
    ...
}
```

可以看到，自动配置类是由一系列注解组成的。

1. @AutoConfiguration 注解

自动配置注解 @AutoConfiguration 是 Spring Boot 2.7 中新增的，该注解的源码如下：

```
@Target(ElementType.TYPE)
@Retention(RetentionPolicy.RUNTIME)
@Documented
@Configuration(proxyBeanMethods = false)
@AutoConfigureBefore
@AutoConfigureAfter
public @interface AutoConfiguration {

    /**
     * Explicitly specify the name of the Spring bean definition associated with the
     * {@code @AutoConfiguration} class. If left unspecified (the common case), a bean
     * name will be automatically generated.
     * <p>
     * The custom name applies only if the {@code @AutoConfiguration} class is picked up
     * via component scanning or supplied directly to an
     * {@link AnnotationConfigApplicationContext}. If the {@code @AutoConfiguration} class
     * is registered as a traditional XML bean definition, the name/id of the bean element
     * will take precedence.
     * @return the explicit component name, if any (or empty String otherwise)
     * @see AnnotationBeanNameGenerator
     */
    @AliasFor(annotation = Configuration.class)
    String value() default "";

    /**
     * The auto-configure classes that should have not yet been applied.
     * @return the classes
     */
    @AliasFor(annotation = AutoConfigureBefore.class, attribute = "value")
    Class<?>[] before() default {};

    /**
     * The names of the auto-configure classes that should have not yet been applied.
     * @return the class names
     */
    @AliasFor(annotation = AutoConfigureBefore.class, attribute = "name")
    String[] beforeName() default {};

    /**
     * The auto-configure classes that should have already been applied.
```

```
     * @return the classes
     */
    @AliasFor(annotation = AutoConfigureAfter.class, attribute = "value")
    Class<?>[] after() default {};

    /**
     * The names of the auto-configure classes that should have already been applied.
     * @return the class names
     */
    @AliasFor(annotation = AutoConfigureAfter.class, attribute = "name")
    String[] afterName() default {};

}
```

自动配置注解 @AutoConfiguration 组合了以下三个注解：

- @Configuration(proxyBeanMethods = false)：配置类注解，并不代理 @Bean 方法。
- @AutoConfigureBefore：自动配置在 XX 配置之前。
- @AutoConfigureAfter：自动配置在 XX 配置之后。

这个注解是专门为自动配置定制的专用注解，其实就是用来代替之前的 @Configuration、@AutoConfigureAfter 和 @AutoConfigureBefore 三个注解，通过这三个注解共同完成了 @Configuration 配置类，以及自动配置类需要在指定的自动配置类完成配置之前或者之后再自动配置。

之前三个注解用起来确实比较烦琐，现在统一使用一个注解，自动配置类有了它自己的注解，这样也可以用来区分用 @Configuration 标识的普通配置类。

所以，DataSourceAutoConfiguration 自动配置类上的 @AutoConfiguration(before=SqlInitialization- AutoConfiguration.class) 就表示这是一个自动配置类，需要在 SqlInitializationAuto Configuration 自动配置类配置之前进行配置。

2. @ConditionalOn*

在自动配置类中还有各种 @ConditionalOn* 的注解，这是一种条件注解，表示在满足指定条件时才会进行自动配置，这也是 Spring Boot 能实现自动配置的核心注解。

以第一个注解 @ConditionalOnClass({ DataSource.class, EmbeddedDatabaseType.class }) 为例，它表示如果类路径下有那两个类才开启这个类的自动配置，否则不开启自动配置。

这个注解的源码如下：

```
@Target({ ElementType.TYPE, ElementType.METHOD })
```

```
@Retention(RetentionPolicy.RUNTIME)
@Documented
@Conditional(OnClassCondition.class)
public @interface ConditionalOnClass {

        /**
         * The classes that must be present. Since this annotation is parsed by loading class
         * bytecode, it is safe to specify classes here that may ultimately not be on the
         * classpath, only if this annotation is directly on the affected component and
         * <b>not</b> if this annotation is used as a composed, meta-annotation. In order to
         * use this annotation as a meta-annotation, only use the {@link #name} attribute.
         * @return the classes that must be present
         */
        Class<?>[] value() default {};

        /**
         * The classes names that must be present.
         * @return the class names that must be present.
         */
        String[] name() default {};

}
```

它组合了一个最基础的 @Conditional(OnClassCondition.class) 条件注解，并指定了这个 @ConditionalOnClass 条件注解的条件类为 OnClassCondition 类，对应的条件类需要实现 Condition 接口。

基础条件注解 @Conditional 最初是在 Spring 4 中引进的，然后 Spring Boot 基于它组装了各种子条件注解，了解了上面介绍的 @ConditionalOnClass 条件注解，其他条件注解也容易理解，原理是一样的。

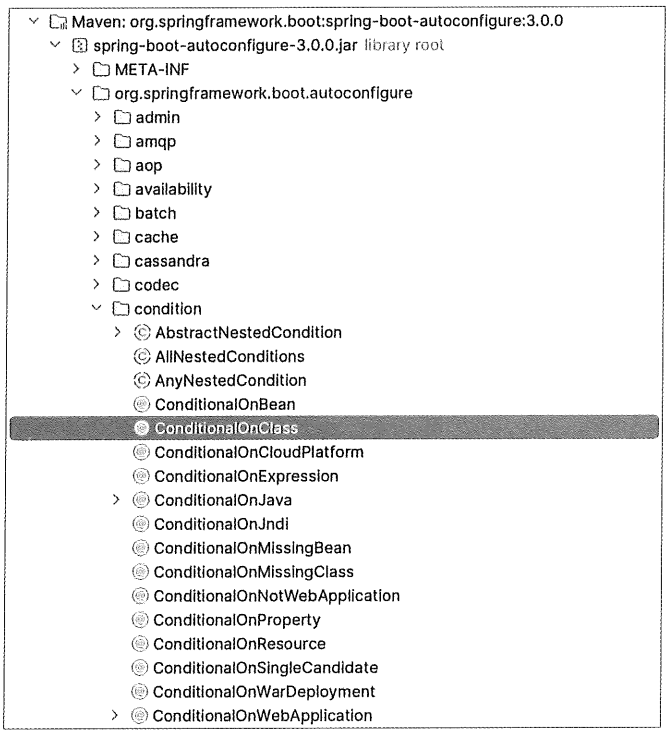
更多的条件注解如下表所示。

注解	描述	条件类
@Conditional	基础条件注解	自由指定
@ConditionalOnClass	类路径下有指定的类才开启配置	OnClassCondition
@ConditionalOnMissingClass	类路径下没有指定的类才开启配置	OnClassCondition
@ConditionalOnBean	Spring 容器中有指定的 Bean 才开启配置	OnBeanCondition
@ConditionalOnMissingBean	Spring 容器中没有指定的 Bean 才开启配置	OnBeanCondition
@ConditionalOnProperty	Spring 环境中有指定的参数才开启配置	OnPropertyCondition

续表

注解	描述	条件类
@ConditionalOnExpression	指定的 SpEL 表达式结果为 true 时才开启配置	OnExpressionCondition
@ConditionalOnResource	类路径下有指定的资源才开启配置	OnResourceCondition
@ConditionalOnWebApplication	是指定的 Web 应用类型才开启自动配置	OnWebApplicationCondition
@ConditionalOnJava	是指定的 JDK 版本才开启配置	OnJavaCondition
@ConditionalOnNotWebApplication	不是指定的 Web 应用类型才开启自动配置	OnWebApplicationCondition
@ConditionalOnCloudPlatform	是指定的云平台才开启配置	OnCloudPlatformCondition

以上总结的条件注解覆盖了大部分使用场景，不同的版本可能会有一些变更，更多、更新的条件注解可以查看 Spring Boot 自动配置包下的 condition 目录，如下图所示。



了解了以上各种注解，再回过来看上面的数据源的自动配置源码就能理解了，无非就是根据各种条件开启自动配置而已，比如数据源需要自动配置，类路径下必须得有数据源的相关类才能进行配置，而这些相关的类都是通过 Starter 引入的。

3.4.5 自动配置报告

Spring Boot 提供了查看自动配置报告的方法，包括激活的和未激活的自动配置，如果不确定自动配置类是否被加载了，则可以通过查看自动配置报告的方式来定位问题。

自动配置报告可以在应用的 debug 模式下查看：

```
$ java -jar xx.jar –debug
```

或者：

```
$ java -jar xx.jar –Ddebug
```

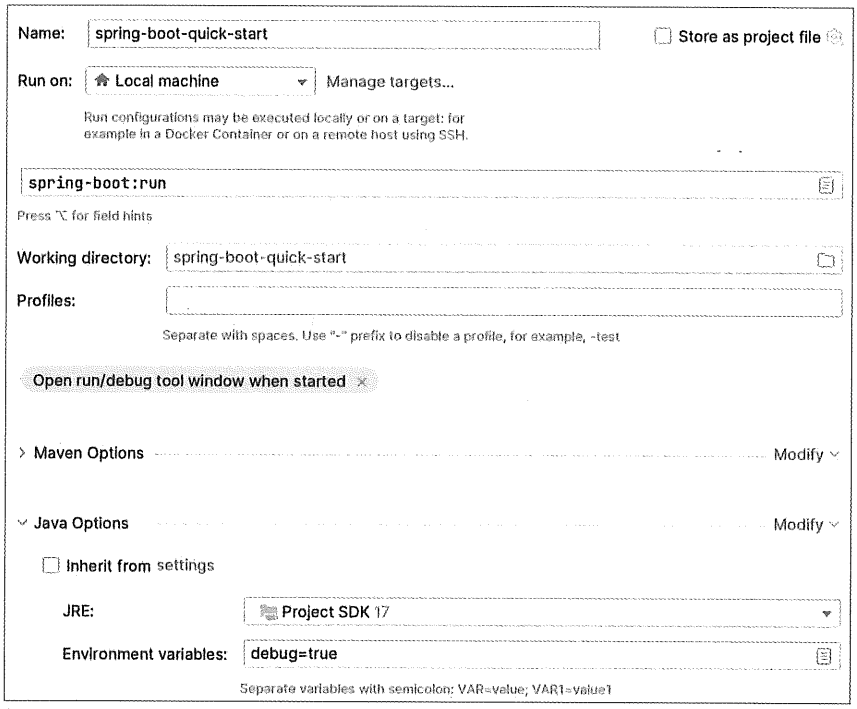
如果在 IDE 中启动应用，则可以直接在应用配置文件中指定：

```
debug: true
```

或者在启动命令行参数中指定：

```
spring-boot:run -Dspring-boot.run.arguments="--debug"
```

以及在环境变量中指定，如下图所示。



方法很多，只要能设置到 Spring 环境参数中就行。另外，如果集成了 spring-boot-starter-actuator 依赖，那么通过 conditions 端点也可以查看，这个 actuator 会在后面的章节介绍。

启动应用后会在控制台看到以下自动配置报告信息：

```
============================
CONDITIONS EVALUATION REPORT
============================

Positive matches:
-----------------
 AopAutoConfiguration matched:
    - @ConditionalOnProperty (spring.aop.auto=true) matched (OnPropertyCondition)

 AopAutoConfiguration.ClassProxyingConfiguration matched:
    - @ConditionalOnMissingClass did not find unwanted class 'org.aspectj.weaver.Advice'
(OnClassCondition)
    - @ConditionalOnProperty (spring.aop.proxy-target-class=true) matched
(OnPropertyCondition)

 DispatcherServletAutoConfiguration matched:
    - @ConditionalOnClass found required class 'org.springframework.web.servlet.
DispatcherServlet' (OnClassCondition)
    - found 'session' scope (OnWebApplicationCondition)

Negative matches:
-----------------

 AopAutoConfiguration.AspectJAutoProxyingConfiguration:
    Did not match:
       - @ConditionalOnClass did not find required class 'org.aspectj.weaver.Advice'
(OnClassCondition)

 ArtemisAutoConfiguration:
    Did not match:
       - @ConditionalOnClass did not find required class 'jakarta.jms.ConnectionFactory'
(OnClassCondition)
 ...

Exclusions:
-----------

  None
```

```
Unconditional classes:
----------------------

   org.springframework.boot.autoconfigure.context.ConfigurationPropertiesAutoConfiguration

   org.springframework.boot.autoconfigure.context.LifecycleAutoConfiguration

   org.springframework.boot.autoconfigure.context.PropertyPlaceholderAutoConfiguration

   ...
```

下面是报告中几个项目的说明：

- Positive matches：已经启用的自动配置。
- Negative matches：未启用的自动配置。
- Exclusions：被排除的自动配置。
- Unconditional classes：没有条件的配置类。

知道了这个自动配置报告，在后续的章节中就可以派上用场了。

3.4.6 排除自动配置

Spring Boot 提供的自动配置的功能非常强大，在某些情况下，自动配置的功能可能不符合我们的需求，需要我们自定义配置。比如数据源，不想使用 Spring Boot 默认提供的 DataSourceAutoConfiguration 自动配置，完全想自己创建自定义配置，这时就需要先排除 / 禁用默认的数据源自动配置类。

不同的使用情况，排除自动配置的方法也不一样，但也可以有统一的排除方案。

1. 使用 @SpringBootApplication 注解

使用 @SpringBootApplication 注解的时候，可以使用 exclude 属性排除指定的类：

```
@SpringBootApplication(exclude = { DataSourceAutoConfiguration.class })
public class Application {
    // ...
}
```

也可以使用 excludeName 属性排除指定的类名全路径：

```
@SpringBootApplication(excludeName = { "org.springframework.boot.autoconfigure.jdbc.
DataSourceAutoConfiguration" })
public class Application {
    // ...
}
```

2. 使用 @EnableAutoConfiguration 注解

独立使用 @EnableAutoConfiguration 注解的时候，用法和 @SpringBootApplication 注解是一样的，因为前者就是组合使用了后者的功能。

```
@...
@EnableAutoConfiguration(exclude = { DataSourceAutoConfiguration.class })
public class Application {
    // ...
}
```

也可以使用 excludeName 属性排除指定的类名全路径：

```
@...
@EnableAutoConfiguration(excludeName = { "org.springframework.boot.autoconfigure.jdbc.
DataSourceAutoConfiguration" })
public class Application {
    // ...
}
```

3. 使用 @SpringCloudApplication 注解

使用 Spring Cloud 和 @SpringCloudApplication 注解的时候，Spring Cloud 必须建立在 Spring Boot 应用之上，所以直接使用 @EnableAutoConfiguration 注解排除即可：

```
@...
@EnableAutoConfiguration(exclude = { DataSourceAutoConfiguration.class })
@SpringCloudApplication
public class Application {
    // ...
}
```

但是，这个 @SpringCloudApplication 注解已经在 3.0.1 版本中废除了，该源码如下：

```
/**
* @author Spencer Gibb
* @deprecated This annotation has been deprecated as of the 3.0.1 release.
```

```
 * <code>@EnableDiscoveryClient</code> is no longer needed, discovery client
 * implementations are enabled as long as an implementation is on the classpath.
 * <code>@EnableCircuitBreaker</code> is no longer used now that Hystrix has been removed
 * from Spring Cloud.
 */
@Deprecated
@Target(ElementType.TYPE)
@Retention(RetentionPolicy.RUNTIME)
@Documented
@Inherited
@SpringBootApplication
@EnableDiscoveryClient
public @interface SpringCloudApplication {

}
```

因为它组合了 @SpringBootApplication 和 @EnableDiscoveryClient 注解，现在的逻辑是只要客户端实现类在类路径上就会自动启用服务发现，所以组合的 @EnableDiscoveryClient 注解就不再需要了，只组合一个 @SpringBootApplication 注解就没意义了。

4. 使用统一的排除方案

在应用配置文件中指定参数 spring.autoconfigure.exclude 统一排除：

```
spring:
  autoconfigure:
    exclude:
      - org.springframework.boot.autoconfigure.jdbc.DataSourceAutoConfiguration
```

不管是 Spring Boot 应用还是 Spring Cloud 应用，又或者使用的哪个注解，这种配置方案都能解决问题。

最后，具体使用哪种方案可以根据实际应用场景判断，被排除后的自动配置类会在应用运行时的自动配置报告中显示，比如：

```
...

Exclusions:
-----------

  org.springframework.boot.autoconfigure.jdbc.DataSourceAutoConfiguration

...
```

3.4.7 替换自动配置

自动配置是非侵入式的，我们完全可以自定义配置来替换 Spring Boot 默认自动配置中的部分组件配置，我们再来看一下之前的数据源的自动配置类源码，再进入导入的 Hikari 连接池 DataSourceConfiguration.Hikari.class 源码：

```
@Configuration(proxyBeanMethods = false)
@ConditionalOnClass(HikariDataSource.class)
@ConditionalOnMissingBean(DataSource.class)
@ConditionalOnProperty(name = "spring.datasource.type", havingValue = "com.zaxxer.
hikari.HikariDataSource",
        matchIfMissing = true)
static class Hikari {

    @Bean
    @ConfigurationProperties(prefix = "spring.datasource.hikari")
    HikariDataSource dataSource(DataSourceProperties properties) {
        HikariDataSource dataSource = createDataSource(properties, HikariDataSource.class);
        if (StringUtils.hasText(properties.getName())) {
            dataSource.setPoolName(properties.getName());
        }
        return dataSource;
    }

}
```

每个自动配置类、方法上都有一个 @ConditionalOnMissingBean 注解，Spring Boot 应用自己创建的 @Bean 会优先注册，如果应用有自己的 @Bean，则默认自动配置类中的 @Bean 就不会再重复注册。

所以，如果 Spring Boot 提供的默认配置不满足要求，那么完全可以在应用中提供自定义的配置来替换默认的配置。比如，如果使用了 Hikari 连接池，应用注册了一个自定义的 DataSource，那么连接中的 HikariDataSource 就不会重复注册，这就是 @ConditionalOnMissingBean 注解的意义。

3.5 邮件 Starter

3.5.1 概述

前面了解了 Starter 的基本概念、命名规范、分类，以及自动配置原理，也介绍了 Spring Boot

内置的各种各样的 Starter，本节介绍 spring-boot-starter-mail 邮件启动器的集成与应用。

Spring 框架提供了一个发送邮件的抽象和实现：

org.springframework.mail.javamail.JavaMailSender

org.springframework.mail.javamail.JavaMailSenderImpl

没有 Spring Boot 之前，开发者需要手动构建 JavaMailSenderImpl 实例类，而有了 Spring Boot，就没有必要手工构建了，Spring Boot 提供了一个邮件自动配置类：

org.springframework.boot.autoconfigure.mail.MailSenderAutoConfiguration

它同样被注册在新的 org.springframework.boot.autoconfigure.AutoConfiguration.imports 自动配置文件中，该自动配置类的源码如下：

```
@AutoConfiguration
@ConditionalOnClass({ MimeMessage.class, MimeType.class, MailSender.class })
@ConditionalOnMissingBean(MailSender.class)
@Conditional(MailSenderCondition.class)
@EnableConfigurationProperties(MailProperties.class)
@Import({ MailSenderJndiConfiguration.class, MailSenderPropertiesConfiguration.class })
public class MailSenderAutoConfiguration {

    /**
     * Condition to trigger the creation of a {@link MailSender}. This kicks in if either
     * the host or jndi name property is set.
     */
    static class MailSenderCondition extends AnyNestedCondition {

        MailSenderCondition() {
            super(ConfigurationPhase.PARSE_CONFIGURATION);
        }

        @ConditionalOnProperty(prefix = "spring.mail", name = "host")
        static class HostProperty {

        }

        @ConditionalOnProperty(prefix = "spring.mail", name = "jndi-name")
        static class JndiNameProperty {

        }

    }

}
```

自动配置类上使用了各种 @Conditional* 注解，在保证默认自动配置机制的情况下，也会在有自定义配置的情况下忽略默认配置。邮件自动配置类使用 @Import 注解引入了 JNDI 和 Properties 两种配置注册方式，在引入的其中一个 MailSenderPropertiesConfiguration 配置类中可以看到自动构建的 JavaMailSenderImpl 实例：

```
@Configuration(proxyBeanMethods = false)
@ConditionalOnProperty(prefix = "spring.mail", name = "host")
class MailSenderPropertiesConfiguration {

        @Bean
        @ConditionalOnMissingBean(JavaMailSender.class)
        JavaMailSenderImpl mailSender(MailProperties properties) {
                JavaMailSenderImpl sender = new JavaMailSenderImpl();
                applyProperties(properties, sender);
                return sender;
        }

        private void applyProperties(MailProperties properties, JavaMailSenderImpl sender) {

          ...

    }

}
```

对应的参数绑定类为 MailProperties，需要 spring.mail.host 配置参数及 spring-boot-starter-mail 依赖，略微配置 spring.mail.* 自定义参数就能自动注册这个 Bean，然后就可以在应用中直接注入 JavaMailSender 并使用了，没有必要重复造轮子。

3.5.2 发邮件实践

首先在 Maven 的 pom.xml 配置文件中加入 spring-boot-starter-mail 邮件启动器依赖：

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-mail</artifactId>
</dependency>
```

然后在应用的 application.properties 配置文件中加入以下邮件自动配置参数：

```
spring:
```

```
  mail:
    host: smtp.exmail.qq.com
    username: xxx@xxx.com
    password: xxx
    properties:
      "[mail.smtp.socketFactory.class]": javax.net.ssl.SSLSocketFactory
      "[mail.smtp.socketFactory.fallback]": false
      "[mail.smtp.socketFactory.port]": 465
      "[mail.smtp.connectiontimeout]": 5000
      "[mail.smtp.timeout]": 3000
      "[mail.smtp.writetimeout]": 5000

mail:
  from: xxx@xxx.com
  personal: 栈长
  bcc: xxx@xxx.com
  subject: Spring Boot 发邮件测试主题
```

其中 xxx 为敏感处理内容，需要替换为真实的邮件信息。

笔者写了一个简单的发送邮件的小例子，邮件发送成功后返回 true，若发送失败则返回 false。

```
@Slf4j
@RequiredArgsConstructor
@RestController
public class EmailController {

    private final JavaMailSender javaMailSender;

    private final MailProperties mailProperties;

    @RequestMapping("/sendEmail")
    @ResponseBody
    public boolean sendEmail(@RequestParam("email") String email,
                             @RequestParam("text") String text) {
        try {
            MimeMessage msg = createMimeMsg(email, text, "java.png");
            javaMailSender.send(msg);
        } catch (Exception ex) {
            log.error("邮件发送失败：", ex);
            return false;
        }
        return true;
    }
```

```
    /**
     * 创建复杂邮件
     * @param email
     * @param text
     * @param attachmentClassPath
     * @return
     * @throws MessagingException
     * @throws UnsupportedEncodingException
     */
    private MimeMessage createMimeMsg(String email, String text, String attachmentClassPath)
throws MessagingException, UnsupportedEncodingException {
        MimeMessage msg = javaMailSender.createMimeMessage();
        MimeMessageHelper mimeMessageHelper = new MimeMessageHelper(msg, true);
        mimeMessageHelper.setFrom(mailProperties.getFrom(), mailProperties.getPersonal());
        mimeMessageHelper.setTo(email);
        mimeMessageHelper.setBcc(mailProperties.getBcc());
        mimeMessageHelper.setSubject(mailProperties.getSubject());
        mimeMessageHelper.setText(text);
        mimeMessageHelper.addAttachment("附件",
                new ClassPathResource(attachmentClassPath));
        return msg;
    }

    /**
     * 创建简单邮件
     * @param email
     * @param text
     * @return
     */
    private SimpleMailMessage createSimpleMsg(String email, String text) {
        SimpleMailMessage msg = new SimpleMailMessage();
        msg.setFrom(mailProperties.getFrom());
        msg.setTo(email);
        msg.setBcc(mailProperties.getBcc());
        msg.setSubject(mailProperties.getSubject());
        msg.setText(text);
        return msg;
    }

}
```

可以看到，JavaMailSender是直接注入就能使用了，另外这里提供了两个创建邮件消息的示例方法：

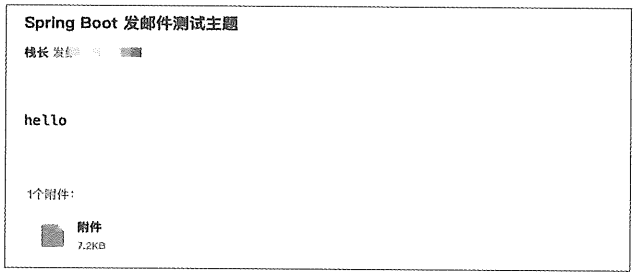
- SimpleMailMessage：用于构建简单的纯文本邮件。
- MimeMessage：用于构建复杂邮件，比如要支持 HTML 内容邮件，以及发送附件等。

MailProperties 参数类用于绑定一些邮件及发送人的固定信息，如发件人、昵称等，可以根据自身情况提供参数配置化。

启动应用后，调用发送邮件接口：

```
http://localhost:8080/sendEmail?email=xxx@xxx.com&text=hello
```

复杂邮件发送正常了，附件也能正常下载和显示，如下图所示。



当然，JavaMailSender 也是可以自定义的，比如在需要多个 JavaMailSender 的情况下，这时默认注册的一个 JavaMailSender 实例便不满足要求。

3.6 自定义 Starter

前面介绍了 Starter 的实现机制，以及 Spring Boot 封装的发邮件自动配置原理，可以看到，自定义一个 Starter 并不难，本节实现一个简单的自定义 Spring Boot Starter。

3.6.1 创建 Starter 工程

根据 Starter 定义的规范，一般一个第三方的应用应该以 *-spring-boot-starter 的形式命名，所以需要先创建一个 javastack-spring-boot-starter 的工程。除了命名要注意，其他不再赘述，就是一个普通的 Spring Boot 应用，创建过程略。

3.6.2 创建自动配置类

下面创建了一个简单的自动配置类：

```
@AutoConfiguration
```

```
@ConditionalOnProperty(prefix = "javastack.starter", name = "enabled", havingValue = "true")
public class TestServiceAutoConfiguration {

    @Bean
    public TestService testService() {
        return new TestService();
    }

}
```

这个自动配置类很简单，就是判断 Spring 环境配置中是否有 javastack.starter.enabled=true 这个参数的值，如果有就配置一个 TestService 的 Bean。

TestService 类的示例代码如下：

```
public class TestService {

    public String getServiceName() {
        return "Java 技术栈 ";
    }

}
```

这个类只有一个方法 getServiceName，仅返回一个字符串：Java 技术栈。

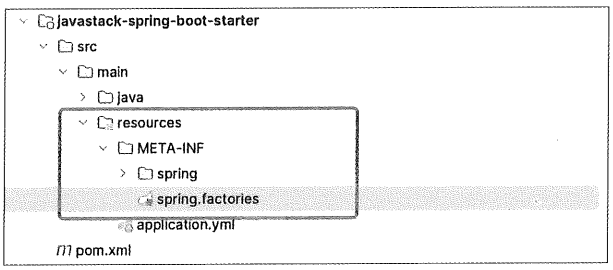
3.6.3 注册自动配置类（spring.factories）

老的自动配置类注册方式（spring.factories）已经退出历史舞台了，但主流应用并不会很快升级 Spring Boot 3.0.0，市面上大多的应用还是以 Spring Boot 2.x 为主，所以笔者这里也提供了老的自动配置类注册规范的实现方式。

先在 Spring Boot 应用的 resource 资源目录下创建 spring.factories 自动配置文件：

META-INF/spring.factories

工程结构如下图所示。



然后添加要注册的自动配置类：

```
org.springframework.boot.autoconfigure.EnableAutoConfiguration=\
cn.javastack.springboot.starter.config.TestServiceAutoConfiguration
```

这种老的自动配置类的注册方式适用于 Spring Boot 2.7 及以下版本，Spring Boot 3.0+ 的自动配置类注册方式见 3.6.5 节。

3.6.4 使用 Starter

自定义的 Spring Boot Starter 应用搭建好之后，可以测试一下它是否生效了，一般是把它打成 jar 包并上传到 Maven 仓库供其他同事调用，这里笔者为了测试，先在本地打包，再新建另外一个应用并引用它进行测试。所以，再新建一个 javastack-spring-boot-starter-sample 应用，并添加这个自定义 Starter 的依赖：

```
<dependencies>
  <dependency>
    <groupId>cn.javastack</groupId>
    <artifactId>javastack-spring-boot-starter</artifactId>
    <version>1.0</version>
    <scope>compile</scope>
  </dependency>
</dependencies>
```

在启动类中注入该 Starter 自动配置类中的 TestService 并输出其方法的值：

```
@Slf4j
@SpringBootApplication
public class Application {

    public static void main(String[] args) {
        SpringApplication.run(Application.class);
    }

    @Bean
    public CommandLineRunner commandLineRunner(TestService testService) {
        return (args) -> {
            log.info(testService.getServiceName());
        };
    }

}
```

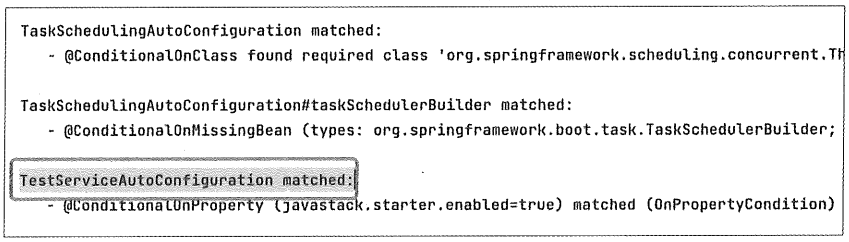
自定义 Starter 的时候定义了需要有 javastack.starter.enabled=true 这个参数的值才会自动配置，所以在测试应用 application.yml 配置文件中添加这个配置参数：

```
javastack:
  starter:
enabled: true
```

最后启动应用：

```
c.j.s.starter.sample.Application          : Java 技术栈
```

结果正常输出，说明引入的自定义 Starter 自动配置成功了，在应用启动时的自动配置报告中也看到了自定义的自动配置，如下图所示。



再把配置故意改错进行测试：

```
javastack:
  starter:
    enabled: false
```

然后重新启动应用：

```
***************************
APPLICATION FAILED TO START
***************************

Description:

Parameter 0 of method commandLineRunner in cn.javastack.springboot.starter.sample.
Application required a bean of type 'cn.javastack.springboot.starter.service.TestService'
that could not be found.

Action:
```

```
Consider defining a bean of type 'cn.javastack.springboot.starter.service.TestService'
in your configuration.

Process finished with exit code 1
```

此时应用运行报错，因为把自动配置需要的参数改为了 false，自定义的 Starter 就不能完成自动配置，Spring 容器中没有 TestService 这个实例就抛出异常了。

本节简单演示了如何自定义一个 Spring Boot Starter，根据某个参数的值来决定是否自动配置，还可以根据是否有某个类、某个 Bean 等系列条件注解完成更加复杂的 Starter 自动配置。

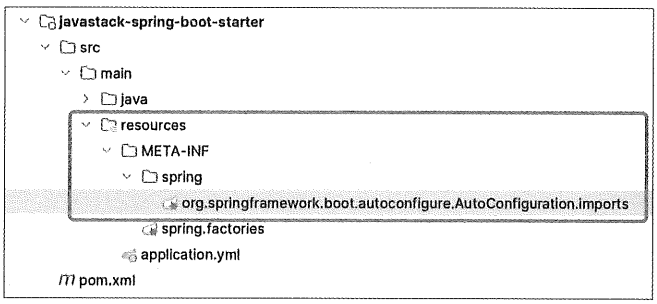
3.6.5 注册自动配置类（新规范）

前面的章节使用的是老规范自动配置文件，Spring Boot 2.7 开始约定了自动配置类注册文件新规范，并且在 Spring Boot 3.0.0 中正式废除了老的自动配置类注册方式，所以建议读者在 Spring Boot 2.7+ 中使用新的自动配置文件注册自动配置类，避免后续升级产生兼容问题。

先在 javastack-spring-boot-starter 工程中创建新规范自动配置文件：

META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports

工程结构如下图所示。



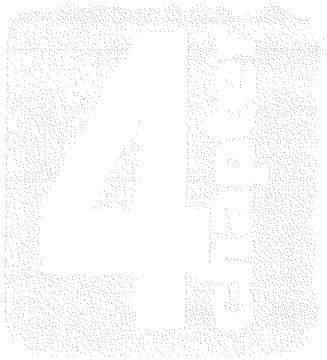
创建完直接在新的自动配置类注册文件中输入自动配置类：

```
cn.javastack.springboot.starter.config.TestServiceAutoConfiguration
```

多个自动配置类直接“回车”配置即可，简单方便，然后重启 javastack-spring-boot-starter-sample 应用：

```
c.j.s.starter.sample.Application         : Java 技术栈
```

应用正常启动，结果也正常输出，说明新规范自动配置文件生效了。



第 4 章 Spring Boot 启动过程与扩展应用

Spring Boot 框架的启动过程与扩展应用是学习 Spring Boot 框架必不可少的部分，本章会介绍 Spring Boot 启动过程中需要掌握的内容，包括 Spring Boot 应用的引导类、启动类、启动方法、核心注解、启动失败分析、启动图案、启动事件和监听器、*Runners 运行器，以及启动流程及源码分析等，这些内容可以帮助开发者更好地运用 Spring Boot。

4.1 启动入口

4.1.1 应用启动类

在 Spring Boot 基础入门一章中，我们看到了 Spring 网站自动生成的 Application 启动类，它是 Spring Boot 应用的启动入口类。如果没有必要，一般建议启动入口类要放置于根目录下，以便使用注解扫描并管理所有子目录包的组件。

Spring Boot 启动类必须包含一个标准的 main 方法，这也是启动 Java 应用的入口方法，然后在 main 方法中添加 SpringApplication.run 方法启动 Spring Boot 应用：

```
@SpringBootApplication
public class Application {

    public static void main(String[] args) {
        SpringApplication.run(Application.class, args);
    }

}
```

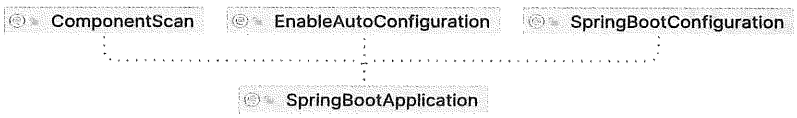
启动类上面的 @SpringBootApplication 注解用于标识应用启动的入口类，也负责启动类统一筹划配置类、组件扫描、自动配置等工作，它是 Spring Boot 启动类上的核心注解，源码如下：

```
@Target(ElementType.TYPE)
@Retention(RetentionPolicy.RUNTIME)
@Documented
@Inherited
@SpringBootConfiguration
@EnableAutoConfiguration
@ComponentScan(excludeFilters = { @Filter(type = FilterType.CUSTOM, classes =
TypeExcludeFilter.class),
        @Filter(type = FilterType.CUSTOM, classes = AutoConfigurationExcludeFilter.class) })
public @interface SpringBootApplication {
```

它主要组合了以下 3 个注解：

- @SpringBootConfiguration：标识为一个 Spring Boot 配置类。
- @EnableAutoConfiguration：开启应用的自动配置功能。
- @ComponentScan：开启自动扫描 Spring 组件功能。

类关系如下图所示。



1. @SpringBootConfiguration 注解

@SpringBootConfiguration 注解是 Spring Boot 配置类注解，组合了 @Configuration 注解，该注解的源码如下：

```
@Target(ElementType.TYPE)
@Retention(RetentionPolicy.RUNTIME)
@Documented
@Configuration
@Indexed
public @interface SpringBootConfiguration {
```

@SpringBootConfiguration 注解在 Spring Boot 配置管理一章有详细介绍，它只是用来修饰 Spring Boot 配置类而已，或者可用于 Spring Boot 后续的扩展，也就是说，如果在 Application 启动

类中使用了 @SpringBootApplication 或者 @SpringBootConfiguration 注解，则其可以作为一个配置类使用。

2. @EnableAutoConfiguration 注解

@EnableAutoConfiguration 注解表示开启 Spring Boot 的自动配置功能，一般不会单独使用，如果单独使用，则可以用来关闭某个自动配置的选项，如关闭数据源自动配置功能：

```
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.boot.autoconfigure.jdbc.DataSourceAutoConfiguration;

@SpringBootApplication(exclude = { DataSourceAutoConfiguration.class })
public class Application {

}
```

3. @ComponentScan 注解

@ComponentScan 是 Spring 框架扫描 Spring 组件并进行注册的注解，这是 Spring 3.1 添加的一个注解，用来代替配置文件中的 component-scan 配置，默认会扫描当前目录及其所有子目录，这也是为什么推荐把启动类放在根目录下的原因。另外，@ComponentScan 是可重复注解，即可以配置多个，用来扫描并注册不同的子包。

了解了以上三个子注解，我们也就知道了这个 @SpringBootApplication 核心注解并不是必需的，可以分开单独使用。

4.1.2 应用启动方法

1. SpringApplication

Spring Boot 应用的启动是通过一个带有 main 方法的启动类完成的，SpringApplication 默认调用的是 run 静态方法，第一个参数一般为应用启动类，也可以是多个其他的 @Configuration 类或者 @Component 类，第二个参数为传递给应用的 main 方法参数。

这个方法的源码如下：

```
public static ConfigurableApplicationContext run(Class<?>[] primarySources, String[] args) {
    return new SpringApplication(primarySources).run(args);
}
```

它最终也构建了一个默认的 SpringApplication 实例，并调用实例的 run 普通方法，所以，不仅

可以在 application 配置文件中配置启动环境，还可以自定义创建 SpringApplication 实例的方式：

```
public static void main(String[] args) {
    SpringApplication springApplication = new SpringApplication(Application.class);

    // 自定义设置
    springApplication.xxx();

    springApplication.run(args);
}
```

2. SpringApplicationBuilder

除了使用 SpringApplication 启动，还能使用链式的 API：

```
public static void main(String[] args) {
    new SpringApplicationBuilder()
        .sources(Parent.class)
        .child(Application.class)
        .bannerMode(Banner.Mode.OFF)
        .run(args);
}
```

SpringApplicationBuilder 的构建器的源码如下：

```
public SpringApplicationBuilder(Class<?>... sources) {
    this(null, sources);
}

public SpringApplicationBuilder(ResourceLoader resourceLoader, Class<?>... sources) {
    this.application = createSpringApplication(resourceLoader, sources);
}

protected SpringApplication createSpringApplication(ResourceLoader resourceLoader,
Class<?>... sources) {
    return new SpringApplication(resourceLoader, sources);
}
```

可以看到 SpringApplicationBuilder 其实也是对 SpringApplication 类的包装，其中的 sources 和 child 方法用于构建多应用上下文，两者共享同一个 Environment 环境，但 Web 组件必须包含在 child 上下文环境中。

了解了本节的内容，就知道如何通过 Java 类在 Spring Boot 中定制启动环境，对了解后续章节

的内容也有帮助。

4.1.3 启动引导类

Spring Boot 应用虽然会有一个带有 main 方法的 Application 启动类，但它一般只会在开发工具中执行启动，也就是在 Spring Boot 应用未打包的情况下运行，如果打包成可执行的 jar 包运行，它就不是启动 Spring Boot 应用最前置的入口了。

Spring Boot 应用打包可以参考第 11 章 Spring Boot 打包和部署。

应用打包成可执行的 jar 包后，可以直接通过 java 命令运行，如下所示。

```
$ java -jar spring-boot-quick-start-1.0.jar
```

这个 jar 包的目录结构如下：

```
.
├── BOOT-INF
│   ├── classes
│   │   └── cn
│   │       └── javastack
│   │           └── springboot
│   │               └── quckstart
│   └── lib
├── META-INF
│   └── maven
│       └── cn.javastack
│           └── spring-boot-quick-start
└── org
    └── springframework
        └── boot
            └── loader
                ├── archive
                ├── data
                ├── jar
                ├── jarmode
                └── util
```

主要包含以下三个目录：

- BOOT-INF：包含应用启动所需要的类库。
- META-INF：包含应用打包的相关描述文件。
- org：包含 Spring Boot 启动所需要的引导类。

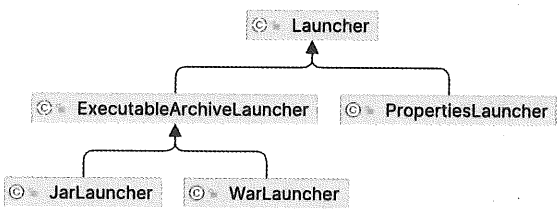
在 META-INF 目录下可以找到 MANIFEST.MF 文件：

```
Manifest-Version: 1.0
Created-By: Maven JAR Plugin 3.3.0
Build-Jdk-Spec: 17
Implementation-Title: spring-boot-quick-start
Implementation-Version: 1.0
Main-Class: org.springframework.boot.loader.JarLauncher
Start-Class: cn.javastack.springboot.quckstart.Application
Spring-Boot-Version: 3.0.0
Spring-Boot-Classes: BOOT-INF/classes/
Spring-Boot-Lib: BOOT-INF/lib/
Spring-Boot-Classpath-Index: BOOT-INF/classpath.idx
Spring-Boot-Layers-Index: BOOT-INF/layers.idx
```

它包括一个 jar 包的相关描述信息，除了基本的项目构建信息，其中还有两个最重要的应用启动信息：

- Main-Class：应用的启动引导类，即打包后最前置的启动入口类，启动 jar 包的引导类为 org.springframework.boot.loader.JarLauncher，启动 war 包的引导类为 org.springframework.boot.loader.WarLauncher。
- Start-Class：即前面章节描述的 Application 启动类。

Main-Class 参数指定了 Spring Boot 应用具体要使用的引导类，Spring Boot 应用的引导机制是由 spring-boot-loader 模块实现的，引导实现类顶层的是 Launcher 抽象类，Launcher 引导类的结构如下图所示，



Launcher 引导类主要分为以下两种引导类型：

- **根据可执行文件引导**：ExecutableArchiveLauncher 抽象类，按打包类型又分为 JarLauncher 和 WarLauncher 引导实现类。
- **根据参数文件引导**：PropertiesLauncher 引导实现类。

可执行文件 JarLauncher 引导类相关的源码如下：

```
public class JarLauncher extends ExecutableArchiveLauncher {

    ...

    public static void main(String[] args) throws Exception {
        new JarLauncher().launch(args);
    }

}
```

JarLauncher 引导类调用的 launch 方法是 Launcher 抽象父类中的方法，如下面的 Launcher 抽象类的源码所示。

```
public abstract class Launcher {

    private static final String JAR_MODE_LAUNCHER = "org.springframework.boot.loader.
jarmode.JarModeLauncher";

    /**
     * Launch the application. This method is the initial entry point that should be
     * called by a subclass {@code public static void main(String[] args)} method.
     * @param args the incoming arguments
     * @throws Exception if the application fails to launch
     */
    protected void launch(String[] args) throws Exception {
        if (!isExploded()) {
            JarFile.registerUrlProtocolHandler();
        }
        ClassLoader classLoader = createClassLoader(getClassPathArchivesIterator());
        String jarMode = System.getProperty("jarmode");
        String launchClass = (jarMode != null && !jarMode.isEmpty()) ? JAR_MODE_LAUNCHER:
getMainClass();
        launch(args, launchClass, classLoader);
    }

    ...

    /**
     * Returns the main class that should be launched.
     * @return the name of the main class
     * @throws Exception if the main class cannot be obtained
```

```
     */
    protected abstract String getMainClass() throws Exception;

    ...

}
```

Launcher#launch 引导方法提供了一个 getMainClass 的抽象方法，用于获取实际的应用启动类，这个便是在 ExecutableArchiveLauncher 可执行文件引导抽象类中实现的，实现的逻辑则是获取应用描述文件 META-INF/MANIFEST.MF 中的 Start-Class 参数的值，如下面的 ExecutableArchiveLauncher 类的源码所示。

```
public abstract class ExecutableArchiveLauncher extends Launcher {

    private static final String START_CLASS_ATTRIBUTE = "Start-Class";

    ...

    @Override
    protected String getMainClass() throws Exception {
        Manifest manifest = this.archive.getManifest();
        String mainClass = null;
        if (manifest != null) {
            mainClass = manifest.getMainAttributes().getValue(START_CLASS_ATTRIBUTE);
        }
        if (mainClass == null) {
            throw new IllegalStateException("No 'Start-Class' manifest entry specified in"
+ this);
        }
        return mainClass;
    }

    ...

}
```

获取到应用启动类之后，进入 Launcher#launch 方法：

```
public abstract class Launcher {

    ...

    protected void launch(String[] args, String launchClass, ClassLoader classLoader)
throws Exception {
```

```
        Thread.currentThread().setContextClassLoader(classLoader);
        createMainMethodRunner(launchClass, args, classLoader).run();
    }

    protected MainMethodRunner createMainMethodRunner(String mainClass, String[] args,
ClassLoader classLoader) {
        return new MainMethodRunner(mainClass, args);
    }

    ...

}
```

首先通过上一个 launch 方法创建的一个类加载器传递并绑定到当前线程，即上下文类加载器，然后创建一个 MainMethodRunner 实例并调用其 run 方法，如下面 MainMethodRunner 类的源码所示。

```
public class MainMethodRunner {

    private final String mainClassName;

    private final String[] args;

    /**
     * Create a new {@link MainMethodRunner} instance.
     * @param mainClass the main class
     * @param args incoming arguments
     */
    public MainMethodRunner(String mainClass, String[] args) {
        this.mainClassName = mainClass;
        this.args = (args != null) ? args.clone() : null;
    }

    public void run() throws Exception {
        Class<?> mainClass = Class.forName(this.mainClassName, false, Thread.currentThread()
.getContextClassLoader());
        Method mainMethod = mainClass.getDeclaredMethod("main", String[].class);
        mainMethod.setAccessible(true);
        mainMethod.invoke(null, new Object[] { this.args });
    }

}
```

即先把启动类和入口参数传过来，然后在运行 run 方法时，通过 Java 反射的方式调用启动类中

的 main 方法并运行。所以，Main-Class 和 Start-Class 两者是先后调用的关系，先由 Main-Class 参数指定引导类，引导类执行时再调用 Start-Class 参数指定的应用启动类。这样处理的好处是，应用只需要一个启动类，不同类型的应用由不同的引导类引导启动。

4.2 关闭启动日志

Spring Boot 启动过程中，系统默认会输出一些详细的启动日志：

```
  .   ____          _            __ _ _
 /\\ / ___'_ __ _ _(_)_ __  __ _ \ \ \ \
( ( )\___ | '_ | '_| | '_ \/ _` | \ \ \ \
 \\/  ___)| |_)| | | | | || (_| |  ) ) ) )
  '  |____| .__|_| |_|_| |_\__, | / / / /
 =========|_|==============|___/=/_/_/_/
 :: Spring Boot ::                (v3.0.0)

..... : .....
..... : No active profile set, falling back to 1 default profile: "default"
..... : Tomcat initialized with port(s): 8081 (http)
..... : Starting service [Tomcat]
..... : Starting Servlet engine: [Apache Tomcat/10.0.21]
..... : Initializing Spring embedded WebApplicationContext
..... : Root WebApplicationContext: initialization completed in 830 ms
..... : Tomcat started on port(s): 8081 (http) with context path '/javastack'
..... : Started Application in 1.6 seconds (process running for 1.884)
```

Spring Boot 3.0 新变化：

在应用的启动日志中不再记录主机名，这样可以避免不必要的网络查找，有助于减少应用的启动时间。

如果不想输出详细的启动日志，则可以通过以下参数关闭：

```
spring:
  main:
    log-startup-info: false
```

或者在启动方法上禁用：

```
public static void main(String[] args) {
    SpringApplication springApplication = new SpringApplication(Application.class);
```

```
        springApplication.setLogStartupInfo(false);
        springApplication.run(args);
    }
```

4.3 启动失败分析

4.3.1 失败分析器

Spring Boot 应用经常启动失败就会显示一大堆错误信息，比如我们重复启动同一个端口，应用启动时就会输出端口已经在使用的异常：

```
***************************
APPLICATION FAILED TO START
***************************

Description:

Embedded servlet container failed to start. Port 8080 was already in use.

Action:

Identify and stop the process that's listening on port 8080 or configure this
application to listen on another port.
```

Spring Boot 应用启动失败的场景是由 FailureAnalyzer 失败分析器拦截并处理的，Spring Boot 注册了许多 FailureAnalyzer，它能分析启动失败异常并给用户显示有用的诊断信息。

FailureAnalyzer 失败分析器对应的是 FailureAnalyzer 接口：

```
/**
 * A {@code FailureAnalyzer} is used to analyze a failure and provide diagnostic
 * information that can be displayed to the user.
 *
 * @author Andy Wilkinson
 * @since 1.4.0
 */
@FunctionalInterface
public interface FailureAnalyzer {

      /**
```

```
     * Returns an analysis of the given {@code failure}, or {@code null} if no analysis
     * was possible.
     * @param failure the failure
     * @return the analysis or {@code null}
     */
    FailureAnalysis analyze(Throwable failure);

}
```

Spring Boot 内置的失败分析器目前还注册在老的自动配置类注册文件中，比如 Spring Boot 基础包中的 spring.factories 文件：

spring-boot-3.0.0.jar!/META-INF/spring.factories

其中注册的所有系统级的失败分析器列表如下：

```
# Failure Analyzers
org.springframework.boot.diagnostics.FailureAnalyzer=\
org.springframework.boot.context.config.ConfigDataNotFoundFailureAnalyzer,\
org.springframework.boot.context.properties.IncompatibleConfigurationFailureAnalyzer,\
org.springframework.boot.context.properties.NotConstructorBoundInjectionFailureAnalyzer,\
org.springframework.boot.diagnostics.analyzer.BeanCurrentlyInCreationFailureAnalyzer,\
org.springframework.boot.diagnostics.analyzer.BeanDefinitionOverrideFailureAnalyzer,\
org.springframework.boot.diagnostics.analyzer.BeanNotOfRequiredTypeFailureAnalyzer,\
org.springframework.boot.diagnostics.analyzer.BindFailureAnalyzer,\
org.springframework.boot.diagnostics.analyzer.BindValidationFailureAnalyzer,\
org.springframework.boot.diagnostics.analyzer.UnboundConfigurationPropertyFailureAnalyzer,\
org.springframework.boot.diagnostics.analyzer.MutuallyExclusiveConfigurationProperties-
FailureAnalyzer,\
org.springframework.boot.diagnostics.analyzer.NoSuchMethodFailureAnalyzer,\
org.springframework.boot.diagnostics.analyzer.NoUniqueBeanDefinitionFailureAnalyzer,\
org.springframework.boot.diagnostics.analyzer.PortInUseFailureAnalyzer,\
org.springframework.boot.diagnostics.analyzer.ValidationExceptionFailureAnalyzer,\
org.springframework.boot.diagnostics.analyzer.InvalidConfigurationPropertyNameFailureAn-
alyzer,\
org.springframework.boot.diagnostics.analyzer.InvalidConfigurationPropertyValueFailure-
Analyzer,\
org.springframework.boot.diagnostics.analyzer.PatternParseFailureAnalyzer,\
org.springframework.boot.liquibase.LiquibaseChangelogMissingFailureAnalyzer,\
org.springframework.boot.web.context.MissingWebServerFactoryBeanFailureAnalyzer,\
org.springframework.boot.web.embedded.tomcat.ConnectorStartFailureAnalyzer
```

再回到上面的端口重复启动失败异常，它其实就是注册了 PortInUseFailureAnalyzer 失败分析器，可以看到 PortInUseFailureAnalyzer 失败分析器就在注册列表中。

PortInUseFailureAnalyzer 的源码如下：

```
/**
 * A {@code FailureAnalyzer} that performs analysis of failures caused by a
 * {@code PortInUseException}.
 *
 * @author Andy Wilkinson
 */
class PortInUseFailureAnalyzer extends AbstractFailureAnalyzer<PortInUseException> {

    @Override
    protected FailureAnalysis analyze(Throwable rootFailure, PortInUseException cause) {
        return new FailureAnalysis("Web server failed to start. Port " + cause.getPort() +
 " was already in use.",
                "Identify and stop the process that's listening on port " + cause.getPort()+
"or configure this"
                        + "application to listen on another port.",
                cause);
    }

}
```

只要应用启动过程中抛出了 PortInUseException 异常就会被这个失败分析器拦截并输出可读性的错误信息。

4.3.2 自定义失败分析器

默认的错误信息对于开发人员来说是很明显的，但不是所有的错误信息都很好理解，或者对于其他需要查看日志的非开发人员就不是很友好，所以，可以通过扩展 FailureAnalyzer 失败分析器接口来达到自定义错误信息的目的。

但从上面内置的端口重复使用失败分析器这一点可以发现，所有的分析器都继承了这个 AbstractFailureAnalyzer 抽象类，然后由它实现 FailureAnalyzer 接口，所以一般基于这个抽象基类就可以实现自定义失败分析器的扩展。

比如说上面的 PortInUseFailureAnalyzer 失败分析器输出的信息是英文，不是很直观，接下来笔者自定义实现一个类似的中文失败分析器。

首先创建一个 PortInUseFailureAnalyzer 失败分析器继承 AbstractFailureAnalyzer 抽象类：

```
public class PortInUseFailureAnalyzer extends AbstractFailureAnalyzer<PortInUseException> {
```

```
    @Override
    protected FailureAnalysis analyze(Throwable rootFailure, PortInUseException cause) {
        return new FailureAnalysis("你启动的端口 " + cause.getPort() + " 被占用了.",
                                   "快检查下端口 " + cause.getPort() + " 被哪个程序占用了,
                                   或者强制杀掉进程.",
                                   cause);
    }

}
```

重写 analyze 方法，并返回一个 FailureAnalysis 对象，FailureAnalysis 类的三个主要信息如下面的源码所示。

```
public FailureAnalysis(String description, String action, Throwable cause) {
    this.description = description;
    this.action = action;
    this.cause = cause;
}
```

它们分别表示要展示的三个信息如下：

- 可读性的错误描述。
- 建议的检查修复动作。
- 原始异常。

然后在应用的 META-INF/spring.factories 配置文件中进行注册：

```
org.springframework.boot.diagnostics.FailureAnalyzer=\
cn.javastack.springboot.features.analyzer.PortInUseFailureAnalyzer
```

需要注意的是：

失败分析器还需要在 spring.factories 文件中注册，这个注册逻辑没有任何变化。

如果把失败分析器放在新的自动配置类注册文件中，则不会生效。虽然自动配置类的注册文件已经从 spring.factories 移到另外一个新文件中了，但它仅用于注册自动配置类，不用能注册其他系统组件。

然后重复启动多个相同端口的应用，输出日志如下：

```
***************************
APPLICATION FAILED TO START
***************************
```

```
Description:
你启动的端口 8080 被占用了 .
Action:
快检查下端口 8080 被哪个程序占用了，或者强制杀掉进程 .
```

下面再自定义一个全新的失败分析器，这样可以更清楚地认识失败分析器。我们在创建 Bean 的过程中手动抛出一个自定义的异常：

```
@Bean
public CommandLineRunner commandLineRunner() {
    return (args) -> {
        throw new JavastackException("Java 技术栈异常 ");
    };
}
```

如果不注册该失败分析器，则这个自定义的异常就不会被内置的失败分析器拦截，应用启动后会输出大堆的异常信息：

```
java.lang.IllegalStateException: Failed to execute CommandLineRunner
    at org.springframework.boot.SpringApplication.callRunner (SpringApplication.
java:780) ~[spring-boot-3.0.0-M3.jar:3.0.0-M3]
    at org.springframework.boot.SpringApplication.callRunners (SpringApplication.
java:761) ~[spring-boot-3.0.0-M3.jar:3.0.0-M3]
    at org.springframework.boot.SpringApplication.run(SpringApplication.java:317)
~[spring-boot-3.0.0-M3.jar:3.0.0-M3]
    at cn.javastack.springboot.features.Application.main(Application.java:30)
~[classes/:na]
Caused by: cn.javastack.springboot.features.analyzer.JavastackException: Java 技术栈异常
    at cn.javastack.springboot.features.Application.lambda$commandLineRunner$0
(Application.java:41) ~[classes/:na]
    at org.springframework.boot.SpringApplication.callRunner(SpringApplication.
java:777) ~[spring-boot-3.0.0-M3.jar:3.0.0-M3]
    ... 3 common frames omitted
```

如果不想看到大篇幅的启动错误，则可以添加一个失败分析器拦截该异常：

```
public class JavastackFailureAnalyzer extends AbstractFailureAnalyzer<JavastackException> {

    @Override
    protected FailureAnalysis analyze(Throwable rootFailure, JavastackException cause) {
        return new FailureAnalysis("Java 技术栈发生异常了……",
                                   " 赶快去检查一下吧！ ",
                                   cause);
```

```
    }
}
```

添加失败分析器并注册：

```
org.springframework.boot.diagnostics.FailureAnalyzer=\
cn.javastack.springboot.features.analyzer.PortInUseFailureAnalyzer,\
cn.javastack.springboot.features.analyzer.JavastackFailureAnalyzer
```

然后重新启动应用：

```
***************************
APPLICATION FAILED TO START
***************************
Description:
Java 技术栈发生异常了……
Action:
赶快去检查一下吧！
```

Spring Boot 提供的失败分析器以友好的错误信息和修复建议代替了大堆的错误异常信息，使用失败分析器能很直观地看出是什么错误及如何修复这个错误，可以帮助我们更直观地定位应用启动故障。

4.4 全局懒加载

Spring Boot 可以开启全局懒加载，懒加载的意思是 Bean 不会在应用启动时全部创建，只会在真正使用时才创建，Spring Boot 默认不开启懒加载，如需开启，则可以在应用配置文件中配置：

```
spring:
  main:
lazy-initialization: true
```

也可以在 SpringApplication 或者 SpringApplicationBuilder 启动类上设置，这里不再赘述。

Spring Boot 懒加载可以大大减少应用的启动时间，还能节省系统资源，但也会造成一些在应用启动时就能发现的问题不能被及时发现，而要等到懒加载时才被发现，比如可能会引发以下几个问题：

- JVM 内存配置的问题，因为不是全部加载，所以不能在启动时判断所需内存的大小，在应

用真正运行时可能引发内存不足等问题。

- 运行时找不到指定的类等运行时异常，或者是配置错误引发的系列问题，如果能在启动时及时发现问题，就不要等到运行时再去处理。
- 如果是 HTTP 服务，则可能因为临时实例化而造成第一个请求变慢，响应延迟，体验不是很好，对负载均衡和自动伸缩也可能造成影响。

因为懒加载是真正使用的时候才去实例化，所以是否开启全局懒加载完全看应用类型，如果是 HTTP 服务为主的应用，要求即时响应，则不建议开启全局懒加载。

4.5 启动图案

4.5.1 默认图案

Spring Boot 应用在启动的时候会显示一个默认的 Spring 图案：

```
  .   ____          _            __ _ _
 /\\ / ___'_ __ _ _(_)_ __  __ _ \ \ \ \
( ( )\___ | '_ | '_| | '_ \/ _` | \ \ \ \
 \\/  ___)| |_)| | | | | || (_| |  ) ) ) )
  '  |____| .__|_| |_|_| |_\__, | / / / /
 =========|_|==============|___/=/_/_/_/
 :: Spring Boot ::                (v3.0.0)
```

4.5.2 输出模式

Spring Boot 是通过 Banner 接口在日志中输出启动图案的：

```
@FunctionalInterface
public interface Banner {

    /**
    * Print the banner to the specified print stream.
    * @param environment the spring environment
    * @param sourceClass the source class for the application
    * @param out the output print stream
    */
    void printBanner(Environment environment, Class<?> sourceClass, PrintStream out);
```

```
    /**
    * An enumeration of possible values for configuring the Banner.
    */
    enum Mode {

        /**
        * Disable printing of the banner.
        */
        OFF,

        /**
        * Print the banner to System.out.
        */
        CONSOLE,

        /**
        * Print the banner to the log file.
        */
        LOG

    }

}
```

从 Banner.Mode 接口枚举可以看到支持的输出模式：

- OFF：关闭图案。
- CONSOLE（默认）：输出到 System.out。
- LOG：输出到日志文件。

所以，如果想关闭启动图案，或者将启动图案输出到日志文件，那么只需要在启动类上设置即可，比如关闭启动图案：

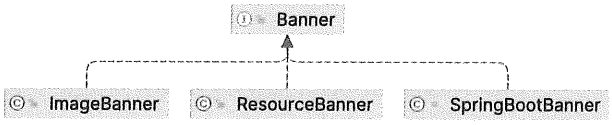
```
public static void main(String[] args) {
    SpringApplication springApplication = new SpringApplication(Application.class);

    // 关闭图案
    springApplication.setBannerMode(Banner.Mode.OFF);

    springApplication.run(args);
}
```

也可以通过以下参数设置：

```
spring:
  main:
    banner-mode: OFF
```

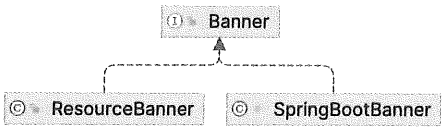
4.5.3 图案实现类

Spring Boot 2.7 中默认有三个实现类，用于不同的实现，类结构如下图所示。



Spring Boot 3.0.0 新变化：

Spring Boot 3.0.0 已经删除了 ImageBanner 图案类，即不再支持 spring.banner.image.* 系列参数配置 Image 图像来作为启动图案，Spring Boot 3.0.0 只支持纯文本形式的启动图案了，类结构如下图所示。



默认的是 SpringBootBanner 实现类，源码如下：

```
class SpringBootBanner implements Banner {

    private static final String[] BANNER = { "", "  .   ____          _            __ _ _",
            " /\\\\ / ___'_ __ _ _(_)_ __  __ _ \\ \\ \\ \\", "( ( )\\___ | '_ | '_| |
'_ \\/ _` | \\ \\ \\ \\",
            " \\\\/  ___)| |_)| | | | | || (_| |  ) ) ) )", "  '  |____| .__|_| |_|_|
|_\\__, | / / / /",
            " =========|_|==============|___/=/_/_/_/" };

    private static final String SPRING_BOOT = " :: Spring Boot :: ";

    private static final int STRAP_LINE_SIZE = 42;

    @Override
```

```
    public void printBanner(Environment environment, Class<?> sourceClass, PrintStream
printStream) {
        for (String line : BANNER) {
            printStream.println(line);
        }
        String version = SpringBootVersion.getVersion();
        version = (version != null) ? " (v" + version + ")" : "";
        StringBuilder padding = new StringBuilder();
        while (padding.length() < STRAP_LINE_SIZE - (version.length() + SPRING_BOOT.
length())) {
            padding.append(" ");
        }

        printStream.println(AnsiOutput.toString(AnsiColor.GREEN, SPRING_BOOT,
AnsiColor.DEFAULT, padding.toString(),
                AnsiStyle.FAINT, version));
        printStream.println();
    }

}
```

从源码可以看到，默认的 Spring 启动图案来源于这个实现类。

4.5.4 自定义图案

如果需要改变应用默认图案，则可以直接在应用资源根目录下创建一个 banner.txt 图案文件：

```
  ____.                                  __                  __
 |    |____ ___  _______    _______/  |______    ____ |  | __
 |    \__  \\  \/ /\__  \  /  ___/\   __\__  \ _/ ___\|  |/ /
/\__|    |/ __ \\   /  / __ \_\___ \  |  |  / __ \\  \___|    <
\________(____  /\_/  (____  /____  > |__| (____  /\___  >__|_ \
             \/           \/     \/             \/     \/     \/
```

也可以通过 spring.banner.location 参数指定图案文件的位置，如果不是 UTF-8 图案方案，则需要使用 spring.banner.charset 指定具体编码。

另外在 banner.txt 图案文件中，也可以使用环境占位符，比如：

```
  ____.                                  __                  __
 |    |____ ___  _______    _______/  |______    ____ |  | __
 |    \__  \\  \/ /\__  \  /  ___/\   __\__  \ _/ ___\|  |/ /
/\__|    |/ __ \\   /  / __ \_\___ \  |  |  / __ \\  \___|    <
\________(____  /\_/  (____  /____  > |__| (____  /\___  >__|_ \
             \/           \/     \/             \/     \/     \/

Spring Boot Version: ${spring-boot.version}
```

启动应用：

```
  ____.                                  __                  __
 |    |____ ___  _______    _______/  |______    ____ |  | __
 |    \__  \\  \/ /\__  \  /  ___/\   __\__  \ _/ ___\|  |/ /
/\__|    |/ __ \\   /  / __ \_\___ \  |  |  / __ \\  \___|    <
\________(____  /\_/  (____  /____  > |__| (____  /\___  >__|_ \
             \/           \/     \/             \/     \/     \/

Spring Boot Version: 3.0.0
```

自定义图案在实际工作中很少使用，不过在启动图案上输出一些应用的特定版本信息还是略有参考意义的。

4.6 启动事件和监听器

Spring Framework 框架中的事件和监听器无处不在，一般来说，我们很少会使用应用的事件和监听器，但我们也不要忘了它们的存在，Spring 框架内部使用了各种不同的事件来处理不同的任务，比如 ContextRefreshedEvent。

除了常用的 Spring 框架中的事件，Spring Boot 在启动过程中还会发送一系列其他的事件，比如 Spring Boot 基础包中的 spring.factories 文件：

spring-boot-3.0.0.jar!/META-INF/spring.factories

其中注册的所有系统级的监听器列表如下：

```
# Application Listeners
org.springframework.context.ApplicationListener=\
org.springframework.boot.ClearCachesApplicationListener,\
org.springframework.boot.builder.ParentContextCloserApplicationListener,\
org.springframework.boot.context.FileEncodingApplicationListener,\
```

```
org.springframework.boot.context.config.AnsiOutputApplicationListener,\
org.springframework.boot.context.config.DelegatingApplicationListener,\
org.springframework.boot.context.logging.LoggingApplicationListener,\
org.springframework.boot.env.EnvironmentPostProcessorApplicationListener
```

这些监听器都实现了 Spring 框架中的 ApplicationListener 应用监听器接口，监听和处理不同的事件。

4.6.1 启动事件的顺序

当 Spring Boot 应用启动后，事件会按以下顺序发送：

- ApplicationStartingEvent

这个事件在 Spring Boot 应用运行开始时，且进行任何处理之前发送（除了监听器和初始化器注册）。

- ApplicationEnvironmentPreparedEvent

这个事件在当已知要在上下文中使用 Spring 环境 Environment 时，在 Spring 上下文 context 创建之前发送。

- ApplicationContextInitializedEvent

这个事件在 Spring 应用上下文 ApplicationContext 准备好了，并且应用初始化器 ApplicationContextInitializers 已经被调用，在 bean definitions 被加载之前发送。

- ApplicationPreparedEvent

这个事件是在 Spring 上下文 context 刷新之前，且在 bean definitions 被加载之后发送。

- ApplicationStartedEvent

这个事件是在 Spring 上下文 context 刷新之后，且在任何 application/ command-line runners 被调用之前发送。

- AvailabilityChangeEvent

这个事件紧随上个事件之后发送，激活状态 ReadinessState.CORRECT，表示应用已处于活动状态。

- ApplicationReadyEvent

这个事件在任何 application/ command-line runners 调用之后发送。

- AvailabilityChangeEvent

这个事件紧随上个事件之后发送，激活状态 ReadinessState.ACCEPTING_TRAFFIC，表示应用

可以开始准备接收请求了。

- ApplicationFailedEvent

这个事件在应用启动异常时发送。

上面介绍的这些事件列表仅包括 SpringApplication 启动类发出的 SpringApplicationEvents 事件，除了这些事件，以下事件也会在 ApplicationPreparedEvent 之后和 ApplicationStartedEvent 之前发送：

- WebServerInitializedEvent

这个 Web 服务器初始化事件在 WebServer 启动之后发送，实现事件类包括 ServletWebServerInitializedEvent（Servlet Web 服务器初始化事件）、ReactiveWebServerInitializedEvent（响应式 Web 服务器初始化事件）。

- ContextRefreshedEvent

这个上下文刷新事件在 Spring 应用上下文 ApplicationContext 刷新之后发送。

关于事件的具体发送流程，可见 4.8.2 节中的启动流程源码分析。

4.6.2 自定义事件监听器

前面介绍了 Spring Boot 在启动过程中的各个事件，如果想在每个环节处理一些我们想做的事情，那么只需要自定义一个监听器来监听某个事件就可以了。

比如我们想监听上面的第 8 个启动事件（AvailabilityChangeEvent），即应用启动完成可以接收请求了，我们定义一个该事件的监听器并简单输出一个成功标识。

首先新建一个自定义监听器：

```
@Slf4j
@Component
public class JavastackListener implements ApplicationListener<AvailabilityChangeEvent> {

    @Override
    public void onApplicationEvent(AvailabilityChangeEvent event) {
        log.info("监听到事件：" + event);
        if (ReadinessState.ACCEPTING_TRAFFIC == event.getState()){
            log.info("应用启动完成，可以接收请求了……");
        }
    }

}
```

监听器需要实现ApplicationListener接口，泛型AvailabilityChangeEvent表示仅监听AvailabilityChangeEvent事件。

方法中的逻辑是，因启动事件AvailabilityChangeEvent发送了两次，每次状态不同，所以监听该事件时可以根据事件的状态来区分到底是哪一个环节的事件。

因为是Spring应用上下文ApplicationContext创建之后的事件，所以可以直接在监听器上使用@Component注解注册。如果ApplicationContext并未创建，那么这时的Bean是不能被加载的，所以这时不能使用@Component注解注册，需要在META-INF/spring.factories配置文件中注册监听器：

```
org.springframework.context.ApplicationListener=\
cn.javastack.springboot.features.listener.JavastackListener
```

或者可以使用SpringApplication.addListeners(…)方法的自动注册。

启动应用来看一下启动日志：

```
... JavastackListener       : 监听到事件：org.springframework.boot.availability.
AvailabilityChangeEvent...
... JavastackListener       : 监听到事件：org.springframework.boot.availability.
AvailabilityChangeEvent...
... JavastackListener       : 应用启动完成，可以接收请求了……
```

可以看到同时输出了两次监听日志，但只输出最后一步启动完成的日志，自定义事件监听器实现成功。

了解了Spring Boot启动过程中的各个事件及监听机制，读者可以“依葫芦画瓢”实现Spring Boot启动过程中的各个自定义操作，比如在启动过程上实现动态注册、移除Bean等。

一般不建议使用事件和监听器实现比较耗时和繁重的任务，这样会影响应用的正常启动，Spring Boot推荐使用Runners进行实现，即在应用启动后再处理，具体见下一节的介绍。

4.7 启动运行器

4.7.1 概述

前面很多章节中都介绍了Runner运行器的使用，它允许Spring Boot应用启动完成之后，在接收请求之前运行一些特定的代码逻辑。

Spring Boot Runner 运行器可以实现以下两个接口：

- ApplicationRunner。
- CommandLineRunner。

下面看一下两个接口的源码。

ApplicationRunner：

```
@FunctionalInterface
public interface ApplicationRunner {

    /**
    * Callback used to run the bean.
    * @param args incoming application arguments
    * @throws Exception on error
    */
    void run(ApplicationArguments args) throws Exception;

}
```

CommandLineRunner：

```
@FunctionalInterface
public interface CommandLineRunner {

    /**
    * Callback used to run the bean.
    * @param args incoming main method arguments
    * @throws Exception on error
    */
    void run(String... args) throws Exception;

}
```

这两个接口的工作原理是一样的，它们都只提供了一个 run 运行方法，只是接收的方法参数不一样而已，ApplicationRunner 用来接收 ApplicationArguments 参数，即 SpringApplication 启动时的应用参数，CommandLineRunner 则用来接收 String 可变参数，所以 CommandLineRunner 的使用要更灵活一些。

4.7.2 使用方式

之前的章节中也都有示例，既可以是传统的实现接口定义方式：

```
@Component
public class JavastackRunner implements CommandLineRunner {

    public void run(String... args) {
        // Do something...
    }

}
```

也可以是结合 Lambda 表达式的 @Bean 方法的定义方式：

```
@Bean
public CommandLineRunner javastackRunner() {
    return (args) -> {
        // Do something...
    };
}
```

一般推荐使用第二种定义方式，简单、清晰，也节约代码和文件量。

另外，如果应用启动时定义了多个 ApplicationRunner 和 CommandLineRunner，则默认是按它们被加载的顺序启动的，如果想控制它们的启动顺序，则可以实现 org.springframework. core.Ordered 接口或者使用 org.springframework.core.annotation.Order 注解，这里不再赘述。

4.8 应用启动流程

前面的章节介绍了一些 Spring Boot 启动相关的扩展应用，本节通过分析启动源码的方式介绍 Spring Boot 应用的具体启动流程，看一下 Spring Boot 启动过程中是如何处理以上扩展应用的。

4.8.1 实例化流程

通过之前的章节我们知道，Spring Boot 应用的启动入口就在 SpringApplication # run 方法中，跟踪 run 方法进入 SpringApplication 最核心的构造方法：

```
public class SpringApplication {
```

```
    ...

    public static ConfigurableApplicationContext run(Class<?> primarySource, String... args) {
        return run(new Class<?>[] { primarySource }, args);
    }

    public static ConfigurableApplicationContext run(Class<?>[] primarySources, String[] args) {
        return new SpringApplication(primarySources).run(args);
    }

    public SpringApplication(Class<?>... primarySources) {
        this(null, primarySources);
    }

    public SpringApplication(ResourceLoader resourceLoader, Class<?>... primarySources) {
        // 1
        this.resourceLoader = resourceLoader;

        // 2
        Assert.notNull(primarySources, "PrimarySources must not be null");

        // 3
        this.primarySources = new LinkedHashSet<>(Arrays.asList(primarySources));

        // 4
        this.webApplicationType = WebApplicationType.deduceFromClasspath();

        // 5
        this.bootstrapRegistryInitializers = new ArrayList<>(
                getSpringFactoriesInstances(BootstrapRegistryInitializer.class));

        // 6
        setInitializers((Collection) getSpringFactoriesInstances(ApplicationContextIni-
tializer.class));

        // 7
        setListeners((Collection) getSpringFactoriesInstances(ApplicationListener.class));

        // 8
        this.mainApplicationClass = deduceMainApplicationClass();
    }

      ...

}
```

1. 设置资源加载器

```
this.resourceLoader = resourceLoader;
```

ResourceLoader 是加载资源的策略接口，默认为 null，不需要指定，如果需要自定义资源加载策略，则可以通过手动 new SpringApplication(...) 的方式实例化 SpringApplication。

2. 判断主资源类是否存在

```
Assert.notNull(primarySources, "PrimarySources must not be null");
```

这个主资源类一般是指应用启动类，如 SpringApplication#run 方法所示。

```
@SpringBootApplication
public class Application {

    public static void main(String[] args) {
        SpringApplication.run(Application.class, args);
    }

}
```

这里的 Application.class 便是主资源类，主资源类可以指定多个，但不能为 null，否则启动报错。

3. 设置主资源类集合

```
this.primarySources = new LinkedHashSet<>(Arrays.asList(primarySources));
```

如果第二步中主资源类不为 null，则使用 LinkedHashSet(List) 构造方式初始化主资源类并去重。

4. 推断并设置 Web 应用类型

```
this.webApplicationType = WebApplicationType.deduceFromClasspath();
```

deduceFromClasspath 方法的相关源码如下：

```
static WebApplicationType deduceFromClasspath() {
    if (ClassUtils.isPresent(WEBFLUX_INDICATOR_CLASS, null) && !ClassUtils.isPresent
(WEBMVC_INDICATOR_CLASS, null)
            && !ClassUtils.isPresent(JERSEY_INDICATOR_CLASS, null)) {
        return WebApplicationType.REACTIVE;
    }
```

```
        for (String className : SERVLET_INDICATOR_CLASSES) {
            if (!ClassUtils.isPresent(className, null)) {
                return WebApplicationType.NONE;
            }
        }
        return WebApplicationType.SERVLET;
    }
```

根据类路径下是否有对应应用类型的 Class 类推断出对应的 Web 应用类型，Spring Boot 目前支持的 Web 应用类型枚举如下面的 WebApplicationType 类源码所示。

```
public enum WebApplicationType {

    /**
     * The application should not run as a web application and should not start an
     * embedded web server.
     */
    NONE,

    /**
     * The application should run as a servlet-based web application and should start an
     * embedded servlet web server.
     */
    SERVLET,

    /**
     * The application should run as a reactive web application and should start an
     * embedded reactive web server.
     */
    REACTIVE;

    ...

}
```

Spring Boot 应用主要包括 Servlet 和 Reactive 两种 Web 应用类型，现在的应用主要还是以传统的 Servlet 类型为主，比如，加入 spring-boot-starter-web 依赖，应用就能判断出是 Servlet 应用，加入 spring-boot-starter-webflux 依赖，应用就能判断出是 Reactive 应用。

5. 设置引导注册初始化器

```
this.bootstrapRegistryInitializers = new ArrayList<>(
                getSpringFactoriesInstances(BootstrapRegistryInitializer.class));
```

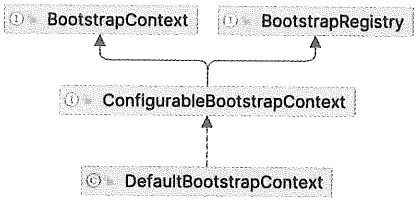
从 spring.factories 文件中获取注册的 BootstrapRegistryInitializer 接口的实现类，该接口的源码如下所示。

```
/**
 * Callback interface that can be used to initialize a {@link BootstrapRegistry} before it
 * is used.
 *
 * @author Phillip Webb
 * @since 2.4.5
 * @see SpringApplication#addBootstrapRegistryInitializer(BootstrapRegistryInitializer)
 * @see BootstrapRegistry
 */
@FunctionalInterface
public interface BootstrapRegistryInitializer {

      /**
       * Initialize the given {@link BootstrapRegistry} with any required registrations.
       * @param registry the registry to initialize
       */
      void initialize(BootstrapRegistry registry);

}
```

BootstrapRegistryInitializer 是一个回调接口，主要用于在后面创建 BootstrapRegistry 实例时对它进行初始化回调操作，如下面的 DefaultBootstrapContext 实例的源码所示。

```
private DefaultBootstrapContext createBootstrapContext() {
    DefaultBootstrapContext bootstrapContext = new DefaultBootstrapContext();
    this.bootstrapRegistryInitializers.forEach((initializer) -> initializer.initialize
(bootstrapContext));
    return bootstrapContext;
}
```

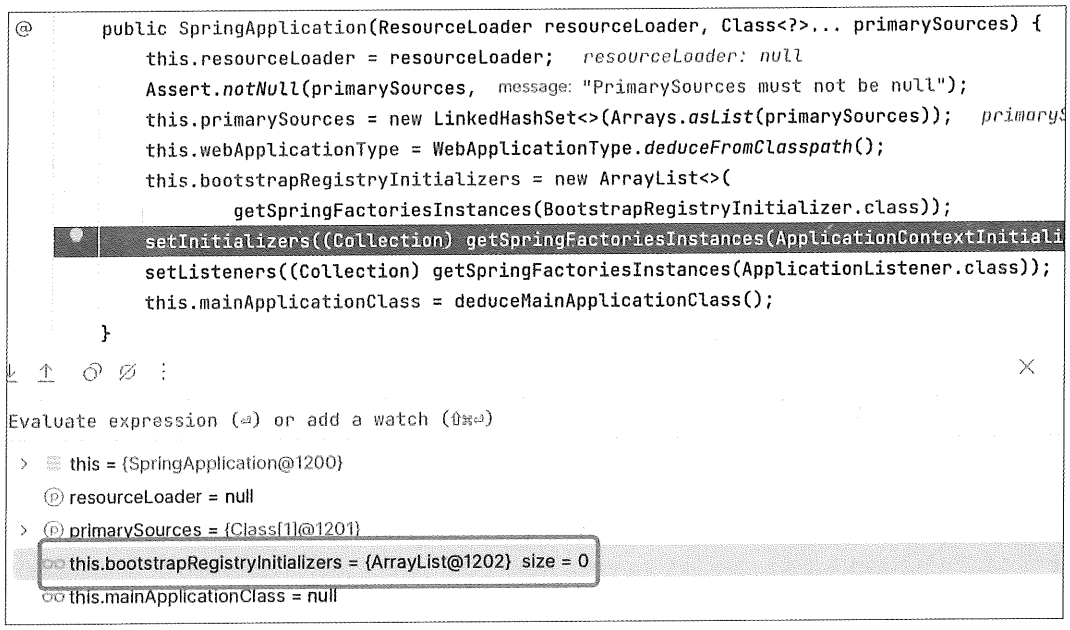
DefaultBootstrapContext 是 BootstrapContext 和 BootstrapRegistry 接口默认的实例，类结构所下图所示。



DefaultBootstrapContext 实现的两个重要接口：

- BootstrapContext：它是一个简单的引导上下文接口，在应用启动期间和环境后处理（Environment post-processing）期间可用，直到准备好 ApplicationContext 为止，对创建成本很高或者需要在 ApplicationContext 可用之前共享的单例对象提供延迟访问。
- BootstrapRegistry：它是一个简单的对象注册表接口，在应用启动期间和环境后处理（Environment post-processing）期间可用，直到准备好 ApplicationContext 为止，用于注册创建成本较高的实例，或者在 ApplicationContext 可用之前需要共享的实例。

虽然 createBootstrapContext 方法有初始化 BootstrapRegistry 的回调逻辑，但据笔者研究，从 Spring Boot 2.7 到 3.0.0，这个 BootstrapRegistryInitializer 接口目前还没有任何实现类，也没有在 spring.factories 文件中注册任何实现类，断点调试所下图所示。



这个可能是保留逻辑，有待后续的版本更新。

6. 设置应用上下文初始化器

```
setInitializers((Collection) getSpringFactoriesInstances(ApplicationContextInitializer.class));
```

应用上下文初始化器 ApplicationContextInitializer 接口的源码如下所示。

```
/**
 * Callback interface for initializing a Spring {@link ConfigurableApplicationContext}
 * prior to being {@linkplain ConfigurableApplicationContext#refresh() refreshed}.
 *
 * <p>Typically used within web applications that require some programmatic initialization
 * of the application context. For example, registering property sources or activating
 * profiles against the {@linkplain ConfigurableApplicationContext#getEnvironment()
 * context's environment}. See {@code ContextLoader} and {@code FrameworkServlet} support
 * for declaring a "contextInitializerClasses" context-param and init-param, respectively.
 * ...
 */
@FunctionalInterface
public interface ApplicationContextInitializer<C extends ConfigurableApplicationContext> {

      /**
       * Initialize the given application context.
       * @param applicationContext the application to configure
       */
      void initialize(C applicationContext);

}
```

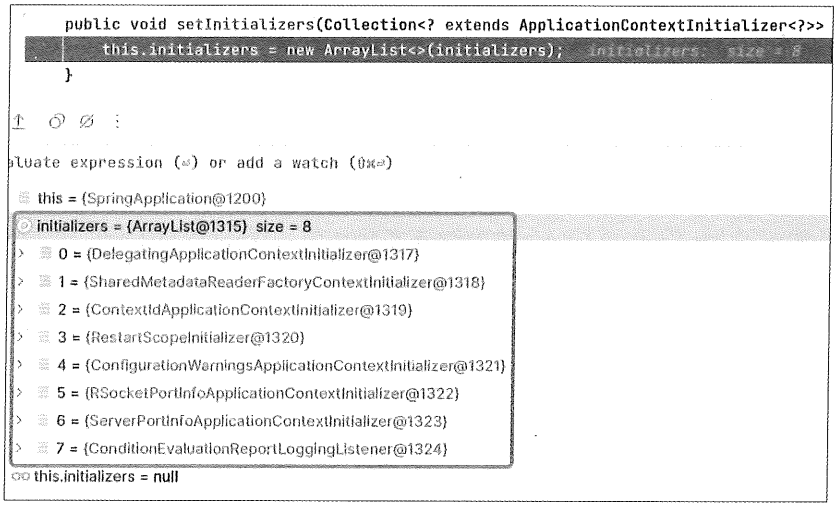
这是一个回调接口，主要用来在 Spring 应用上下文 ConfigurableApplicationContext 刷新之前进行一些初始化操作，如注册属性资源、激活 Profile 等，它在 Spring Boot 各个包中都注册了多个不同的实现，如 Spring Boot 基础包中的 spring.factories 文件：

```
# Application Context Initializers
org.springframework.context.ApplicationContextInitializer=\
org.springframework.boot.context.ConfigurationWarningsApplicationContextInitializer,\
org.springframework.boot.context.ContextIdApplicationContextInitializer,\
org.springframework.boot.context.config.DelegatingApplicationContextInitializer,\
org.springframework.boot.rsocket.context.RSocketPortInfoApplicationContextInitializer,\
org.springframework.boot.web.context.ServerPortInfoApplicationContextInitializer
```

如 Spring Boot 自动配置包中的 spring.factories 文件：

```
# Initializers
org.springframework.context.ApplicationContextInitializer=\
org.springframework.boot.autoconfigure.SharedMetadataReaderFactoryContextInitializer,\
org.springframework.boot.autoconfigure.logging.ConditionEvaluationReportLoggingListener
```

一共注册了 8个应用上下文初始化器，这一点从断点调试也可以看出来，如下图所示。



获取 spring.factories 配置文件的原理在 Spring Boot 自动配置章节中有详细讲解，这里不再赘述。

7. 设置应用事件监听器

```
setListeners((Collection) getSpringFactoriesInstances(ApplicationListener.class));
```

Spring 中的事件监听器是 ApplicationListener 接口，是所有 Spring 事件的抽象，如下面的源码所示。

```
@FunctionalInterface
public interface ApplicationListener<E extends ApplicationEvent> extends EventListener {

    /**
    * Handle an application event.
    * @param event the event to respond to
    */
    void onApplicationEvent(E event);


    /**
    * Create a new {@code ApplicationListener} for the given payload consumer.
    * @param consumer the event payload consumer
    * @param <T> the type of the event payload
    * @return a corresponding {@code ApplicationListener} instance
    * @since 5.3
    * @see PayloadApplicationEvent
    */
```

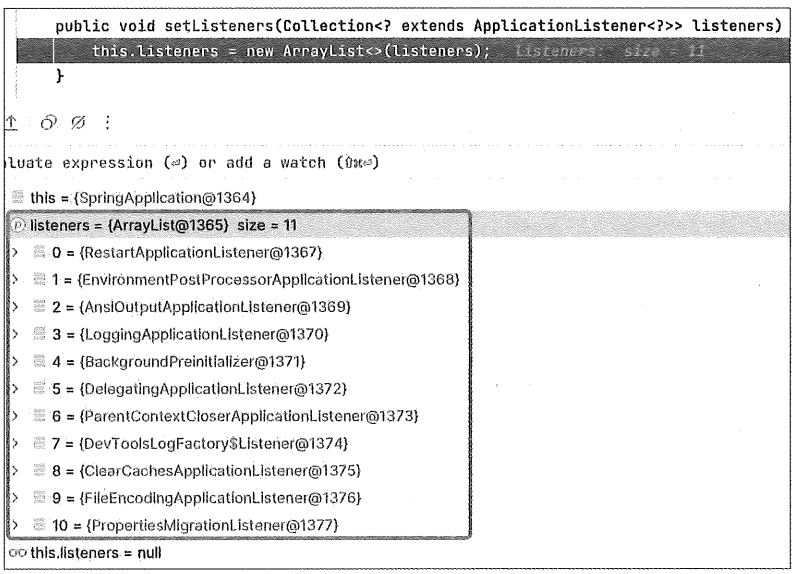
```
    static <T> ApplicationListener<PayloadApplicationEvent<T>> forPayload(Consumer<T>
consumer) {
        return event -> consumer.accept(event.getPayload());
    }

}
```

这个事件监听器 ApplicationListener 接口继承了 JDK 的 java.util.EventListener 接口，实现了观察者模式，它一般用来监听对应的事件，事件类型限定于 ApplicationEvent 及其子接口，这个接口同样继承了 JDK 的 java.util.EventObject 接口。

ApplicationListener 应用监听器同样在 spring.factories 文件中进行注册，如 Spring Boot 基础包中的 spring.factories 文件：

```
# Application Listeners
org.springframework.context.ApplicationListener=\
org.springframework.boot.ClearCachesApplicationListener,\
org.springframework.boot.builder.ParentContextCloserApplicationListener,\
org.springframework.boot.context.FileEncodingApplicationListener,\
org.springframework.boot.context.config.AnsiOutputApplicationListener,\
org.springframework.boot.context.config.DelegatingApplicationListener,\
org.springframework.boot.context.logging.LoggingApplicationListener,\
org.springframework.boot.env.EnvironmentPostProcessorApplicationListener
```

除此之外，在 Spring Boot 所有核心包中一共注册了 11 个事件监听器，这一点从断点调试也可以看出来，如下图所示。



每个监听器监听的事件和启动阶段可能不一样，这一点在 4.6 节中介绍过了，这里不再赘述，但是在后续的启动过程分析中可以看到监听的事件是从哪个启动阶段发出来的。

8. 推断并设置应用启动类

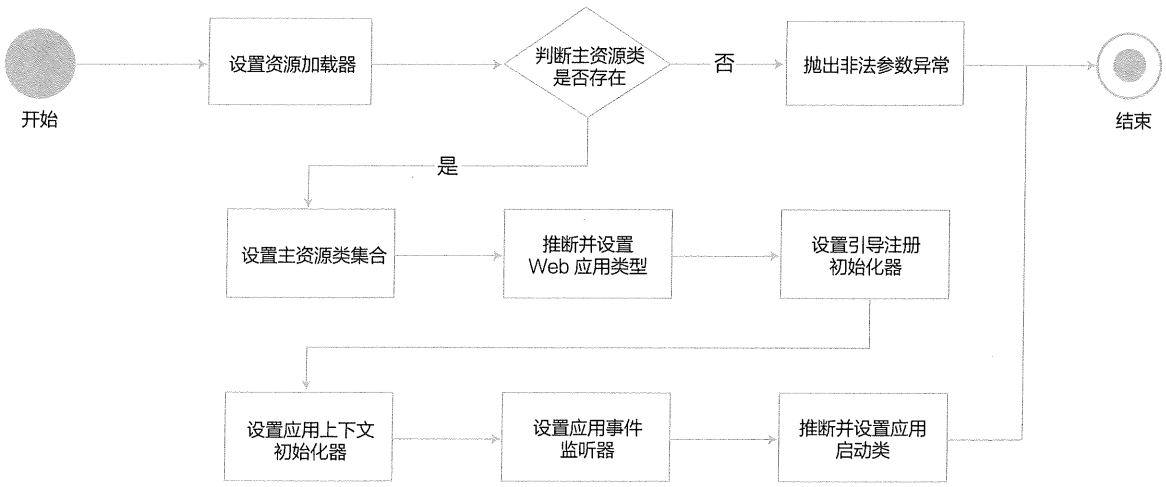
```
this.mainApplicationClass = deduceMainApplicationClass();
```

deduceMainApplicationClass 方法的源码如下：

```
private Class<?> deduceMainApplicationClass() {
    return StackWalker.getInstance(StackWalker.Option.RETAIN_CLASS_REFERENCE).walk
(this::findMainClass)
            .orElse(null);
}

private Optional<Class<?>> findMainClass(Stream<StackFrame> stack) {
    return stack.filter((frame) -> Objects.equals(frame.getMethodName(), "main"))
.findFirst()
            .map(StackWalker.StackFrame::getDeclaringClass);
}
```

推断应用启动类的方式有点特别，也很新颖，和 Spring Boot 2.7 中的实现方式完全不同，Spring Boot 3.0.0 使用了最新的 Java 9+ 中的 StackWalker 栈帧跟踪技术，先创建一个包含 Class 对象的 StackWalker 实例，然后筛选出第一个包含 main 方法的 Class 类。

最后总结一下 SpringApplication 实例化的大概流程，如下图所示。



4.8.2 启动流程

前面介绍了SpringApplication构造器的实例化流程，它只是初始化一些应用的必要信息，下面进入真正的应用启动流程，Spring Boot 3.0.0和Spring Boot 2.7又略有不同，如下面最新的Spring Boot 3.0.0的SpringApplication#run方法的源码所示。

```
public ConfigurableApplicationContext run(String... args) {
    // 1
    long startTime = System.nanoTime();

    // 2
    DefaultBootstrapContext bootstrapContext = createBootstrapContext();
    ConfigurableApplicationContext context = null;

    // 3
    configureHeadlessProperty();

    // 4
    SpringApplicationRunListeners listeners = getRunListeners(args);
    listeners.starting(bootstrapContext, this.mainApplicationClass);

    try {
        // 5
        ApplicationArguments applicationArguments = new DefaultApplicationArguments(args);

        // 6
        ConfigurableEnvironment environment = prepareEnvironment(listeners,
bootstrapContext, applicationArguments);

        // 7
        Banner printedBanner = printBanner(environment);

        // 8
        context = createApplicationContext();
        context.setApplicationStartup(this.applicationStartup);

        // 9
        prepareContext(bootstrapContext, context, environment, listeners,
applicationArguments, printedBanner);

        // 10
        refreshContext(context);
```

```
        // 11
        afterRefresh(context, applicationArguments);

        // 12
        Duration timeTakenToStartup = Duration.ofNanos(System.nanoTime() - startTime);
        if (this.logStartupInfo) {
            new StartupInfoLogger(this.mainApplicationClass).logStarted
(getApplicationLog(), timeTakenToStartup);
        }
        listeners.started(context, timeTakenToStartup);

        // 13
        callRunners(context, applicationArguments);
    }
    catch (Throwable ex) {
        if (ex instanceof AbandonedRunException) {
            throw ex;
        }
        handleRunFailure(context, ex, listeners);
        throw new IllegalStateException(ex);
    }
    try {
        // 14
        if (context.isRunning()) {
            Duration timeTakenToReady = Duration.ofNanos(System.nanoTime() - startTime);
            listeners.ready(context, timeTakenToReady);
        }
    }
    catch (Throwable ex) {
        if (ex instanceof AbandonedRunException) {
            throw ex;
        }
        handleRunFailure(context, ex, null);
        throw new IllegalStateException(ex);
    }
    return context;
}
```

1. 初始化启动开始时间

```
long startTime = System.nanoTime();
```

初始化一个开始时间，通过 System 类获取，单位为纳秒。

2. 初始化默认引导上下文

```
DefaultBootstrapContext bootstrapContext = createBootstrapContext();
ConfigurableApplicationContext context = null;
```

创建一个默认的引导上下文 DefaultBootstrapContext，源码如下：

```
private DefaultBootstrapContext createBootstrapContext() {
    DefaultBootstrapContext bootstrapContext = new DefaultBootstrapContext();
    this.bootstrapRegistryInitializers.forEach((initializer) -> initializer.initialize
(bootstrapContext));
    return bootstrapContext;
}
```

具体内容在前面实例化流程一节中介绍过了，这里不再赘述。

3. 设置 Headless 模式

```
configureHeadlessProperty();
```

设置 Headless 模式，该方法的源码如下：

```
private static final String SYSTEM_PROPERTY_JAVA_AWT_HEADLESS = "java.awt.headless";

private boolean headless = true;

private void configureHeadlessProperty() {
    System.setProperty(SYSTEM_PROPERTY_JAVA_AWT_HEADLESS,
            System.getProperty(SYSTEM_PROPERTY_JAVA_AWT_HEADLESS, Boolean.toString
(this.headless)));
}
```

这里的逻辑是，如果没有设置 java.awt.headless 参数的值，则将它默认设置为 true。

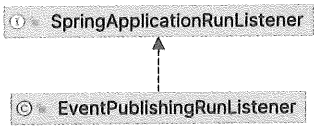
为什么要设置 java.awt.headless = true？

AWT 是 Java 生态中用于构建 GUI 应用的标准 API 接口，一般用于 Java 标准模式下。而 Headless 是 Java 中另一套配置模式，指的是没有显示屏、鼠标或者键盘时的系统模式，当设置 java.awt.headless = true 时，可以强制应用使用 Headless 版本的 AWT 实现类，从而避免缺失相应的设备而导出应用出错，后端服务器一般都没有这些设备，所以后端应用一般都要将这个参数设置为 true。

4. 初始化启动方法监听器

```
SpringApplicationRunListeners listeners = getRunListeners(args);
listeners.starting(bootstrapContext, this.mainApplicationClass);
```

从 spring.factories 文件中获取所有 SpringApplicationRunListener 类型的监听器，并封装成 SpringApplicationRunListeners 监听器包装类，SpringApplicationRunListener 监听器目前只有一个实现类，类结构如下图所示。



SpringApplicationRunListeners 包装类主要组合了以下信息：

- Log：日志对象。
- List<SpringApplicationRunListener>：SpringApplicationRunListener 监听器集合。
- ApplicationStartup：应用启动管理类，它能步骤化监视应用的启动阶段。

接下来调用 SpringApplicationRunListeners#starting 方法，如下面的源码所示。

```
class SpringApplicationRunListeners {

    private final Log log;

       private final List<SpringApplicationRunListener> listeners;

       private final ApplicationStartup applicationStartup;

    ...

    void starting(ConfigurableBootstrapContext bootstrapContext, Class<?>
mainApplicationClass) {
        doWithListeners("spring.boot.application.starting", (listener) -> listener.
starting (bootstrapContext),
                (step) -> {
                    if (mainApplicationClass != null) {
                        step.tag("mainApplicationClass", mainApplicationClass.getName());
                    }
```

```
                });
    }

    private void doWithListeners(String stepName, Consumer<SpringApplicationRunListener>
listenerAction,
            Consumer<StartupStep> stepAction) {
        StartupStep step = this.applicationStartup.start(stepName);
        this.listeners.forEach(listenerAction);
        if (stepAction != null) {
            stepAction.accept(step);
        }
        step.end();
    }

    ...

}
```

先标记启动步骤，然后执行 EventPublishingRunListener 监听器的 starting 方法，如下面的源码所示。

```
class EventPublishingRunListener implements SpringApplicationRunListener, Ordered {

    private final SpringApplication application;

    private final String[] args;

    private final SimpleApplicationEventMulticaster initialMulticaster;

    EventPublishingRunListener(SpringApplication application, String[] args) {
        this.application = application;
        this.args = args;
        this.initialMulticaster = new SimpleApplicationEventMulticaster();
    }

    ...

    @Override
    public void starting(ConfigurableBootstrapContext bootstrapContext) {
        multicastInitialEvent(new ApplicationStartingEvent(bootstrapContext,
this.application, this.args));
    }
```

```
    ...
}
```

如上所示，这里发布了 ApplicationStartingEvent 应用启动事件。

5. 初始化应用参数类

```
ApplicationArguments applicationArguments = new DefaultApplicationArguments(args);
```

先通过 main 方法提供的入口参数构建一个默认的 DefaultApplicationArguments 应用参数类，后续可以通过这个类来获取应用参数。

6. 准备 Spring 环境

```
ConfigurableEnvironment environment = prepareEnvironment(listeners, bootstrapContext,
applicationArguments);
```

准备 Spring 环境，prepareEnvironment 方法的源码如下：

```
private ConfigurableEnvironment prepareEnvironment(SpringApplicationRunListeners listeners,
        DefaultBootstrapContext bootstrapContext, ApplicationArguments applicationArgu-
ments) {
    // (1)
    // Create and configure the environment
    ConfigurableEnvironment environment = getOrCreateEnvironment();

    // (2)
    configureEnvironment(environment, applicationArguments.getSourceArgs());
    ConfigurationPropertySources.attach(environment);

    // (3)
    listeners.environmentPrepared(bootstrapContext, environment);
    DefaultPropertiesPropertySource.moveToEnd(environment);
    Assert.state(!environment.containsProperty("spring.main.environment-prefix"),
            "Environment prefix cannot be set via properties.");

    // (4)
    bindToSpringApplication(environment);
```

```
    // (5)
    if (!this.isCustomEnvironment) {
        EnvironmentConverter environmentConverter = new EnvironmentConverter
(getClassLoader());
        environment = environmentConverter.convertEnvironmentIfNecessary(environment,
deduceEnvironmentClass());
    }
    ConfigurationPropertySources.attach(environment);
    return environment;
}
```

这里主要是根据上面获得的 listeners、bootstrapContext、applicationArguments 再去准备 Spring Environment 环境，大概分为以下几个步骤：

（1）创建对应 Web类型的 Spring Environment 环境。

（2）把 ApplicationArguments 参数类及其他应用参数设置到 Spring Environment 环境中。

（3）调用监听器的 environmentPrepared 方法，发布 ApplicationEnvironmentPreparedEvent 应用环境已准备好事件。

（4）把 Spring Environment 环境绑定到 SpringApplication 类。

（5）非自定义的 Spring Environment 环境，如果有需要还会进行环境转换。

7. 打印启动图案

```
private Banner printBanner(ConfigurableEnvironment environment) {
    if (this.bannerMode == Banner.Mode.OFF) {
        return null;
    }
    ResourceLoader resourceLoader = (this.resourceLoader != null) ? this.resourceLoader
            : new DefaultResourceLoader(null);
    SpringApplicationBannerPrinter bannerPrinter = new SpringApplicationBannerPrinter
(resourceLoader, this.banner);
    if (this.bannerMode == Mode.LOG) {
        return bannerPrinter.print(environment, this.mainApplicationClass, logger);
    }
    return bannerPrinter.print(environment, this.mainApplicationClass, System.out);
}
```

如果应用开启了打印启动图案的功能，则创建一个默认的 DefaultResourceLoader 资源加载器，

用于加载类路径下的 Banner 图案文件，然后封装成一个 SpringApplicationBannerPrinter 图案打印类，判断不同的图案输出模式，最后打印 Spring Boot 启动图案。如果没有自定义启动图案，则使用的是 SpringBootBanner 实现类中的默认图案，如下所示。

```
  .   ____          _            __ _ _
 /\\ / ___'_ __ _ _(_)_ __  __ _ \ \ \ \
( ( )\___ | '_ | '_| | '_ \/ _` | \ \ \ \
 \\/  ___)| |_)| | | | | || (_| |  ) ) ) )
  '  |____| .__|_| |_|_| |_\__, | / / / /
 =========|_|==============|___/=/_/_/_/
 :: Spring Boot ::                (v3.0.0)
```

8. 创建 Spring 上下文

```
context = createApplicationContext();
context.setApplicationStartup(this.applicationStartup);
```

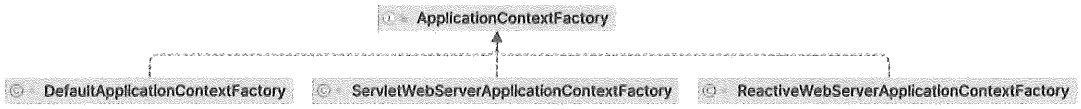
先创建一个对应 Web类型的 Spring 上下文，然后创建一个默认的 ApplicationStartup 应用启动管理类并设置到该上下文对象中。Spring 上下文是通过 ApplicationContextFactory 工厂创建的，如下面的 createApplicationContext 方法的源码所示。

```
protected ConfigurableApplicationContext createApplicationContext() {
    return this.applicationContextFactory.create(this.webApplicationType);
}
```

ApplicationContextFactory 上下文工厂有三种实现类：

- DefaultApplicationContextFactory：默认的应用上下文工厂类。
- ServletWebServerApplicationContextFactory：Servlet 的应用上下文工厂类。
- ReactiveWebServerApplicationContextFactory：Reactive 的应用上下文工厂类。

类结构如下图所示。



Spring Boot默认为 DefaultApplicationContextFactory 工厂类，由它来驱动其他两个不同类型的上下文工厂类，具体见下面的源码分析：

```
public class SpringApplication {

    ...

    private ApplicationContextFactory applicationContextFactory = ApplicationContextFactory.DEFAULT;

    ...

}
```

DefaultApplicationContextFactory 创建上下文对象的核心源代码如下所示。

```
class DefaultApplicationContextFactory implements ApplicationContextFactory {

    ...

    @Override
    public ConfigurableApplicationContext create(WebApplicationType webApplicationType) {
        try {
            return getFromSpringFactories(webApplicationType, ApplicationContextFactory::
                    create, this::createDefaultApplicationContext);
        }
        catch (Exception ex) {
            throw new IllegalStateException("Unable create a default ApplicationContext
                    instance, "+"you may need a custom ApplicationContextFactory", ex);
        }
    }

    ...

    private <T> T getFromSpringFactories(WebApplicationType webApplicationType,
            BiFunction<ApplicationContextFactory, WebApplicationType, T> action,
Supplier<T> defaultResult) {
        for (ApplicationContextFactory candidate : SpringFactoriesLoader.loadFactories
(ApplicationContextFactory.class,
                getClass().getClassLoader())) {
            T result = action.apply(candidate, webApplicationType);
            if (result != null) {
                return result;
            }
```

```
        }
        return (defaultResult != null) ? defaultResult.get() : null;
    }

}
```

核心逻辑就是，DefaultApplicationContextFactory 工厂在创建上下文时会加载 Spring Boot 基础包的 spring.factories 文件中的 ApplicationContextFactory 注册的工厂实现类，具体如下所示。

```
# Application Context Factories
org.springframework.boot.ApplicationContextFactory=\
org.springframework.boot.web.reactive.context.ReactiveWebServerApplicationContextFactory,\
org.springframework.boot.web.servlet.context.ServletWebServerApplicationContextFactory
```

然后循环所有注册的 ApplicationContextFactory 工厂实现类，再通过 ApplicationContextFactory::create 方法引用一一判断当前 Web 应用类型是否成功创建上下文对象，如果创建成功则直接返回，否则返回默认的上下文对象。因为这里是 Servlet 应用，所以它会成功匹配 ServletWebServerApplicationContextFactory 上下文工厂，如该工厂的源码所示。

```
class ServletWebServerApplicationContextFactory implements ApplicationContextFactory {

    ...

    @Override
    public ConfigurableApplicationContext create(WebApplicationType webApplicationType) {
        return (webApplicationType != WebApplicationType.SERVLET) ? null : createContext();
    }

    private ConfigurableApplicationContext createContext() {
        if (!AotDetector.useGeneratedArtifacts()) {
            return new AnnotationConfigServletWebServerApplicationContext();
        }
        return new ServletWebServerApplicationContext();
    }

}
```

因为这里不涉及 AOT 技术，所以它最终会创建并返回一个 AnnotationConfigServletWebServerApplicationContext 上下文对象。

9. 准备 Spring上下文

```
prepareContext(bootstrapContext, context, environment, listeners, applicationArguments,
printedBanner);
```

到了准备 Spring 上下文阶段了，prepareContext 方法的源码如下所示。

```
private void prepareContext(DefaultBootstrapContext bootstrapContext,
ConfigurableApplicationContext context,
        ConfigurableEnvironment environment, SpringApplicationRunListeners listeners,
        ApplicationArguments applicationArguments, Banner printedBanner) {
    // (1)
    context.setEnvironment(environment);
    postProcessApplicationContext(context);
    addAotGeneratedInitializerIfNecessary(this.initializers);

    // (2)
    applyInitializers(context);

    // (3)
    listeners.contextPrepared(context);

    // (4)
    bootstrapContext.close(context);
    if (this.logStartupInfo) {
        logStartupInfo(context.getParent() == null);
        logStartupProfileInfo(context);
    }

    // (5)
    // Add boot specific singleton beans
    ConfigurableListableBeanFactory beanFactory = context.getBeanFactory();
    beanFactory.registerSingleton("springApplicationArguments", applicationArguments);
    if (printedBanner != null) {
        beanFactory.registerSingleton("springBootBanner", printedBanner);
    }
    if (beanFactory instanceof AbstractAutowireCapableBeanFactory autowireCapableBean-
Factory) {
        autowireCapableBeanFactory.setAllowCircularReferences(this.allowCircularReferences);
        if (beanFactory instanceof DefaultListableBeanFactory listableBeanFactory) {
```

```
            listableBeanFactory.setAllowBeanDefinitionOverriding (this.allowBeanDefinition
Overriding);
        }
    }

    // (6)
    if (this.lazyInitialization) {
        context.addBeanFactoryPostProcessor(new LazyInitializationBeanFactoryPostProcessor());
    }
    context.addBeanFactoryPostProcessor(new PropertySourceOrderingBeanFactoryPostProcessor
(context));

    // (7)
    if (!AotDetector.useGeneratedArtifacts()) {
        // Load the sources
        Set<Object> sources = getAllSources();
        Assert.notEmpty(sources, "Sources must not be empty");
        load(context, sources.toArray(new Object[0]));
    }

    // (8)
    listeners.contextLoaded(context);
}
```

这里的逻辑比较复杂，笔者简单概括一下。

（1）绑定 Spring环境到 Spring 应用上下文，以及相关后置处理。

（2）遍历所有 ApplicationContextInitializer 上下文初始化器，逐个初始化 Spring 上下文。

（3）调用 SpringApplicationRunListeners#contextPrepared 方法，发布 ApplicationContext-InitializedEvent 应用上下文初始化事件。

（4）调用 DefaultBootstrapContext#close 方法，发布 BootstrapContextClosedEvent 引导上下文关闭事件。

（5）获取 BeanFactory，先注册 springApplicationArguments、springBootBanner 注入单例对象，然后设置是否允许循环引用、是否允许覆盖 Bean 参数。

（6）为 Spring 上下文添加 BeanFactoryPostProcessor 相关实现类，此接口一般可用来扩展或者修改 Bean 的定义，比如这里添加了 Bean 懒加载处理器及 PropertySource 重排序处理器。

（7）如果是非 AOT 模式，则获取所有资源类并把 Beans 加载到上下文环境中。

（8）调用 SpringApplicationRunListeners#contextLoaded 方法，发布 ApplicationPreparedEvent 应用已准备好事件。

10. 刷新 Spring 上下文

```
refreshContext(context);
```

Spring 上下文准备好之后就进入上下文刷新阶段了，refreshContext 方法的源码如下所示。

```
public class SpringApplication {

    ...

    private boolean registerShutdownHook = true;

    static final SpringApplicationShutdownHook shutdownHook = new SpringApplicationShutdownHook();

    private void refreshContext(ConfigurableApplicationContext context) {
        if (this.registerShutdownHook) {
            shutdownHook.registerApplicationContext(context);
        }
        refresh(context);
    }

    protected void refresh(ConfigurableApplicationContext applicationContext) {
        applicationContext.refresh();
    }

    ...

}
```

SpringApplicationShutdownHook 是一个实现了 Runnable 接口的线程类，它是优雅关闭 Spring Boot 应用时的一个关闭钩子，这个关闭钩子在这里将 Spring 上下文进行了注册，默认为允许注册。

然后 refresh 方法再调用 ApplicationContext#refresh 方法对应用上下文进行刷新，Servlet 应用则进入 AnnotationConfigServletWebServerApplicationContext 父类 ServletWebServerApplicationContext#refresh 方法，如下面的源码所示。

```
public class ServletWebServerApplicationContext extends GenericWebApplicationContext
        implements ConfigurableWebServerApplicationContext {

    ...

    @Override
    public final void refresh() throws BeansException, IllegalStateException {
        try {
            super.refresh();
        }
        catch (RuntimeException ex) {
            WebServer webServer = this.webServer;
            if (webServer != null) {
                webServer.stop();
            }
            throw ex;
        }
    }

    @Override
    protected void onRefresh() {
        super.onRefresh();
        try {
            createWebServer();
        }
        catch (Throwable ex) {
            throw new ApplicationContextException("Unable to start web server", ex);
        }
    }

    ...

}
```

执行Spring上下文的刷新操作，这里会通过重写Spring中的AbstractApplicationContext抽象类的onRefresh方法来创建一个Web Server服务器，并发布ContextRefreshedEvent、WebServerInitializedEvent事件，如果创建成功后发生异常则立即停止运行Web Server服务器。

11. 刷新 Spring 上下文之后

```
afterRefresh(context, applicationArguments);
```

这是刷新 Spring 上下文之后的动作，该 afterRefresh 方法目前还没有任何实现逻辑，该方法的源码如下所示。

```
protected void afterRefresh(ConfigurableApplicationContext context, ApplicationArguments args) {
}
```

可以看到 afterRefresh 方法是 protected 类型，说明可以继承 SpringApplication 类来自定义实现这个逻辑。

12. 应用启动成功

```
Duration timeTakenToStartup = Duration.ofNanos(System.nanoTime() - startTime);
if (this.logStartupInfo) {
    new StartupInfoLogger(this.mainApplicationClass).logStarted(getApplicationLog(),
timeTakenToStartup);
}
listeners.started(context, timeTakenToStartup);
```

到此，应用已经启动成功，先计算出应用的启动总耗时，再输出应用启动日志，最后调用 SpringApplicationRunListeners#started 方法，发布 ApplicationStartedEvent 应用已成功启动事件。

13. 运行 Spring Boot Runner

```
callRunners(context, applicationArguments);
```

应用启动成功之后，再执行所有的 Spring Boot Runner 运行器，如该方法的源码所示。

```
private void callRunners(ApplicationContext context, ApplicationArguments args) {
    List<Object> runners = new ArrayList<>();
    runners.addAll(context.getBeansOfType(ApplicationRunner.class).values());
    runners.addAll(context.getBeansOfType(CommandLineRunner.class).values());
    AnnotationAwareOrderComparator.sort(runners);
    for (Object runner : new LinkedHashSet<>(runners)) {
        if (runner instanceof ApplicationRunner applicationRunner) {
            callRunner(applicationRunner, args);
        }
```

```
            if (runner instanceof CommandLineRunner commandLineRunner) {
                callRunner(commandLineRunner, args);
            }
        }
    }

    private void callRunner(ApplicationRunner runner, ApplicationArguments args) {
        try {
            (runner).run(args);
        }
        catch (Exception ex) {
            throw new IllegalStateException("Failed to execute ApplicationRunner", ex);
        }
    }

    private void callRunner(CommandLineRunner runner, ApplicationArguments args) {
        try {
            (runner).run(args.getSourceArgs());
        }
        catch (Exception ex) {
            throw new IllegalStateException("Failed to execute CommandLineRunner", ex);
        }
    }
```

先将所有的 ApplicationRunner 和 CommandLineRunner 实例添加到一个集合中，再排序并去重，最后遍历该集合并依次调用其 run 方法。

14. 应用已就绪

```
if (context.isRunning()) {
    Duration timeTakenToReady = Duration.ofNanos(System.nanoTime() - startTime);
    listeners.ready(context, timeTakenToReady);
}
```

检查 Spring 上下文是否正在运行中，如果是，则调用 SpringApplicationRunListeners#ready 方法，并发布以下两个事件：

- ApplicationReadyEvent：应用已就绪事件。
- AvailabilityChangeEvent：应用可用性变更事件，可用状态变更为 ReadinessState.ACCEPTING_TRAFFIC，即应用可以接收请求了。

15. 启动流程异常处理

```
catch (Throwable ex) {
    if (ex instanceof AbandonedRunException) {
        throw ex;
    }
    handleRunFailure(context, ex, listeners);
    throw new IllegalStateException(ex);
}
```

在执行上述启动流程中，会捕捉 Throwable 总异常，如下面的 handleRunFailure 方法的源码所示。

```
private void handleRunFailure(ConfigurableApplicationContext context, Throwable exception,
        SpringApplicationRunListeners listeners) {
    try {
        try {
            // (1)
            handleExitCode(context, exception);

            // (2)
            if (listeners != null) {
                listeners.failed(context, exception);
            }
        }
        // (3)
        finally {
            reportFailure(getExceptionReporters(context), exception);
            if (context != null) {
                context.close();
                shutdownHook.deregisterFailedApplicationContext(context);
            }
        }
    }
    catch (Exception ex) {
        logger.warn("Unable to close ApplicationContext", ex);
    }
    // (4)
    ReflectionUtils.rethrowRuntimeException(exception);
}
```

大概的失败处理逻辑如下：

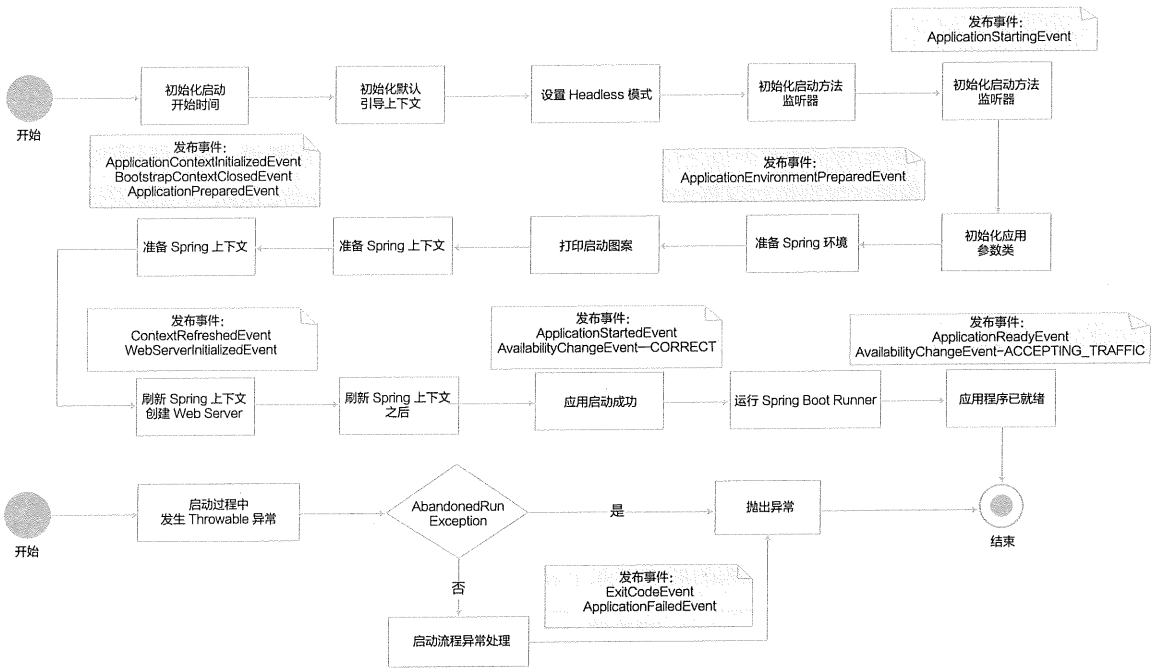
（1）处理 ExitCode 退出错误码，发布 ExitCodeEvent 错误码事件。

（2）调用 SpringApplicationRunListeners#failed 方法，发布 ApplicationFailedEvent 应用启动失败事件。

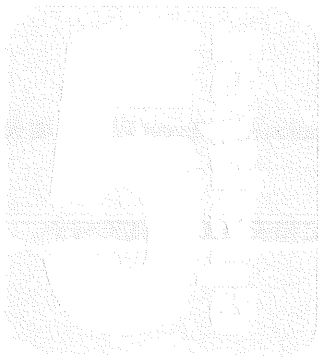
（3）finally 部分先记录失败异常，再关闭 Spring 上下文并取消与之相关的优雅关闭的钩子。

（4）将运行时异常抛出。

最后总结一下 Spring Boot 应用启动的大概流程，如下图所示。



Spring Boot 启动过程中主要创建了内嵌式的 Web Server 服务器，然后结合 Spring 环境集成了丰富的启动特性，底层还运用了大量的启动事件和监听器，以完成一些特定启动环节的逻辑。



第5章 Spring Boot日志管理

日志是每个Java应用框架必不可少的基本组件，也代表着应用的运行轨迹，输出日志可以帮助开发者有效追踪和排查应用运行过程中出现的疑难杂症。本章会介绍Spring Boot如何使用日志，包括支持的日志框架、日志格式、日志级别、日志分组、日志归档、自定义日志配置文件，以及切换其他日志框架及版本、输入彩色日志、日志关闭钩子等。

5.1 概述

Spring Boot框架内部所有日志的记录使用的是Apache开源的Commons Logging日志框架，它只是一种日志门面，提供了一套日志规范接口，底层的日志框架实现是可以自由切换的。

Spring Boot提供的几种日志框架的启动器如下表所示。

日志框架	启动器
Java Util Logging	spring-boot-starter-logging
Logback	spring-boot-starter-logging
Log4j2	spring-boot-starter-log4j2

其中spring-boot-starter-logging日志启动器是Spring Boot Starter启动器默认集成的，无须自行导入，这个可以从任何一个包括spring-boot-starter的启动器包中找到，比如spring-boot-starter-web依赖了spring-boot-starter，而spring-boot-starter就会依赖spring-boot-starter-logging：

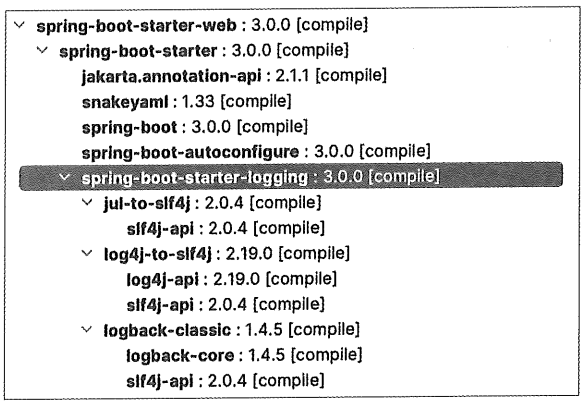
```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-logging</artifactId>
```

```
    <version>3.0.0</version>
    <scope>compile</scope>
  </dependency>
```

所以，任何一个 spring-boot-starter-* 启动器都会默认导入一个 spring-boot-starter-logging 启动器依赖，如果引入了其他官方的 Spring Boot Starter，则无须自行导入 spring-boot-starter-logging 启动器依赖。

Spring Boot 默认集成的是 Logback 日志框架，因为默认的 spring-boot-starter-logging 日志启动器使用的是 Logback 日志框架，并为 Logback 提供了支持 Java Util Logging、Commons Logging、Log4J、SLF4J 的桥接器以便能从这些日志门面中自由切换，因此项目中不管使用哪个日志门面，Logback 都能正常工作。

如下图所示，可以从 spring-boot-starter-web 依赖树中看到它包含了默认日志框架 Logback 及其他桥接器。



5.2 日志格式

Spring Boot 日志的格式如下：

```
2022-7-17T09:38:15.072+08:00  INFO 66672 --- [  restartedMain] o.s.b.w.embedded.tomcat.
TomcatWebServer : Tomcat started on port(s): 8443 (https) 8080 (http) with context path ''
```

日志主要包含以下信息：

- 日期时间：精确到毫秒，易于排序。
- 日志级别：具体见后面的章节。

- 应用进程 ID。
- 分隔符（---）：用来区分实际开始的日志。
- 线程名称。
- 日志名称：一般是指日志输出的当前类的缩写。
- 日志具体内容。

5.3 控制台日志

Spring Boot 默认从控制台输出日志，默认情况下，会记录 ERROR、WARN、INFO 级别的日志，如果要让应用输出更多日志，则可以在应用启动时使用 --debug 标志来启用调试模式，比如以下示例：

```
$ java –jar javastack.jar –-debug
```

其实还有更多开启应用调试模式的方法，已在第 3 章 Spring Boot Starter 与自动配置的自动配置报告一节中有全面的介绍。

需要注意的是：

启用应用的 Debug 调试模式，并不是让应用启用并输出所有的 DEBUG 级别的日志，而是让应用输出更多的框架核心日志，框架核心日志包括嵌入式容器、Spring Boot 等组件的更多的日志。

另外，还可以开启应用的 trace 追踪模式，日志要比 Debug 调试模式更详细，使用方法、注意事项和 Debug 调试模式是一样的，比如以下示例：

```
$ java -jar javastack.jar –trace
```

5.4 日志文件

Spring Boot 默认从控制台输出日志，而不会将日志写到日志文件中，这在实际项目中肯定是不可行的，在实际项目中不可能一直开着控制台，控制台日志随时会挂失，日志输出到日志文件可有利于归档并排查 N 天之前的日志。

将日志输出到日志文件，可以设置以下两个属性，如下表所示。

配置参数	说明	示例
logging.file.name	指定需要输出的日志文件名称	javastack.log

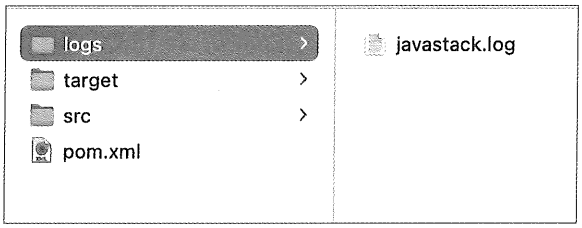
续表

配置参数	说明	示例
logging.file.path	指定需要输出的日志文件路径	/var/log 此时默认日志文件为 spring.log

Spring Boot 根据以上两个配置参数来输出日志文件，可以任用一个，也可以两个结合使用，可以是指定的路径，也可以是当前目录的相对路径，比如以下日志文件配置：

```
logging:
  file:
name: ./logs/javastack.log
```

应用启动之后就会在当前目录下自动创建 logs/javastack.log 目录及日志文件，如下图所示。



具体原理就是在Spring Boot主包的自动配置文件中注册一个LoggingApplicationListener监听器，然后监听 ApplicationEnvironmentPreparedEvent 事件：

```
private void onApplicationEnvironmentPreparedEvent(ApplicationEnvironmentPreparedEvent event) {
    SpringApplication springApplication = event.getSpringApplication();
    if (this.loggingSystem == null) {
        this.loggingSystem = LoggingSystem.get(springApplication.getClassLoader());
    }
    initialize(event.getEnvironment(), springApplication.getClassLoader());
}

protected void initialize(ConfigurableEnvironment environment, ClassLoader classLoader) {
    getLoggingSystemProperties(environment).apply();
    this.logFile = LogFile.get(environment);
    if (this.logFile != null) {
        this.logFile.applyToSystemProperties();
    }
    this.loggerGroups = new LoggerGroups(DEFAULT_GROUP_LOGGERS);
    initializeEarlyLoggingLevel(environment);
    initializeSystem(environment, this.loggingSystem, this.logFile);
    initializeFinalLoggingLevels(environment, this.loggingSystem);
```

```
    registerShutdownHookIfNecessary(environment, this.loggingSystem);
}
```

然后在 initialize 初始化方法中通过 LogFile#get 方法获取具体的日志文件：

```
public static LogFile get(PropertyResolver propertyResolver) {
    String file = propertyResolver.getProperty(FILE_NAME_PROPERTY);
    String path = propertyResolver.getProperty(FILE_PATH_PROPERTY);
    if (StringUtils.hasLength(file) || StringUtils.hasLength(path)) {
        return new LogFile(file, path);
    }
    return null;
}
```

根据前面的 Spring Boot 启动过程与扩展应用一章可以知道，这个 ApplicationEnvironment-PreparedEvent 事件是在 prepareEnvironment 方法准备环境阶段时发布的。

Spring Boot 在获取日志配置的时候为什么不是用 @ConfigurationProperties 注解绑定配置，而是手动获取配置呢？这是因为日志是在 ApplicationContext 创建之前初始化的，所以此时还无法获取环境中的配置。

5.5 日志级别

Spring Boot 支持的所有日志级别如下表所示。

日志级别	说明
OFF	关闭日志
TRACE	追踪
DEBUG	调试
INFO	信息
WARN	警告
ERROR	错误
FATAL	致命

Spring Boot 支持的最新的所有日志级别可以查看 LogLevel 这个类的源码：

```
public enum LogLevel {

    TRACE, DEBUG, INFO, WARN, ERROR, FATAL, OFF

}
```

Spring Boot 默认会输出 INFO 级别及以上的日志，另外，Logback 不支持 FATAL 日志级别，它会映射到 ERROR 级别上面。

日志级别的配置格式为：

```
logging.level.<logger-name>=<level>
```

根包的日志配置格式为：

```
logging.level.root=<level>
```

比如以下日志级别的配置：

```
logging:
  level:
    root: INFO
org.springframework: WARN
```

这里配置了根包的日志级别为 INFO，org.springframework 包的日志级别为 WARN。

除了在应用配置文件中配置，还可以通过环境变量配置日志级别，比如添加一个环境变量 LOGGING_LEVEL_ORG_SPRINGFRAMEWORK=DEBUG，它就会将 org.springframework 包日志级别调整为 DEBUG。

日志级别是在 LoggingApplicationListener 监听器的 initialize > ... > setLogLevels 初始化方法中设置的，如该方法的源码所示。

```
protected void setLogLevels(LoggingSystem system, ConfigurableEnvironment environment) {
    BiConsumer<String, LogLevel> customizer = getLogLevelConfigurer(system);
    Binder binder = Binder.get(environment);
    Map<String, LogLevel> levels = binder.bind(LOGGING_LEVEL, STRING_LOGLEVEL_MAP).
orElseGet(Collections::emptyMap);
    levels.forEach((name, level) -> configureLogLevel(name, level, customizer));
}
```

5.6 日志分组

上一节在配置对应包的日志级别的时候，需要一个个包地指定日志级别，如果要指定的包比较多，再为它们批量更换日志级别就比较麻烦，所以，Spring Boot 提供了日志分组功能，可以把相关的包配置成一个日志组，再针对日志组统一设置日志级别。

配置示例如下：

```
logging:
  group:
    tomcat: org.apache.catalina,org.apache.coyote,org.apache.tomcat
  level:
    tomcat: ERROR
```

这里配置了一个 Tomcat 组，组下面包含了三个包，然后把该日志分组设置为 ERROR 日志级别，即组下面的所有包都统一设置了，如果现在要更换这些包的日志级别，则只需要更换该组的日志级别，简单方便。

Spring Boot 也提供了几个可“开箱即用”的日志组，如下表所示。

日志组名称	包含的包
web	org.springframework.core.codec org.springframework.http org.springframework.web org.springframework.boot.actuate.endpoint.web org.springframework.boot.web.servlet.ServletContextInitializerBeans
sql	org.springframework.jdbc.core org.hibernate.SQL org.jooq.tools.LoggerListener

这些日志组可以直接配置日志级别，不需要再额外配置日志组，比如以下配置示例：

```
logging:
  level:
    web: INFO
```

这样配置后，Web 日志组下面的所有包都是 INFO 日志级别。

5.7 日志归档

Spring Boot 的日志在达到默认的 10MB 之后就会自动归档，如果使用的是默认的 Logback 日志框架，则可以直接在 application 配置文件中配置日志滚动规则，其他日志框架需要在其日志配置文件中配置（如 Log4j2）。

Logback 日志归档的配置参数对应的参数类为 LogbackLoggingSystemProperties 类，具体参数如下表所示。

日志滚动配置参数	说明
logging.logback.rollingpolicy.file-name-pattern	创建日志归档的文件名模式
logging.logback.rollingpolicy.clean-history-on-start	应用启动时是否进行日志归档清理
logging.logback.rollingpolicy.max-file-size	日志归档文件的最大文件容量，超过就进行归档（默认为 10MB）
logging.logback.rollingpolicy.total-size-cap	日志归档文件可占用的最大容量，超过会删除旧的归档日志文件
logging.logback.rollingpolicy.max-history	日志归档文件要保留的最长天数（默认为 7 天）

参考配置如下：

```
logging:
  file:
    name: ./logs/javastack.log
  logback:
    rollingpolicy:
      max-file-size: 1KB
```

这里我们指定文件日志超到 1KB 就进行归档，实际项目中请根据自己的需求进行配置。

应用重启后再来看一下日志目录，如下图所示。

可以看到应用生成了好几个日志归档文件，日志归档文件默认是按日期进行归档的，同一天的多次归档文件又会按索引进行归档，索引从 0 开始。

5.8 日志配置文件

5.8.1 概述

虽然 Spring Boot 可以通过参数配置的方式来定义日志的行为，但也仅限于一些常用的使用场景，如果涉及复杂的日志配置，则需要通过日志配置文件的方式来处理。

Spring Boot 默认加载的日志配置文件如下表所示。

日志框架	配置文件
Logback	logback-spring.xml logback-spring.groovy logback.xml logback.groovy
Log4j2	log4j2-spring.xml log4j2.xml
Java Util Logging	logging.properties

Spring Boot 建议使用 *-spring 格式的日志配置文件，比如 logback-spring.xml，而不是使用 logback.xml，因为标准的 logback.xml 日志配置文件的初始化太早了，会导致 Spring 无法完全控制日志的初始化，使用 *-spring 格式的日志配置文件的好处是可以使用 Spring 中的日志扩展功能。

前面的章节也提到了，因为日志的初始化较早，所以无法读取 Spring 环境中的配置参数，但是可以通过 System Property 系统参数的方式来配置，Spring Boot 提供了以下对应的配置参数，如下表所示。

Spring Environment	System Property
logging.exception-conversion-word	LOG_EXCEPTION_CONVERSION_WORD
logging.file.name	LOG_FILE
logging.file.path	LOG_PATH
logging.pattern.console	CONSOLE_LOG_PATTERN
logging.pattern.dateformat	LOG_DATEFORMAT_PATTERN
logging.charset.console	CONSOLE_LOG_CHARSET
logging.pattern.file	FILE_LOG_PATTERN
logging.charset.file	FILE_LOG_CHARSET
logging.pattern.level	LOG_LEVEL_PATTERN
PID	PID

另外，日志框架自身的配置也有对应的系统参数，比如 Logback 的 logging.logback.rollingpolicy.file-name-pattern 配置参数对应 LOGBACK_ROLLINGPOLICY_FILE_NAME_PATTERN 参数，等等。

5.8.2 日志配置模板

Spring Boot 提供了以下几个 Logback 默认的内置配置模板：